YI-HOT · GITHUB MAIN · 2026-11-11

YiHot README · 项目展示与快速入门

由 GitHub 仓库原始 Markdown 导出；仓库版本见 CochraneK/Yi-hot。

目录 / Contents

为什么做 Yi-Hot? 2
核心能力 2
信源图谱：从来源到有效内容 3
救助儿童会方向：13 个使命主题 4
Yi-Hot 与 AIHOT：哪些是继承，哪些是定制? 5
快速开始 5
架构与数据边界 6
接下来想做的 6
同一套框架长出的其他站 6
文档导航 8
当前状态、反馈与贡献 9
许可与致谢 9



Yi-Hot · 公益动态

Save the Children

救助儿童会方向的儿童公益主题适配 · 机构标识归原权利人所有；本站独立维护，展示不代表官方合作或背书。

让值得关注的公益信息，更容易被发现、理解与追踪。

面向公益从业者、研究者与关注公共议题的读者的公益资讯聚合与 AI 辅助整理平台。

浏览线上站点 · 自行部署 · 全景使用与交接指南 · 最新数据核验清单 · 完整维护手册 · 项目交接

MIT License | TypeScript and Node.js 24 | PostgreSQL 17 | Self-hosted with Docker Compose

> [!NOTE] 本项目是以 AIHOT 为技术基础的**独立公益领域定制版本**，不是 AIHOT 或救助儿童会的官方发行版或代表其立场的产品。本文的可视化为功能与架构**示意图**，并非真实线上页面截图。原始新闻版权归发布方所有。

为什么做 Yi-Hot?

公益信息散落在政府公告、社会组织网站、行业媒体、微信公众号和邮件简报里。逐个阅读费时间，直接订阅又容易被重复信息淹没。

Yi-Hot 想减少信息搜寻和整理的重复劳动：持续发现公开信息，区分原始发布日期与抓取时间，通过可追溯的内容分析形成动态、主题和周期性总报，帮助读者更快回到可靠的一手来源。

核心能力

发现	理解	阅读	持续使用
多渠道信源：网站、RSS、邮件简报，以及**网页搜索 API**（按主题词主动发现公益事件）；公众号与微博通道已暂停	**内容处理**：去重、相关性判断、摘要、翻译与结构化标签	**动态与热点**：按原文日期查看，保留原始来源链接；动态页支持「只看未读 / 只看精选」筛选	**周报与月报**：按自然周期汇总行业信息
信源治理：候选发现、预览、隔离启用、健康检查	**公益语境适配**：政策、项目、机构、儿童与其他公益领域	**主题与机构页**：按议题交叉阅读，不只依靠关键词；另有**监管记录**，民政部的处罚与异常名录每日同步	**邮件订阅**：双重确认、按期投递、可退订
历史资产留存：NewsMail 与 charity-intelligence 信源线索	**人工可复核**：留存判断链、来源和模型用量	**分享海报**：每条都能生成带二维码的图，发群里或存下来	**后台运维**：异常诊断、费用管理、反馈与审计；对外另有 **RSS / API / MCP / Agent Skill** 四种读取入口

从信源到总报：一个完整、可审查的流程

不是简单抓取新闻，而是整理信息、保留出处，再交给读者判断。



输出边界：摘要及原文链接为主 · 全文展示须有授权 · 候选信源不默认启用

按需要选入口： 动态 看最新信息（顶部可切「只看未读 / 只看精选」）；主题 按领域阅读；热点 看近期最受关注的事件；周报 / 月报 回顾周期内的重要进展。

手机与电脑内容一致：手机端底部固定「总报 / 动态 / 热点 / 主题 / 更多」，电脑端是左侧常驻目录栏；收藏、Agent、更新这些在手机上要进「更多」，在电脑上直接摆在侧栏。订阅入口两端都常驻。监管页目前只有手机入口和直链，电脑侧栏还没接上，见下文「接下来想做的」。

> 热点榜的「关注度」同时参考正规信源（政府、机构、媒体、邮件）与网页搜索信号：同一事件被越多**独立来源**提及、且越近发生，热度越高；只有网页搜索信号、没有权威来源背书的事件不会单独上榜。

信源图谱：从来源到有效内容

信源多，不代表有效内容多。 下图使用 **2026-10-25 13:09（北京时间）的生产快照**，区分登记、启用、实际入池与主力源，并按采集入口展示有效内容贡献；**不是今天的实时监控**。



展开：发布主体、技术入口、运营状态如何分层?



看完整信源分层说明 • 逐源台账、采集链接与失败复盘 • 候选信源治理原则

救助儿童会方向：13 个使命主题

Yi-Hot 针对儿童公益议题进行了领域定制：主题词表、别名映射、前台主题筛选，以及存量内容的标签回填机制。项目维护者另开展过救助儿童会微信公众号历史文章的采集与数据分析：**原始语料（公众号全文）不入库、不公开**，而由它汇总出的《公众号内容分析》已作为只读页面发布在 关于 → 看看这十二年写过什么，覆盖 2014–2026 年 1,992 篇推文，主题分布、年度特征词与共现关系都可逐项查看。它的入口在关于页最下方「关于我们」那一屏。



- 主题既用于发现信息，也用于前端导航、标签与历史文章回填；具体规则见 主题轴说明。
- **内容形式**（政策、研究、项目等）与**公益主题**（儿童保护、营养健康等）是不同维度，可以同时出现。
- “其他领域”是兜底标签，不应与明确的使命领域同时出现。标签不是科学测量结论，实际质量需要持续抽样复核。

Yi-Hot 与 AIHOT：哪些是继承，哪些是定制？

层面	说明
继承 AIHOT 的核心底座	信息处理引擎、Web/API、任务调度、数据库模型及大量基础组件；保留原作者署名和许可
Yi-Hot 的公益定制	公益信源、领域词表、儿童议题主题轴、相关性与精选规则、公益内容文案与主题入口
Yi-Hot 的运营扩展	WeRSS 公众号接入（2026-10-25 起暂停）、QQ 邮件简报、总报邮件订阅、后台候选信源治理、云端部署与交接
跨项目资产	NewsMail 与 charity-intelligence 的历史线索以候选形式保留；**不等于**已批准或全部接入采集
上游更新	`main` 是现用版本；`vendor/aihot-main` 与 `rebuild/aihot-aligned` 用于隔离追踪、验证更新，**不会自动替换生产**

详情：上游升级和重构方案 • 信息源治理原则。

快速开始

前置条件： Node.js 24.11+、Docker / Docker Compose、npm，以及可供测试的 PostgreSQL 环境。默认配置用于本机体验；真实采集、模型、公众号和邮件功能需要分别配置对应凭据与开关。

```
git clone https://github.com/CochraneK/Yi-hot.git
cd Yi-hot
npm ci
cp .env.example .env
node scripts/init-env.ts
docker compose up -d --build
```

本地通常访问 **http://localhost:3000**；后台通常为 **http://localhost:3000/admin**。不要将生产密钥复制到测试环境。详见 完整维护手册 与 部署说明。

> [!IMPORTANT] 仓库已公开，可直接克隆。第三方代码、机构标识与新闻内容的授权边界见 NOTICE 与 第三方归属说明——机构标识的使用不代表合作或背书。

架构与数据边界

```
flowchart LR
  A["政府网站 / 公益机构 / RSS / 公众号 / 邮件"] --> B["采集与来源治理"]
  B --> C["规范化、发布时间、去重"]
  C --> D["AI 相关性判定、摘要与标签"]
  D --> E["统一发布层"]
  E --> F["动态 / 热点 / 主题"]
  E --> G["周报 / 月报"]
  G --> H["订阅确认、邮件投递与退订"]
  B -. "候选需人工审核" .-> I["历史信源线索库"]
```

守住三条边界： 只在允许的范围内采集；默认仅显示摘要和原文链接，不擅自转载全文；`industry/sources.json` 是 seed，**生产信源以 PostgreSQL 中的运行时配置为准**。项目不保证所有来源全天候可访问，也不保证 AI 摘要永远准确，重要事实请以原文为准。

接下来想做的

方向	重点
跨项目整合	NewsMail · charity-intelligence · 公益信息专家团
信息质量	公众号信源 · 人工校准
使用体验	监管入口 · 问答与朗读探索
持续维护	AIHOT 上游同步 · 数据保全

完整 TODO · 跨项目整合路线

同一套框架长出的其他站

AIHOT 是开源的，社区里有人把它换成自己的信源和判断标准，长成了各自的行业站。下面这些在 **2026-10-30 逐个打开确认在线；都不是本站运营**，列出来不构成推荐或背书。之所以列，是因为它们说明了一件事：同一套引擎换掉领域和标准，就能长成完全不同的样子——这也是公益版敢这么做的前提。

站点	方向	与本项目的关系
AIHOT	AI 行业热点	上游原站

站点	方向	与本项目的关系
PsyHOT	心理学与精神医学，学界与业界双视角	**姊妹项目**，本站的 Agent Skill 通道与付费采集写法来自它
大厂观察	互联网大厂动态与职场信息	同框架，接在自己已有的网站上
RoboHot	机器人产业，面向研发者与准备入行的人	同框架
MediaStack	音视频技术情报	同框架
FilmTechNews	影视技术日报	同框架
青砚	环保与双碳，另附一套公众号免费采集方案	同框架
拾玖跨境资讯	跨境电商，按平台政策与运营重建了主题体系	同框架
品牌AI外刊	海外品牌营销与 AI 变化，先写准入规则再筛候选	同框架
营销通鉴 · 平台动态	营销平台的规则变化与新功能早报	同框架
AI4Math Radar	AI for Mathematics 研究动态	同框架
聚身之家 AIMAN.World	机器人产业数据库与图谱，另做 Agent 接入	形态相近，独立实现
VulnHOT	漏洞情报	形态相近，独立实现
URHOT	AI 与用户研究，零服务器的静态方案	形态相近，用 Python 重写了同一条管线
AI 快讯	AI 新闻与日报	形态相近，独立实现
NewsIQ	新闻事件聚合	形态相近，独立实现
xabcnews	AI 新闻	形态相近，独立实现

各自的做法、成本对比和我们采纳/不采纳的理由，见 同类项目调研；上游讨论区 **21 条帖子的全量索引**（谁解决了什么、哪些问题被反复问）在同文档的 2.5 节。上面有站点打不开了，欢迎提 Issue 告知。

把 21 条帖子读完，收获可以压成三句：

- **被重复解决最多的不是功能，是「适配」**——政府站没有 RSS、日期写法没见过、Node 握手协商不上、公众号通道太贵。解法都躺在各人自己的仓库里，没有地方搬。青砚那套 130+ 号的免费公众号管道、URHOT 的金标工具链，此前只有作者自己知道。
- **成本是个人站的共同天花板**，问法几乎一模一样（「一天几刀怎么办」），却没有一份能照着做的答案。
- **「独立实现」比 fork 多，是因为 fork 之后不知道怎么跟上游**——2,164 个 fork 里只有 130 个有自有提交，其中 55 个落后上游 100 多个提交。改得越深越不敢跟，最后自己维护一条死分支。

共养与共创。 这些站共用一套引擎，但重复的劳动和每月的账单，目前都还由各自的作者独自承担：上游有 2,125 个 fork，至少 5 个站把同一条管线独立重写了一遍；每个站也都在各自解决「这个政府站没有 RSS」「这个日期写法没见过」。这件事有解，而且不需要谁先捐钱——先登记谁在做什么，再把适配器作为数据回贡而不是代码 fork，最后才是每人每月一元的订阅式共养（定性是订阅，不是募捐）。完整推演、成本量级、以及「跨站

汇总为什么只能做索引不能混池」，见 共创与共养。整理出的社区索引与三个协作提议（领域注册表、适配器以数据回贡、跨站索引）已发到上游 Discussion #284。

文档导航

我要做什么	从这里开始
只想了解产品	线上站点
第一次接手维护	完整维护手册（前三分之一是「接手人最常问的 14 个问题」人话版）
换机器 / 换 AI 冷启动	PROJECT_CONTEXT（必读：拓扑、口径、凭据、接手清单）
看上一轮会话与决策沉淀	会话归档 2026-10-24
下一阶段做什么 / 哪些别再提	想法台账
参与共创 / 共养，或评估这套模型	共创与共养
新增公益信源	信源配置
看信源现状与失败复盘	信源全名单
试用候选信源	候选信源试用版
配置邮件简报	邮件采集部署
邮件订阅与投递	公益总报邮件投递
理解系统如何运作	技术架构
评分与筛选口径	精选与校准
看钱花在哪 / 压成本	成本口径与护栏
队列积压与容量缺口	队列可持续
同步和重新部署	同步工具
域名、备案与海外主机	决策记录
跟踪 AIHOT 上游	升级与重构方案
运维经验与踩坑索引	OPS 笔记

> 完整索引（历代交接单、产品更新记录、专题设计文档共 40+ 篇）见 MAINTENANCE.md 第二十一节・专题文档索引。

当前状态、反馈与贡献

- **部署状态与代码状态需要分别核验。** GitHub 更新不代表腾讯云已自动部署；上线后须进行文件哈希比对和实际运行检查。
- **`main` 的 Check 已全绿**（截至 2026-10-29 的一次运行，查看运行记录）。注意：涉及数据库的后端测试须在服务器容器内的 `aihot_ci` 库运行，与本地和 CI 环境都不同；改完后端请按 `.github/workflows/check.yml` 逐步复现，别只看绿勾。
- 邮件订阅与反馈涉及个人信息。公开隐私页面的运营主体与联系信息等模板字段**尚待实际维护者完善**；在正式对外推广前需要核实。
- 欢迎通过 Issues 反馈失效信源、误判、来源归属、界面可用性与安全问题（需有仓库访问权限）。不要在公开 Issue 提交密码、个人隐私或未获授权的全文数据。

许可与致谢

本项目基于 **KKKKhazix/AIHOT** 的 MIT 许可代码定制。感谢 AIHOT 的原始作者和开源贡献者。本仓库保留原始 LICENSE、NOTICE 和 第三方归属说明。AIHOT 的商标、名称及 Logo 不因代码许可而自动获得授权；救助儿童会及新闻来源的名称、标识和内容亦各有权利归属。

Agent 接入（`/api/v1/agent` 的 Markdown 回答、外部资料围栏）的**骨架来自 AIHOT**；同框架的姊妹项目 **PsyHOT**（MIT，心理学方向）在这层骨架上做了减法（去掉 MCP 层），并加了站内可分发的 Agent Skill 包；付费采集通道（EveryInfra）的适配写法也是 PsyHOT 的。后两项 AIHOT 从未有过，已按公益场景重写。逐文件清单（含核对日期与行数）见 THIRD_PARTY_NOTICES.md，同类项目的写法取舍见 docs/RELATED_PROJECTS.md。

Yi-Hot • 少一些重复检索，多一些真正有用的信息。 | 完整维护手册

YI-HOT · GITHUB MAIN · 2026-11-11

YiHot 维护手册 · Living Handbook

由 GitHub 仓库原始 Markdown 导出；仓库版本见 CochraneK/Yi-hot。

目录 / Contents

接手人最常问的 14 个问题（人话版） 3
这个站现在怎么工作 7
新维护者第一次接手 7
日常维护只做这些 8
三种常见情况 8
不要直接做的事 9
备份、凭据与换电脑 9
给技术维护者的入口 10
来源、版权与 AIHOT 归属 11
技术概览 11
一、完整运行链路 12
二、后台怎么用 12
三、后台密码与凭据 14
四、模型 API 从零说明 15
五、信息源结构与新增来源 16
六、分类、主题和标签 27
七、日期和重复内容 28
八、公益总报和邮件订阅 28
热度（热点榜）算法与网页搜索源的影响 29
「只看当地」（按访客 IP 城市过滤）设计说明（规划中，未实现） 30
九、公开 API、RSS 和 MCP 30
十、服务器容量基线 31
十一、巡检和故障处理 35
十二、本地开发、测试、同步和部署 37
十三、备份、换电脑和交接 39
十四、代码和行业配置地图 40
十五、后台所有设置逐项说明 41
十六、增加信息源的完整操作手册 44
十七、未来同步 AIHOT 上游改动 55
十八、迁移到 NAS 或另一台服务器 56
十九、常见需求该改哪里 58
二十、必须守住的安全边界 59
二十一、专题文档索引 59
二十二、持续维护规则（Living Handbook） 60
2026-10-18 本次维护变更 62
二十三、儿童权益国际信源与长期覆盖缺口（2026-10-21） 62

二十四、DeepSeek V4.1 Flash 默认模型安全切换（2026-10-21）.....62
二十五、发布时间精度：为什么「文章有，时间没有」（2026-10-21）.....63
二十六、聚合平台（今日热榜 tophub）实测与「栏目/词表」定词法（2026-10-21）.....65
二十七、网页搜索型信源（kind=search）与费用核算口径修正（2026-10-22）.....66
二十八、站点「更新」页的维护：写什么、怎么生效（2026-10-22）.....68
二十九、卡片落位改按收录时间、高峰由全停改限速、预筛补未成年人与妇女条款（2026-10-22 下午）.....71
三十、两类「怎么办都修不好」的问题：读不到的页面、重复的卡片（2026-10-22 下午）.....73
三十一、邮件简报把「往期年刊」当成了本期正文（2026-10-22 下午）.....74
三十二、2026-10-24 会话归档与最新信源图谱.....75
三十三、2026-11-02 卡片改按原文发布日归位，搜索源改每日.....76

Yi-Hot 公益动态

> **这是 Yi-Hot 的持续维护手册（Living Handbook）。** 面向日常维护者、开发者以及接手的 AI Agent；简洁项目展示见 README。完整正文保留了之前的详尽操作手册，后续功能、部署、故障与研究成果按本文末尾的「二十二、持续维护规则」增补，不以重写 README 代替更新这里。
>
> **阅读提示（2026-10-18）**：下文的“当前信源数量”、模型价格、运营状态、服务器清单和测试结论有些是特定日期的历史快照，并不代表此刻的生产状态。生产信源以 PostgreSQL `sources` 为准；代码和测试以 GitHub `main` 为准；腾讯云部署状态要另行核验，不能从 GitHub 提交推断。尤其是 2026-10-18 后 Docker CI 的种子数量校验已改为从 `industry/sources.json` 读取，而不是固定的 20 条。



Save the Children

Yi-Hot 是一个公益行业动态聚合站。它定时读取政府部门、公益机构、媒体、微信公众号和邮件简报，生成中文标题与摘要，保留原文链接，并按周、按月生成「总报」。

线上站点：https://49.233.72.222/yihot/

管理后台：https://49.233.72.222/yihot/admin

> **交接给新维护者时，先读本页即可。** 日常工作通常只需要打开后台查看运行状态和信源，不需要理解模型、提示词或代码。需要技术处理时，再按文末的文档入口操作。

> **本文件分两半**：上面「接手人最常问（人话版）」给**不懂技术、要接手日常使用的人**看；下面从「完整维护手册」开始是**技术 / AI 维护参考**，普通接手人看不懂可直接跳过，留给以后接手的技术人或 AI Agent。
>
> 两个入口先记牢：
>
> - 网站（读者看）：https://49.233.72.222/yihot/

> • 管理后台：https://49.233.72.222/yihot/admin（管理员密码由原维护者单独交付，**不要写进任何文档或聊天记录**）

接手人最常问的 14 个问题（人话版）

1. 这站是干嘛的，怎么用起来？

它是一个**公益资讯聚合站**：自动从政府网站、公益机构官网、微信公众号、邮件简报、以及网页搜索里收集公益相关信息，用 AI 做去重、摘要、分类，然后按「动态（最新）／热点（最受关注）／主题（按议题）」三种方式呈现。

- 日常阅读：打开网站 →「动态」看最新，「热点」看近期最受关注，「主题」按领域翻。
- 手机底部有五个按钮：**总报 / 动态 / 热点 / 主题 / 更多**；电脑左侧是目录栏。
- 「动态」页顶部有两个开关：**只看未读**（只看你还没点开过的）、**只看精选**（只看被 AI 评为值得重点关注的）。两个可以同时开。

2. 费用成本大概多少，会不会突然欠一大笔？

目前每月大约 **¥43–72**（不含可选备份和域名）：

- **服务器**：腾讯云 2核2GB，优惠价 **¥99/年**（约 ¥8.25/月）；注意这是促销价，续费可能涨到约 ¥510/年。
- **AI 模型调用（DeepSeek）**：约 **¥35–63/月**，按实际用量浮动。
- **公众号采集（WeRSS）**：开源软件，不另收费，但占服务器资源。
- **第三方付费采集接口（EveryInfra / 极致了）**：当前**未启用、预算归零**，一分钱都不会花。
- 真正会失控的只有「AI 模型调用」，但项目已设了**高峰让路**（贵的时候不分析）和**余额托底**（没钱了全站停，不会透支）。每月按「每月成本核对清单」核一次即可。

3. 信息从哪里来？

四类通道（详见技术部分）：

1. **免费 RSS / Atom**：媒体、机构官网的订阅源（主力）。
2. **免费网页列表**：政府、基金会官网的栏目页（已跑通 61 个）。
3. **微信公众号**：服务器上的 WeRSS 每 3 天扫码采集。
4. **邮件简报**：QQ 邮箱每 30 分钟检查指定发件人（如爱德传一《慈善纪事简报》）。
5. **网页搜索 API**（新增）：用 DeepSeek 网页搜索补公益事件发现，见第 9 题。

4. 怎么增加新的信息源？

先别急着开，按这个节奏：

1. 在后台「候选信源」或「新增信源」粘贴网址；
2. 让系统试抓几条，预览标题、发布日期、原文链接对不对；

3. 用唯一、稳定的英文 ID 命名；
4. 初始设为「暂停 / 隔离」，观察几天再启用；
5. 永远先显示摘要 + 原文链接，**不要擅自转载全文**（版权边界见技术部分）。 新增来源会多花 AI 调用费，先拿 10–30 条样本试跑、看后台用量，再决定是否扩大。

5. 怎么部署到自己的 NAS / 服务器?

技术上就是一台装好 Docker 的 Linux 机器（2核2GB 起），按 README「快速开始」clone 仓库、`cp .env.example .env`、`docker compose up -d --build` 即可。模型、公众号、邮件功能需要各自填凭据。

- **NAS 注意**：群晖等 NAS 能跑 Docker 就能跑，但公众号扫码（WeRSS 管理端只监听本机 127.0.0.1）需要你从电脑用 `ssh -L` 打隧道才能打开管理页面（命令见技术部分）。
- 部署纪律：改完先 `git` 同步、再 `docker compose` 重建**三条 `-f`** 都要带（否则会报 DNS 解析失败）。

6. 怎么购买 AI 模型 API，以后怎么换成更便宜的模型?

当前用的是 **DeepSeek**（模型名 `deepseek-chat` / `deepseek-flash`）：

1. 去 DeepSeek 官网注册、开通模型、创建 **API Key**（服务商控制台里的一串密码，**只给程序用，不要在聊天里发**）；
2. 在服务器 `/opt/yihot/.env` 填 `LLM_BASE_URL`、`LLM_API_KEY`、`LLM_MODEL`；
3. `docker compose up -d` 让配置生效；
4. 去后台「运行／模型与评测／用量与成本」看下一批任务是否正常。 **换更便宜的模型**（比如以后 DeepSeek 出更便宜的档位，或换阿里、智谱等）：只改 `.env` 里的 `LLM_MODEL` 和价格表里对应名字，**千万别只改模型名不更新价格**——否则后台会显示「未定价」、费用核算失真。换之前先拿几条样本测一下 JSON 稳定性、成功率和费用，再全量切换。

7. 海外信源不可达 / 被墙 / 403 怎么办?

有些境外站点（如 UNICEF Data、英国 FCDO、世界银行部分接口）在国内服务器上**根本连不上或被拒绝**。 **处理原则**：按项目纪律**不绕过、不伪造身份、不翻墙代采**。这类源要么放弃，要么等有境内镜像再接。绝不是「改个配置就能通」，别反复重试浪费时间。

8. 有些网站没有 RSS 怎么办?

没有 RSS ≠ 不能接。项目里 61 个网页列表源就是在没有 RSS 的情况下，靠解析网站栏目页接进来的。路线：

- 栏目页是普通 HTML → 用 `web_list` + 选择器抓列表；
- 列表是接口返回的 → 用 `json_list`；
- 实在抓不到（JS 渲染、登录墙、日期无法解析）→ 放弃或走邮件简报 / 公众号替代。 关键一步：接之前**必须看最新几条的实际发布日期**，别只看「能打开、有条目」——很多 RSS 长得活着、内容却停更大半年（这种叫「活着但已死」，接了只会灌旧闻）。

9. 网页搜索 API 进来后，有什么不一样？（重点）

多了一个**搜索型信源** `search-child-rights`：它不抓某个固定网站，而是定时用 DeepSeek 网页搜索，按 **12 组公益主题查询词**去发现近期事件（未成年人保护公益诉讼、困境儿童帮扶、儿童性侵判决、校园欺凌防治、孤儿保障、残障儿童康复、留守儿童关爱、营养改善计划等）。

- 它补的是「正规信源没覆盖到、但网上确实有讨论」的公益事件；
- 发现的内容同样要经过 AI 去重、评分、入池，才会进「动态」；
- **对热度的影响见第 10 题**；
- 查询词是运营可调的（在 `industry/sources.json` 的该源 `config.queries` 里，且需同步到数据库）。加新词前，先拿 20 条搜索结果逐条看是否对口，再反推词表，别凭想象加。

10. 热度（热点榜）是怎么算的？和以前有什么不同？

一句话：**热度 = 最近 48 小时内、来自「不同独立信源」的关注量，按时间衰减**。具体：

- 只看最近 **48 小时**内的信号；越近权重越高（**24 小时半衰**——每过 24 小时权重减半）；
- **讨论偏少时窗口自动放宽到 72 小时**：48 小时窗口里合格事件不足 6 条，且放宽后确实更多，就按 72 小时算（衰减规则不变，老证据照样沉底）。周末和节假日后的周一早上最容易触发——那时窗口起点正好落在周六早上，周六那批信号刚出窗。页头会写明实际用的窗口；
- 计的是**独立参与者（信源）数**，不是文章篇数。同一家媒体连发 10 篇，只算 1 个参与者；
- 搜索源按**站点域名**去重（同一网站多条结果算 1 个）；
- 想进「热点」榜，至少要 **2 个参与者，且其中至少 1 个是原创/编辑型信源**（RSS、官网、公众号、邮件）。**搜索 API 带来的是「信号型参与者」：能抬高热度，但不会只靠它上榜**——这点和以前（没有搜索源时）最大的区别就在这里：现在一个事件如果在「正规信源 + 网页搜索」里都出现，参与者更多、热度更靠前。
- 「热点」页顶部会标 **new / rising / surge** 等状态，对应「新出现 / 在涨 / 短时间内密集出现」。

11. 电脑和手机显示为什么不一样？

这是故意做的适配，不是 bug：

- **手机**：屏幕窄，底部固定五个按钮（总报 / 动态 / 热点 / 主题 / 更多），每页顶部品牌行右侧有一个常驻「订阅」按钮；「主题」「热点」直接进底栏，所以「更多」页不再重复放它们。底栏「更多」用三横线图标——它原先是四宫格，和「主题」的图标几乎分不出来。
- **电脑**：屏幕宽，左侧是常驻目录栏（含主题、热点与「更多」分组全部入口），不需要底部按钮。目录栏底部有一颗实色「订阅更新」按钮（原先是一行 11px 灰字，基本没人看得见，2026-10-23 改成按钮；10-30 已去掉「或用 RSS 订阅」那一行，RSS 入口仍在页脚）。
- **两端「更多」的顺序是同一份**：关于 / 更新 / 反馈 / 监管 / 收藏 / Agent，外观（深浅色）排在这六项之后。桌面在 `apps/web/app/components/shell/nav.ts` 的「更多」分组，手机在 `apps/web/app/routes/more.tsx` 的 `ROWS`，**改一处必须同时改另一处**，否则两端会出现两种顺序（2026-10-31 统一过一次：此前监管只在手机「更多」里，桌面侧栏没有）。
- 同一份数据，两端布局不同但内容一致。如果你在手机上找不到某个入口，到「更多」或切到电脑端基本都能看到。

12. 哪些事我绝对不要自己动手?

- 不要把任何密码、API Key、邮箱授权码提交到 GitHub 或写进文档;
- 不要手动改数据库里的日期、精选结果、费用记录;
- 不要因为一两次采集失败就删信源或历史内容（网站改版、验证码、临时断 RSS 都会失败，先观察）;
- 不要自己改模型、评分门槛、提示词、主题词——这些影响全部内容，需要样本校准和测试;
- 涉及改页面、部署、修程序、调算法：把「现象 + 发生时间 + 页面地址 + 后台错误」交给技术人/AI Agent，并让他先读仓库的 `AGENTS.md` 和本文技术部分。

13. 「只看当地」（按城市筛选）现在能做吗?

暂时不能，但技术上可行，已列入规划。当前内容库没有「地域」标签，所以没法按城市筛。要做的话分两步：先给内容打「全国/省/市」标签（来源可带默认地域，正文用 AI 抽地市级），再在读者打开时按 IP 推断城市、做近似过滤。注意 IP 只用于推断城市级位置、**绝不存储精确 IP 或个人位置**。具体设计见技术部分的「『只看当地』设计说明」。

14. 「关于」页最下面那一屏（关于我们 + 公众号二维码）怎么改?

那一屏讲的是机构本身：救助儿童会是谁、什么时候成立、在中国做什么，右边是公众号二维码。它不是数据说明页，**别把分析数字往上堆**。

- **文字都在 `industry/site.ts` 的 `ABOUT.account`**：标题、几段介绍、四个数字、按钮文案、二维码说明、官网链接，改完重新构建网页就生效。不想要这一屏，把 `account` 改成 `null`。
- **句子的事实要从公众号里抄**，别凭印象写。现用的一版来自招聘文末的「机构介绍」，那是机构自己维护的版本；员工数、国家数、在华省区数会变，**以最新一篇招聘文为准**。
- **二维码**取自后台「设置」里的**公众号码**，没上传就只显示文字、不留空框。
- 按钮「看看这十二年写过什么」跳到独立报告页 **https://49.233.72.222/yihot/about/dashboard**，内容是公众号 2014–2026 年 1,992 篇推文的分析。报告要重算时：在另一台机器上跑 `F:\公众号\救助儿童会\dashboard\build_dashboard.py`（先 `python analyze.py` 重算 `data.json`，再 `python build_dashboard.py --site` 出 `index.html`），把 `index.html` 与 `echarts.min.js` 覆盖到仓库的 `industry/dashboard/` 后重新部署。
- 报告里的图表库是**同目录本地文件**，不依赖 CDN（GitHub/CDN 在国内经常取不到），所以页面断网也能画图。
- **这份 `index.html` 是生成物，配色是发布前手改的**：`build_dashboard.py` 出的原稿仍用旧的 `#E4002B` 主色和 `#008C95` 青色，页头还是「红→粉→青」渐变。重跑脚本覆盖上来之后，要按下面这张表再改一遍（共 8 行，`git diff --ignore-cr-at-eol` 一眼能核）：
  | 位置 | 旧 | 新 | |---|---|---| | `:root` 变量 | `--accent:#E4002B; --accent2:#008C95` | `--accent:#DA291C; --accent2:#9A3324` | | `header` 渐变 | `#E4002B → #f2534a → #008C95` | `#9A3324 → #DA291C → #EF4B3F` | | 发文节奏・柱 | `#E4002B` | `#DA291C` | | 发文节奏・折线 | `#008C95` | `#9A3324` | | 代表关键词・横条 | `#008C95` | `#DA291C` | | 年度特征词・横条 | `#E4002B` | `#DA291C` | | 共现热力图色阶 | `#f4f7fa→#a9cdda→#3f9fb3→#02525f` | `#fdf0ee→#f3b4ab→#da291c→#7a1a12` | | `.btn.on` 投影 | `rgba(228,0,43,.22)` | `rgba(218,41,28,.22)` |
  只改这些，别动 `data` 里的主题色：14 个主题的 `"color"`（`#E4572E` `#3A86FF` 等）是主题身份色，和站内标签一一对应，换成红色系就没法在环形图、treemap 里分辨了。

这个站现在怎么工作

```
公开信源 / 公众号 / 邮件
          ↓
    自动采集和去重
          ↓
  相关性判断、摘要和标签
          ↓
动态 — 热点 — 主题
          ↓
     每周、每月总报
```

- 「动态」是主要阅读入口，优先按**原文发布日期**排列；原文没有日期时才显示收录时间，并明确标注「收录」。
- 「精选」栏目目前隐藏，但精选状态和卡片标签仍保留，用于总报、后台判断和以后恢复。
- 日报每天 08:00 生成（站内不设导航入口，保留为历史存档，可作邮件版内容）；周报每周、月报每月生成。
- 网站部署在腾讯云。采集、分析和发布都在服务器上运行，维护者不需要一直开着自己的电脑。
- 微信公众号由服务器上的 WeRSS 辅助采集；QQ 邮箱中的指定简报由邮件采集服务每 30 分钟检查一次。

新维护者第一次接手

原维护者应通过安全方式分别交付以下内容，**不要把密码写进 README、聊天记录或 Git 仓库**：

1. 腾讯云账号或服务器 SSH 访问方式。
2. GitHub 仓库访问权限。
3. Yi-Hot 后台管理员密码。
4. 凭据归档的解密口令。
5. QQ 邮箱、微信公众平台等外部账号的必要权限。

接手后按下面顺序确认：

1. 打开线上站点，确认「动态」能正常显示。
2. 登录管理后台，查看「运行」和「信源」页面是否有连续失败。
3. 打开「总报」，确认最近一期周报或月报可以阅读。
4. 在 GitHub 确认自己能看到本仓库，但不要随意删除分支、Actions、`LICENSE` 或 `NOTICE`。
5. 把解密口令存进个人密码管理器；不要和加密文件放在同一个位置。

完成以上五项，就具备日常维护条件。

日常维护只做这些

频率	要看什么	正常表现	发现异常怎么办
每周一次	网站首页和动态	最近几天有新内容，原文链接可打开	去后台「运行」和「信源」查看失败原因
每周一次	总报	周报正常生成	记录缺失日期和页面地址，交给技术维护者检查任务队列
每月一次	信源健康	大部分启用信源为正常；个别网站偶发失败可以观察	同一信源连续失败多次再处理，不要立即删除
每月一次	用量与成本	费用和调用量没有突然异常增长	暂停异常来源或模型调用，并请技术维护者核查
增加新来源时	原文质量、发布日期、版权边界	能稳定取得标题、摘要、发布日期和原文链接	先作为候选试抓，不要直接开启全文展示

后台入口：

- **内容诊断**：查看某条动态为什么出现、为什么没有入选。
- **信源**：查看采集是否正常，也可以先试抓再决定是否启用。
- **运行**：查看自动任务成功或失败。
- **用量与成本**：查看模型调用量和费用。
- **反馈**：处理读者提交的更正、下架或其他意见。

三种常见情况

网站能打开，但最近没有新内容

先去后台检查：

1. 「信源」里最近采集是否成功；
2. 「运行」里 worker 是否持续运行；
3. 新抓到的内容是否被判断为与公益无关；
4. 原文发布日期是否确实较早。

不要把本站收录时间当成原文发布日期，也不要为了让页面看起来更新而手工改日期。

某个来源连续失败

先保留来源并记录错误。网站改版、验证码、登录失效和 RSS 临时中断都可能造成失败。公众号失败通常先检查 WeRSS 登录状态；邮件来源失败通常先检查 QQ 邮箱授权是否仍有效。只有确认来源永久停用或不再适合收录时才关闭。

需要修改页面、部署或修复程序

这属于技术维护。把问题现象、发生时间、页面地址和后台错误一起交给技术人员。若使用编码 Agent，可以直接给它下面这段话：

```
请先阅读仓库根目录 AGENTS.md、docs/PROJECT_CONTEXT.md、
docs/memory/MEMORY.md、tools/README.md 和 secrets/README.md。
先核实线上现象和运行状态，再修改、测试、同步 GitHub 并部署腾讯云。
不要输出或提交任何密钥。
```

不要直接做的事

- 不要把 `.env`、邮箱授权码、后台密码、SSH 密码或 API Key 提交到 GitHub。
- 不要直接改数据库里的日期、精选结果或费用记录。
- 不要因为一两次失败就删除信源或历史内容。
- 不要绕过后台和现有脚本在生产服务器上临时改代码。
- 不要随意修改模型、评分门槛或提示词；这些改动会影响全部内容，需要样本校准和测试。
- 不要删除或改写 `LICENSE`、`NOTICE` 中的上游版权信息。
- 不要使用 AIHOT 的名称或 Logo 作为 Yi-Hot 的品牌。

备份、凭据与换电脑

- 生产运行时凭据以服务器 `/opt/yihot/.env` 为准。
- 仓库中的 `secrets/credentials.env.enc` 是加密备份；解密口令不在仓库中。
- 明文凭据只保存在本机忽略文件和服务器，不进入 Git。
- 凭据发生变化后，要更新服务器、更新本机明文归档、重新加密并提交新的密文。
- 换电脑或交接的完整步骤见 `docs/PROJECT_CONTEXT.md` 第 8、9 节和 `secrets/README.md`。

数据库归档（历史流水怎么裁）

站点内容（`articles` / `publications` / `analyses` / `reports`）**一行都不删**——页面、站点地图、旧报告都靠它们。只裁排障用的流水：`receipts` 与 `receipt_attempts`（每次模型请求的完整 request/response）、`article_revisions`（改稿全文）、`fetch_runs`、`hot_rankings`。默认保留期 180 / 180 / 180 / 90 / 365 天，写在 `scripts/archive-db.py` 的表里。

脚本在**宿主**跑（容器里才有 psql）：

```
sudo python3 /opt/yihot/scripts/archive-db.py            # 演练：只数行数
sudo python3 /opt/yihot/scripts/archive-db.py --delete   # 导出并删除
sudo python3 /opt/yihot/scripts/archive-db.py --before 2026-11-01 --delete
        # 按北京时间自然日切：归档并删除 11-01 之前的全部流水
```

导出是 `COPY (SELECT … WHERE 时间 < 截止) TO STDOUT` 直接 gzip 到 `/opt/yihot-archive/<年>/`，文件名带首末行时间；同目录 `manifest.json` 记行数与 sha256。删除按 ctid 分批（5000 行/批），每批独立事务，不会长时间锁表。

规则是「先落盘、再核对、后删除」：导出行数与应删行数不一致就停下不删；删完再查一次过期行必须为 0。宿主 root cron 每月 1 日 04:40 跑一次并带 `--delete`。归档文件请定期 `scp` 到本机留一份，服务器不是唯一副本。

删完记得 VACUUM。 Postgres 的 DELETE 只把行标成死元组，表文件不会缩小，磁盘也不还。要对被删的表跑 `VACUUM FULL` 才真正回收（会短暂锁表，几秒到几十秒）：

```
for t in receipts receipt_attempts article_revisions fetch_runs; do
  sudo docker exec aihot-db-1 psql -U aihot -d aihot -c "VACUUM FULL $t"
done
```

`receipts` 的 `request` / `response` 是整段 JSON，单字段远超 csv 模块 128 KB 默认上限，脚本已把 `field_size_limit` 放到 1 GB；数行数也必须走 csv 解析器而不是按 `\n` 切。

2026-11-11 首次真跑（`--before 2026-11-01`）：归档 17.7 万行 / 96 MB 到 `/opt/yihot-archive/2026/`，同时 `scp` 到本机 `yihot-archive/`；`VACUUM FULL` 后库 1338 MB → 1039 MB，`receipts` 440 → 242 MB。内容表行数前后一致（articles 30718 / publications 25154 / analyses 25284 / reports 91）。

基线全量备份用 `docker exec aihot-db-1 pg_dump -U aihot -d aihot --no-owner --format=custom | gzip > /opt/yihot-archive/base/aihot-$(date +%Y%m%d).dump.gz`，恢复用 `pg_restore`；这一步不影响在线服务。

`/opt/yihot-archive/` 下分三类：`base/` 全库基线（2026-11-11 一份 432 MB）、`<年>/` 按年滚动的流水 CSV 与 `manifest.json`、`scratch/<日期>/` 归档的一次性排查脚本与临时 SQL。服务器磁盘 50 G，按每天约 30 MB 增长估算只够两年，早于保留期的数据一律走「先归档再删」，并定期 `scp` 到本机留副本。

服务器上只应存在、且不进入 Git 的目录：`secrets/`（`.env`、admin 登录口令、邮件源密码、ingest token）、`backups/`（迁移前的库与环境快照）。除此之外 `/opt/yihot` 应与仓库 `main` 完全一致；用 `python3 tools/server_diff.py` 核对，输出 `need upload total=0` 且 `SERVER stray` 只剩上述两类即正常。临时调试脚本不要留在 `/opt/yihot` 根目录——仓库 `tools/` 里已有 `dryrun-web-list.mjs`、`probe-deep.mjs`、`probe-24-sources.mjs` 等同类工具，用完归档到 `scratch/` 即可。

给技术维护者的入口

请依次阅读：

1. `AGENTS.md`：项目规则与安全边界。
2. `docs/PROJECT_CONTEXT.md`：服务器、部署、数据口径和接手清单。
3. `docs/memory/MEMORY.md`：运维经验索引。
4. `tools/README.md`：GitHub 同步、部署和核查脚本。
5. `secrets/README.md`：凭据加密归档。
6. `docs/sources.md`：信源类型与配置。
7. `docs/selection.md`：精选判断与校准。
8. `docs/PAID_CHANNEL_PROVIDERS.md`：付费采集通道（极致了 / EveryInfra）的取舍、价格与接入顺序。

9. `docs/DOMAIN_AND_HOSTING.md`：域名、备案与海外主机的决策记录。

最低验证命令：

```
npm run typecheck
npm run build -w @aihot/web
node --test apps/web/tests/*.test.ts
```

涉及数据库行为时还要在名称以 `_test` 或 `_ci` 结尾的测试库中运行迁移和 `npm test`。本项目在当前网络环境下推送 GitHub 必须使用 `tools/sync.py`；完整说明见 `tools/README.md`。

来源、版权与 AIHOT 归属

Yi-Hot 是在 AIHOT 开源项目基础上修改的公益行业定制版本。 AIHOT 由“数字生命卡兹克 / KKKKhazix”开发，代码依据 MIT License 开放。本仓库保留了上游的 `LICENSE` 和 `NOTICE`。

使用、复制、修改或再发布本项目时必须注意：

1. 按 MIT License 要求，在软件副本或主要部分中保留原版权声明和许可文本。
2. 保留 `NOTICE` 中的上游及第三方材料说明。
3. AIHOT 的名称和 Logo 不属于 MIT 授权范围；本项目使用自己的 Yi-Hot 名称和品牌，也不得让人误以为这是 AIHOT 官方站点。
4. 原始新闻、图片和文章版权归各来源所有。Yi-Hot 默认只展示摘要和原文链接，只有来源明确允许时才展示全文。
5. 新增第三方代码、图片、字体或数据前，应核对其许可证并补充相应归属说明。

本节用于帮助维护者保留清晰来源，并不替代 `LICENSE`、`NOTICE` 或专业法律意见。

> **以下从「技术概览」到文末，是技术 / AI 维护参考。** 普通接手人可跳过，只在前半部分的人话 Q&A 和日常维护表里操作即可。这部分写给以后接手的技术人或 AI Agent，包含运行链路、模型、信源、部署、故障处理等完整细节。

技术概览

- Node.js 24、TypeScript、React Router、Fastify
- PostgreSQL 17、pg-boss
- Docker Compose
- 一个镜像运行 Web、API、worker 和数据库初始化任务
- 公开出口包括网站、RSS、API 和 MCP

项目内部仍保留 `aihot`、`@aihot/*` 和 `aihot-*` 等技术标识，这是为了兼容上游结构和现有部署；对外品牌始终是 **Yi-Hot**。

完整维护手册

下面的内容面向第一次接手、不了解 API、模型或服务器的维护者。它把日常使用、密码、模型、采集、邮件、部署和故障处理放在同一份文档中。遇到复杂改动时，再沿文中的链接阅读专题文档。

一、完整运行链路

```
网站 / RSS / 微信公众号 / QQ 邮件
                ↓
          定时采集、规范化、去重
                ↓
          公益相关性判断和内容分析
                ↓
        中文标题、摘要、分类、主题、标签
                ↓
      全部动态 / 主题页 / RSS / 公开 API
                ↓
            每周、每月公益总报
                ↓
        网站阅读 / 邮件订阅 / 外部程序读取
```

服务器上的 worker 持续执行这些任务。维护者关掉自己的电脑不会影响网站、公众号采集、邮件采集或总报发送。

生产服务如下：

服务	作用	运行方式
`aihot-web-1`	网页和 SSR	Docker
`aihot-api-1`	公开 API、后台 API、订阅 API	Docker
`aihot-worker-1`	采集、分析、归组、发布、总报	Docker
`aihot-db-1`	PostgreSQL 数据库	Docker volume
`yihot-werss-werss-1`	微信公众号 RSS	Docker
`yihot-email-source.timer`	每 30 分钟读取获准邮件来源	systemd
`yihot-email-delivery.timer`	每 30 分钟发送确认信和总报	systemd

二、后台怎么用

后台地址是 https://49.233.72.222/yihot/admin。登录成功后浏览器保存有时效的会话 Cookie；不要在公用电脑保存登录状态。

信源

这里回答“平台正在从哪里取内容”。重点看启用状态、最近采集、连续失败和采集间隔。

- 新来源先预览真实标题、发布日期和原文链接。
- 初始设为暂停或隔离，确认内容范围后再改为编辑候选。
- 一两次网络超时可以观察；连续多次失败再处理。
- 不要为了增加内容量而开启全文展示或降低相关性门槛。
- 部分信源用 `publication_rule` 在源上收紧放行（如小红书：分数硬线 + 类别线 + 机构/项目主体词 + 拦截类别），规则不过的内容连事件都不建。调这类源的手感时改它自己的规则，不要动全局入选门槛；字段含义见 docs/sources.md。

内容诊断

这里回答“某条动态为什么出现、为什么没出现”。可以查看原始文章、模型分析、发布记录、事件归组和调用回执。

- 字段错误通过后台 override 或正规回填脚本修正。
- 重新分析会产生模型费用，先确认原始数据确实有错。
- 隐藏、合并、拆分等操作要写明原因，操作会进入审计记录。

运行

这里回答“自动任务是否正常”。重点关注采集是否持续、是否出现成片失败、付费请求是否结果未知、投递是否需要人工处理。单次失败不一定是故障；所有来源同时停止或同类错误连续出现才需要立即处理。

用量与成本

这里显示模型调用数、输入/输出 token、估算费用和月化区间。费用依赖后台维护的单价；显示“未定价”不代表免费，而是模型名称没有匹配的价格记录。

页面把两种钱分开列，不要混着读：

- **模型估算**（`amount`）：token 数 × 单价表算出来的，不是服务商账单。
- **按次实付**（`paid`）：小红书、公众号这类按请求计费的采集接口，服务商在自己的回执里直接给出扣了多少钱（`receipts.cost`，`cost_basis = 'actual'`）。它们不产生 token，所以在“按用途”表里不会出现，只在“按次计费的服务商”那张表里。
- **费用合计**：两者相加，这才是这段时间真实花掉的钱。

“未扣费”的次数要单独看：调用失败了（402 余额不足、钥匙无效）服务商不会收钱，这类调用计数增长而金额不动，正是排查余额问题的线索。

平台后台预算是第一层保险丝，模型服务商控制台的余额和消费上限是第二层。任一侧达到上限都会停止相应调用。

模型与评测

这里能看到每类任务使用的模型、成功率、延迟、token 和费用，也可以切换模型。切换前应先配置新模型 API Key，用少量真实样本验证结构化输出，维护单价，并观察至少一个采集周期。

反馈与审计

反馈页用于处理读者提交的更正、下架和其他意见。审计页用于追溯管理员操作。发生误操作时应修正并说明原因，不要删除审计记录。

三、后台密码与凭据

密码在哪里

后台密码是环境变量 `ADMIN_PASSWORD`。明文不在 Git 仓库中。当前生产真值位于：

```
/opt/yihot/.env
```

仓库中的加密备份是：

```
secrets/credentials.env.enc
```

拿到原维护者单独交付的解密口令后，在仓库根目录运行：

```
bash tools/secrets.sh decrypt
```

命令会生成被 Git 忽略的 `secrets/credentials.env`。从中查看 `ADMIN_PASSWORD`。不要把这个文件、密码或屏幕截图发到聊天或提交到 GitHub。

忘记密码怎样重置

如果仍能登录服务器：

```
ssh <服务器用户>@<服务器地址>
cd /opt/yihot
sudoedit .env
```

把 `ADMIN_PASSWORD` 改成至少 12 位的随机密码，然后让容器重新读取 `env_file`：

```
sudo docker compose -f docker-compose.yml -f compose.futureme-proxy.yml -f compose.werss.yml up -d
```

只执行 `docker restart` 不会重新读取 `.env`。验证新密码后，把新值同步进本机 `secrets/credentials.env` 并重新加密：

```
bash tools/secrets.sh encrypt
```

如果解密口令和 SSH 权限都丢失，需要先通过腾讯云控制台恢复服务器访问。加密归档没有后门。

各类凭据的位置

凭据	生产位置	用途
Yi-Hot 主配置	`/opt/yihot/.env`	后台、数据库、模型、签名、功能开关
邮件信源	`/opt/yihot-email/.env`	QQ IMAP、来源白名单、ingest token
邮件投递	`/opt/yihot-email-delivery/.env`	QQ SMTP、发件地址、站点地址
WeRSS	`/opt/werss/.env`	管理账号、签名密钥、镜像配置
核心凭据备份	`secrets/credentials.env.enc`	换电脑和灾难恢复

服务器配置是运行时真值，加密归档是备份。凭据变更顺序是：更新服务器、重建并验证服务、更新本机明文、重新加密、只提交密文。

四、模型 API 从零说明

API、Key 和 token 分别是什么

- **API** 是程序调用服务的网络接口。Yi-Hot 通过 API 自动请求模型，不是在浏览器里操作聊天网页。
- **API Key** 类似只给程序使用的密码，通常在模型服务商控制台创建。
- **Base URL** 是服务商的接口地址。
- **Model** 是具体模型名称。
- **Token** 是模型读写文本的计费单位，与 GitHub Token、后台密码不是同一种东西。

不要在聊天里发送 API Key。应直接写入服务器 `/opt/yihot/.env`，或先写入本机被忽略的凭据文件后安全部署。

申请和配置步骤

1. 注册 DeepSeek、阿里云百炼、智谱等支持 OpenAI 兼容接口的服务商。
2. 在服务商控制台开通模型服务并创建 API Key。
3. 充值或设置服务商侧消费上限。
4. 在服务器 `/opt/yihot/.env` 填写接口地址、Key 和模型名。
5. `docker compose up -d` 让新配置生效。
6. 去后台「运行」「模型与评测」「用量与成本」检查下一批任务。

服务商的模型名、价格和接口会变化，应以其当前官方控制台为准。

最小配置示例：

```
LLM_BASE_URL=https://api.deepseek.com/v1
LLM_API_KEY=在服务商控制台创建的Key
```

```
LLM_MODEL=服务商当前支持的模型名
LLM_EXTRA_JSON={"thinking":{"type":"disabled"}}
```

阿里云百炼示例：

```
LLM_BASE_URL=https://dashscope.aliyuncs.com/compatible-mode/v1
LLM_API_KEY=在百炼控制台创建的Key
LLM_MODEL=服务商当前支持的模型名
LLM_EXTRA_JSON={"enable_thinking":false}
```

平台可以给不同步骤分配不同模型：

变量	任务
`PREFILTER_MODEL`	公益相关性预筛
`SCORE_MODEL`	重要性评分
`UNDERSTAND_MODEL`	内容理解
`SUMMARIZE_MODEL`	标题和摘要
`STRUCTURE_MODEL`	结构化抽取
`GROUP_MODEL` / `GROUP_REVIEW_MODEL`	事件归组和复核
`DIGEST_MODEL`	事件摘要
`REPORT_MODEL`	周报和月报
`TRANSLATE_MODEL`	翻译

不设置这些变量时全部使用 `LLM_MODEL`。对非技术维护者，先维持一套稳定默认模型最容易维护。

费用异常时可以把 `MODEL_CALLS_ENABLED=false` 作为紧急止损。`COLLECT_ENABLED=false` 会停止采集。恢复前先查明原因，以免积压内容一次性触发大量付费请求。

五、信息源结构与新增来源

当前成本策略：免费 RSS 优先 + DeepSeek flash + EveryInfra 保留待充值

2026-10-21 起按这个顺序花钱，**任何新增信源默认先找免费通道**：

层	通道	现状
1	免费 RSS/Atom（机构官网、官媒频道、WeRSS）	主力，已在跑
2	免费网页列表（政府部门、基金会官网）	已有 61 个 `web_list` 源

层	通道	现状
3	AI 分析	`LLM_MODEL=deepseek-flash`，`LLM_BASE_URL=https://api.deepseek.com/v1`
4	EveryInfra 付费接口	**通道保留、预算归零**，不再产生费用

EveryInfra 为什么是"保留"而不是"删除"：代码、源配置、正文回补脚本全部原样留着，只是把 `budgets` 里 everyinfra 的 `per_minute/per_hour/per_day` 全设成 0。`checkBudget()` 遇到 `per_* <= 0` 会走内建的 `stopped` 分支——请求在发出前就被拦下，所以**一分钱都不会花**，同时源不会被判失败、不会累计 `fail_count`、不会降级健康度。以后充值，只要把 `per_day` 改回 150（需要补正文时可临时 250），全部八个公众号源立刻恢复原样，**不用改任何代码**。

别踩的坑：看到源显示 `degraded` 且 `last_error` 是 `everyinfra HTTP 503 (capacity_exhausted…)` 时，不要去重跑、不要去改配置。这是 provider 侧容量耗尽（响应体里 `charged:false`，不计费，且自带 `retry_after` 秒数），调度层会照它给的秒数回来重试。

DeepSeek 余额耗尽的表现（2026-10-21 踩过一次）：采集一切正常、文章正常入库，但 `articles.processing_state` 全是 `failed`，`processing_error` 是

```
HTTP 402: {"error":{"message":"Insufficient Balance (request_id: …)"}}
```

这不是代码问题，是账户没钱了，全站分析会一起停（不只是新源）。判断依据很简单：`receipts` 里 `service='llm'` 的失败记录会密集出现同一个 402。

402 之后不会再"每篇都撞一次"（2026-10-30 起，移植上游 `5c2f081`）：第一次 402 会写一条冷却记录

```
SELECT key, value FROM settings WHERE key LIKE 'provider.stop.%';
-- provider.stop.llm:deepseek-flash | {"until":"2026-10-30T06:30:00.000Z","refusedAt":"..."}
```

冷却 15 分钟内，后续文章在发出请求前就被 `checkBudget()` 挡下，抛 `BudgetExceededError(window='provider')`，`jobs/content.ts` 把它当成"预算窗口满了"处理——**文章回到 `new` + `processing_retry_at`，不消耗 `processing_attempts`、不进 `failed` 终态**。所以现在余额耗尽时表现为"一批文章在排队等"，而不是"一批文章彻底失败"；充值后不必再 `requeueFailed` 清 402，等冷却到期自动恢复，或者任意一次成功调用会立刻清掉这条记录。

排查时看这两处就能区分：

```
-- 还在等（健康）：processing_error 里是 Budget for llm exhausted (provider)
SELECT processing_state, count(*) FROM articles WHERE processing_error LIKE '%(provider)%' GROUP BY
1;
-- 真失败：402 只会在冷却里的第一篇上出现
SELECT count(*) FROM articles WHERE processing_error LIKE 'HTTP 402%' AND processing_state =
'failed';
```

注意两点：停止键**按模型记**（`provider.stop.<service>:<model>`），一个模型的额度耗掉不影响同服务别的模型；**429 里的纯限流不算**（只认 402 与账务类 429），所以 TPM/RPM 触顶不会误触发冷却。

恢复三步（2026-10-21 实测走通，可直接照抄；2026-10-30 之后新产生的 402 不必再走第 2 步——它们停在 `new` 等冷却，不在 `failed` 里。这里保留是为了清理改动前就已 failed 的存量）：

1. 先验余额到账，别盲跑。余额接口有小延迟，刚充完第一次查可能还是旧值，隔几分钟再查一次才是真值：

   ```
   ssh yihot "sudo bash -c 'set -a; . /opt/yihot/.env; curl -s -H \"Authorization: Bearer \
   $LLM_API_KEY\" https://api.deepseek.com/user/balance'"
   ```

   看 `is_available:true` 且 `total_balance > 0` 才算真的到账。
2. 重排队。`/app/.env` 在容器里不存在是正常的（变量由 compose 注入），报 `not found. Continuing without it.` 可以忽略：

   ```
   ssh yihot "sudo docker exec -w /app aihot-worker-1 node --env-file-if-exists=/app/.env --input-
   type=module -e \"const c=await import('@aihot/backend/jobs/content'); const q=await
   import('@aihot/backend/jobs/queue'); const d=await import('@aihot/backend/db');
   console.log(await c.requeueFailed(null)); await q.stopBoss(); await d.closeDb();\""
   ```
3. 验结果：`failed` 应为 0，`pgboss.job` 里 `content.analyze` 的 created/active 归零、`state='failed'` 为 0。41 篇积压大约一两分钟跑完（并发 6）。

flash 的真实价钱与续航（2026-10-21 实测对账，官方价目表见 api-docs.deepseek.com「模型 & 价格」）：

项	单价（人民币 / 百万 tokens）
输入・缓存命中	空闲 0.02 / 高峰 0.04
输入・缓存未命中	空闲 1 / 高峰 2
输出	空闲 4 / 高峰 8

高峰 = 北京时间**工作日** 9:00–12:00、14:00–18:00，其余（含周末、法定节假日）都是空闲价，**高峰是空闲的两倍**。

- 实测基准：165 次调用 / 未命中 183K / 命中 355K / 输出 10K，扣费 **¥0.57**；按价目表回算 ¥0.46，对得上。**可以记成 ≈ ¥0.0035/次调用**。
- 平静日（约 300 次调用）≈ **¥1/天**；回填或批量重排的日子（1500+ 次）≈ **¥5/天**。
- 成本八成来自**未命中输入**（命中便宜 50 倍）。24 小时实测命中率约 63%，`score_article` 命中最好（5.1M 命中 / 1.3M 未命中），`prefilter_article` 最差（0.53M / 1.4M）——想省钱先从这里下手，而不是砍信源。
- **`budgets` 表管不住这件事**：`llm` 的 `per_day` 是 40000 次，远高于实际用量，熔断基本不会触发；真正托底的是账户余额。余额见底时仍是 402 全站停摆（现在会挂 15 分钟冷却而非逐篇失败），所以**定期查余额**才是唯一有效的兜底，不能指望熔断。

高峰让路：`ANALYZE_OFFPEAK_ONLY`（2026-10-21 起开启）

DeepSeek 工作日高峰价是空闲价的**两倍**，而原来采集一篇就立刻分析一篇，白天采的稿子全撞在贵的时候。`.env` 里设 `ANALYZE_OFFPEAK_ONLY=true` 后：

- `isPeakAnalysisHour()`（`jobs/content.ts`）把工作日北京时间 **9–12、14–18** 判为高峰；周末全天、其余 17 小时为空闲。
- 高峰**不是全停**，而是一小时放行一小批：
  - `peakSlotsLeft()` 用 `ANALYZE_PEAK_PER_HOUR`（`.env`，默认 8，2026-11-11 起线上设 **20**）限制**当日新稿**的付费分析篇数；计数从 `receipt_attempts` 里 `purpose='prefilter_article'` 且 `origin='live'` 的回执恢复，所以 worker 重启不会多给一整份额度。
  - `holdsHistoryForOffPeak()` 把**历史回填**整段压到空闲窗口——旧稿等几小时不涨价也没有人等着看，而回填曾是高峰花费的大头（10-24 实测：30 天高峰花费 ¥9.93，其中绝大部分是回填）。
  - 正文抓取（extract）照常立即执行，因为抓网页不花钱。
- 管理员手动重评（带 `attemptTag`）**不受限制**——人点「现在就要」就是现在就要。

为什么是「放一点」而不是「全停」：全停三小时会让 feed 从 9 点到 12 点一条不动，然后 12:00 连本带利倒灌一批，读者的观感就是「网站坏了」。2026-11-11 上午的实测正是这个症状——09、10、11 点各恰好 8 篇，卡在默认上限，于是把 `ANALYZE_PEAK_PER_HOUR` 从 8 调到 20。代价是高峰单价翻倍，但每篇全流程只要几分钱，一天多花的量级在个位数人民币。

为什么也不是「全攒到夜里」：`notify/selected.ts` 有 12 小时推送时效门槛（`MAX_AGE_MS`），超过就跳过推送。全攒夜间会让 18:01 采的稿子等到次日 12:00（18 小时），推送直接废掉。所以只压高峰这 7 小时，最长延迟 9 小时（08:59 的稿子 18:00 处理），留在门槛内。

时间语义不受影响（这是最容易被误解的一点）：`discovered_at`（收录时刻）和 `published_at`（原始发布时间）在入库那一刻由 `decideTimeline()` 写死，`timeline_at` 由二者推出。改的只是**模型何时跑**，不是时间轴说什么。延迟分析不会把新闻变成历史回填，也不会挪动任何时间。

- 改法：`.env` 里调 `ANALYZE_PEAK_PER_HOUR`（0 = 高峰完全停，即旧行为），然后 `docker compose ... up -d --no-deps --force-recreate worker`。这是运行时变量，**不需要 `--build`**。
- **验证方式**：高峰时段按小时统计 `select to_char(created_at,'HH24'), count(*) from analyses where created_at > now() - interval '12 hours' group by 1`，数值应贴近 `ANALYZE_PEAK_PER_HOUR` 而不是 0；`select count(*) from articles where processing_state='new'` 不应持续堆积。若每小时都精确等于上限，说明还有更多稿子在排队，可以考虑再上调。

信源抓取节奏：`adaptIntervals`（2026-11-11 调整）

抓取间隔不是手工配置，而是**每天 04:20 由 `sources.adapt-intervals` 按近 7 天日均产出重算**（`packages/backend/src/sources/collect.ts` 的 `adaptIntervals()`）。调度器 `sources.schedule` 每分钟跑一次，捞出 `next_fetch_at <= now()` 的源，抓完按 `interval_minutes` 排下一次。

目标间隔 = `24 × 60 / (perDay × samples)`，再夹到上下限之间：

参数	取值
`max`	`hot_signal` 180；`x_search` 或走 Jina 的 120；其余 60
`min`	走 Jina 的 60；**其余 10**
`samples`	日均 ≥ 20 篇取 6，否则取 3
特例	`perDay <= 0.15` 直接取 `max`；`perDay <= 0` 取 `max(max, 360)`；`kind='search'` 固定 360

关键：手动 `UPDATE sources SET interval_minutes` 只在下次 04:20 之前有效，重算会盖掉它。要长期改变某个源的节奏，只能改上面的公式或改变这个源的实际产出（`perDay`）。2026-11-11 就发现过实例：caixin 表里写 20、按公式应为 21，等于手动值压不过重算。

`samples=6` 那一档留给日均 20 篇以上（平均不到 72 分钟一篇）的源：3 倍采样时期望发现延迟约一个间隔的三分之一，6 倍能把繁忙时段的延迟压到个位数分钟。**这只增加抓取流量，不增加分析成本**——同一篇稿子无论列表被读几次，`articles` 里只有一行。

实测（2026-11-11，近 7 天日均）：`web-thepaper` 81.9 篇 → 10 分钟、`rss-chinanews-scroll` 35.7 篇 → 10 分钟、`web-caixin` 23.1 篇 → 10 分钟，其余 60 个源一个没动。验证节奏查 `fetch_runs`：

```
SELECT s.id, f.started_at AT TIME ZONE 'Asia/Shanghai' AS started,
       round(EXTRACT(EPOCH FROM (f.started_at - lag(f.started_at) OVER (PARTITION BY s.id ORDER BY
f.started_at)))/60.0, 1) AS gap_min
FROM fetch_runs f JOIN sources s ON s.id = f.source_id
WHERE s.id = 'web-thepaper' ORDER BY f.started_at DESC LIMIT 10;
```

顺带一句：**抓取变快解决不了「午间空窗」**。间隔缩短只能让新稿更快被发现，源本身不发稿的时段仍然是空的——那要靠加信源，不是调节奏。

免费官方 RSS 现状（2026-10-21 实测）

已接入并产出：

源 ID	站点	首采入库	命中
`rss-gov-cn`	中国政府网・政策推送	4	4 pass（均分 64.8）
`rss-who-news`	世界卫生组织・新闻	60	53 pass（88%）
`rss-chinanews-health`	中国新闻网・健康	4	3 pass
`rss-chinanews-scroll`	中国新闻网・滚动	2	0（时政新闻，被正确过滤）

探测时务必检查条目日期，不能只看 HTTP 200。 人民网各频道 RSS 至今仍返回 200 且有 item，但内容停在 2025-05~06（教育频道甚至是 2016 年的）——这类"活着但已死"的 feed 接入后只会灌入陈旧内容，已在本次清理中剔除。

41 条待审信源的探测结论

`data/source-registry/proposals-child-rights-2026-10-21.json` 里 41 条全部做了只读探测（GET 首页 + 常见 RSS 路径 + `/robots.txt`），结论：

- **connected 3 条**：已接入（中国政府网、中国新闻网、世界卫生组织）
- **stale_rss 1 条**：人民网——RSS 仍返回 200 和满页 item，但内容停更 → 不接
- **blocked 5 条**：国家卫生健康委（412）、公安部（521）、UNICEF Data（403）、亚洲开发银行（403）、英国 FCDO（不可达）→ 按内容纪律**不绕过、不伪造身份**
- **no_rss 32 条**：站点可达但无官方 RSS → 走下面的接入工序

那个文件是"覆盖 backlog"，**不是已批准的采集端点**，不能批量灌进自动 seed。

2026-10-23 复核：这份清单现在覆盖了多少

按**生产库 102 个启用源**逐条重核（域名 + 机构名双重比对，不是拿域名子串瞎匹配）：

- **41 条 backlog：已接入 18 条、仍未接入 23 条**
- 按你最初 60 行的完整清单算：**37 / 60（约 62%）已经有源**

已接入的 18 条（ID 沿用 backlog 编号）：

#	清单条目	对应源	健康
001	中国政府网	`rss-gov-cn`	ok
002	全国人大	`web-npc-lfdt`	ok
005	国家统计局	`web-stats-tzgg`	ok
006	最高人民检察院	`web-spp-wsfbt`	ok
008	共青团中央	`web-gqt-gzdt`	ok
009	国家发展改革委	`web-ndrc-xwfb`	ok
011	UNICEF（儿童保护）	`web-unicef-global-press` / `web-unicef-innocenti-reports`	ok / degraded
013	世界卫生组织	`rss-who-news`	ok
019	中国人口福利基金会	`web-cpwf-news`	ok
020	中国红十字基金会	`web-crcf-news`	ok
021	北京新阳光慈善基金会	`web-isun-news`	ok
022	心和基金会	`web-xinhe-news`	ok
024	中国社会福利基金会	`web-cswef-news`	degraded
025	中国基金会发展论坛	`web-cfforum-news`	ok

#	清单条目	对应源	健康
031	新华网	`web-xinhua-szlb`	ok
033	中国新闻网	`rss-chinanews-scroll` / `-health` / `-edu`	ok
039	中国妇女报	`web-cnwomen-news`	ok
040	中国青年报	`web-cyol-news`	ok

仍未接入的 23 条，按原因分四类：

原因	条数	条目
对端明确拒绝（按纪律不绕过、不伪造身份）	6	国家卫健委 412、公安部 521、人社部、UNICEF Data 403、亚行 403、英国 FCDO
需 JS 渲染 / 没有静态列表	9	国家疾控局、UNESCO、世界银行、中国发展研究基金会、腾讯公益、欧盟 Funding、澎湃、财新、中国教育报
日期或内容无法解析	4	阿里巴巴公益、南方周末、央视新闻、检察日报
需登录 / SPA / 非 feed	4	UNGM、青基会官网、人民网（RSS 停更）、慈善中国（要走 `json_list` 接口）

两个容易算错的地方，下次复核别再踩：

1. **澎湃不算接了**。`thepaper.cn` 确实出现在库里，但只是在 `web-tophub-news`（今日热榜聚合）的链接白名单里，是热榜带出来的跳转，**不是澎湃的一手源**。
2. **按域名机判会漏**。共青团中央的源是 `qnzz.youth.cn`、新华网是 `news.cn`，和清单里写的 `gqt.org.cn` / `xinhuanet.com` 对不上——按机构名才认得出来。反过来，拿 `gov.cn` 当子串去匹配，会把民政部、教育部、各省民政厅全都算成"覆盖了中国政府网"，一共虚高 40 多条。

无官方 RSS 的站点怎么接

这 32 条不是"不能接"，而是 RSS 那条路走不通。**项目里 61 个 `web_list` 源全部健康**，说明网页解析这条路本来就跑得通，只是要多几步人工工序：

1. **先别信首页地址**。清单里的 URL 几乎都是首页，而首页常常抓不到东西：慈善中国首页只有 64 字节，内容是一句 `location.href="/platform/application.html"` 的 JS 跳转，curl 不执行 JS 就只能拿到空壳。要人工定位到真正的**栏目页/列表页**。
2. **判断列表是静态的还是 XHR 拿的**。抓一次栏目页，看 HTML 里有没有 `<a>` 列表；没有就去页面内联 script 里找 `url:` / `ajax:` / `api` 线索。慈善中国就在壳里留了 `getIndexList1.html`、`getIndexList2.html` 这类接口，直接返回结构化表格。
3. **静态列表 → `web_list` + CSS 选择器**（`itemSelector` / `linkSelector` / `publishedAtSelector`）；**XHR 接口 → `json_list`**（`itemsPath` / `titlePaths` / `publishedAtPath`）。后者更稳，不随页面改版失效。
4. **补 `detail` 抓正文与日期**。只拿到标题不够分析，需要 `detail.publishedAtSelector` / `titleSelector` / `summarySelector`。

5. **配 `requireMarkers` 主题词**。公益/儿童相关的才入库，其余**在入库前就丢掉、不消耗 AI 分析**——这是把成本压住的关键。
6. **初采后必须核验产出**：看入库条数、pass 率、以及抽样看标题是不是真公益。中新网滚动新闻入库 2 条、0 pass，就是过滤器在正确工作。

接之前先想清楚值不值。 这 32 条里有两类要区别对待：

- **优先接**：基金会与公益机构官网（青基会、红基会、人口福利基金会、社会福利基金会、发展研究基金会、心和、新阳光）、慈善中国这类**垂直源头**，内容天然贴合站点定位。
- **慎接**：新华网、澎湃、央视、中青报、财新、南方周末等**综合媒体门户**。它们喂进来的是泛新闻，会稀释公益垂直度，而且要靠更强的过滤词才拦得住——项目已有 25 个 RSS 源在覆盖新闻，重复度很高。真要接，务必先定过滤词再开口子。

垂直机构官网接入记录（2026-10-21，第一批）

按上面的工序实做了一批 P0/P1，**8 条里接成 5 条**：

源 ID	机构	列表页	解析条数	列表项选择器
`web-cpwf-news`	中国人口福利基金会	`/category/13`	10	`.dynamic-item`
`web-cswef-news`	中国社会福利基金会	`/cswef/news/index.html`	12	`.list-cont`
`web-isun-news`	北京新阳光慈善基金会	`?m=content&c=index&a=lists&catid=23`	24	`.item-content`
`web-crcf-news`	中国红十字基金会	首页卡片流	12	`.am-u-lg-4`
`web-xinhe-news`	心和基金会	`/home/info/index/cate_id/21.html`	11	`li.way-list-item`

五个站共同点：日期都在列表页，`detail.maxFetches` 控制每次采几篇正文。红基会和心和用的是**首页卡片流**，靠 `allowUrlPrefixes` 把导航链接挡在外面（红基会限定 `/web/article.thtml`，心和限定 `mp.weixin.qq.com/s/`）。

首采核验结果（2026-10-21，等 DeepSeek 余额恢复后才跑完，共 37 篇全部 analyzed）。这里的 `selected` 用 `publications` 口径（已过硬性 `minScore`），比 `analyses.selected` 严，两个数不能混用：

源 ID	入库	pass	入池	pass 均分	结论
`web-cswef-news`	8	8	4	44.5	最好，留
`web-cpwf-news`	8	6	1	35.0	留
`web-crcf-news`	9	9	1	24.1	留，命中率低
`web-isun-news`	8	8	0	21.0	公示/手册类居多，暂无入池
`web-xinhe-news`	4	0	0	—	**4/4 `unknown`，见下**

和心的坑（已验证，别再假设）：它的列表项全部外链 `mp.weixin.qq.com`，而 `pageFetchable()`（`content/extract.ts`）**明确把 `mp.weixin.qq.com` 排除在正文抓取之外**（X 帖与微信文章的设计是「要么整篇到达，要么没有」）。所以这 4 篇恒为 `body_status='unconfirmed'`，AI 只拿到标题就保守判 `relevance='unknown'`（分数 22–34，reason 为空），**仍然照常消耗 4 次调用却零入池**。曾以为「标题+日期够判定」，实测证伪。官网自身也没有站内详情页（列表页只有栏目导航，无 detail 路由），改 `allowUrlPrefixes` 救不回来。当前保留是因为更新量极小、成本可忽略，且选题（0-3 岁儿童早期发展）对口；若要动它，要么给 `mp.weixin.qq.com` 的正文开一条受控通路，要么关掉。

三条接不了的，**原因都是硬性的**，不要反复重试：

站点	结论	原因
中国青少年发展基金会官网	`spa_auth_required`	Nuxt SPA，首页 481KB 但 HTML 里 0 个 `<a>`；`work.cydf.org.cn/api` 全部接口返回 401 需登录。按纪律不绕过认证。该机构已有公众号源 `wechat-oa-cydf` 覆盖
中国发展研究基金会	`js_rendered`	`/zxcg/index.htm` 是导航骨架，整页只有 1 个日期（2022 年），列表由 JS 异步加载
慈善中国	`not_a_feed`	它是**信息公开查询平台**（查组织、募捐备案、慈善信托），不是资讯站，没有新闻流。`getIndexList*.html` 返回的是查询快照，不适合当内容源

综媒与政务源接入记录（2026-10-21，第二批 A/B 类 16 条）

41 条清单里剩下的 16 条（4 条 A 类 + 12 条 B 类，均标 `no_rss`）逐站实做，**9 条接成、7 条接不了**。

接成的 8 条（全部跑通 `collect`，选择器已落到 `sources.json`）：

源 ID	站点	列表页	解析	列表项选择器	日期选择器
`web-spp-wsfbt`	最高人民检察院	`/xwfbh/wsfbt/`	30	`ul.li_line li`	`span`
`web-stats-tzgg`	国家统计局	`/xw/tjxw/tzgg/`	13	`div.list-content li`	`span`
`web-ndrc-xwfb`	国家发改委	`/xwdt/tzgg/`	25	`ul.u-list li`	`span`
`web-cfforum-news`	基金会发展论坛	`/category/6`	11	`div.media.media-font`	`.tag-group span`
`web-gqt-gzdt`	共青团中央	`qnzz.youth.cn/gzdt/`	46	`ul.tj3_1 li`	`font`
`web-cnwomen-news`	中国妇女报	首页	38	`li.row_one_con_list`	`publishedAtRegex` 取 href 里的日期
`web-cyol-news`	中国青年报	`news.cyol.com`	10	`li.news-li`	`.pdtt_trbs span:last-child`
`web-xinhua-szlb`	新华网·时政联播	`/politics/szlb/`	15	`div.txt`	`.time`

源 ID	站点	列表页	解析	列表项选择器	日期选择器
`web-npc-lfdt`	全国人大·立法动态	`/npc/c2/c183/c199/`	27	`ul.clist li`	`span`

采集器 UA 已全局改为浏览器标识（2026-10-21）。原因：`npc.gov.cn` 的 WAF 拒绝**任何含 `bot` 的 UA**——Googlebot 和 `YiHotBot/1.0` 同样 403，换浏览器 UA 就 200。`web_list` 没有逐源 UA 配置项，而受影响的都是公开政府页（只取标题、日期、链接做索引，不转载全文），所以在 `lib/http-fetch.ts` 把 `DEFAULT_UA` 换成 Chrome 串是唯一一处改动。**副作用**：抓取方不再自报来历，被站点封禁时对方也认不出是谁；若将来要恢复可识别的爬虫身份，正解是给 `web_list` 加 `userAgent` 配置项做逐源覆盖，而不是改回全局。

这一批最该记住的教训：技术接通 ≠ 内容对口。

8 条全都能稳定解析、日期全中，但首采实际产出是 **0 条入池**：共青团入库 8 篇全 pass，均分只有 **22**；妇女报入库 4 篇，最高 38 分。原因是这些站的儿童相关内容是**活动报道**（红领巾讲解员、同心营、手拉手）和**个体故事**（职校女孩、技能大赛），不是儿童权利议题。主题词命中不等于议题相关——`requireMarkers` 只保证"提到了儿童"。

主题词的红线：只收指向儿童处境/权益的硬词（未成年、留守儿童、困境儿童、校园欺凌、监护、收养、抚养、女童、春蕾、青少年权益…）。行业词（统计、公益、基金会）和泛词（妇女、青年、学生、教育、家庭、女孩）会把整站内容放进来——实测共青团命中率冲到 **76%**、基金会论坛 **91%**，等于没有过滤。收窄后回到共青团 6/46、妇女报 0/38 的合理区间。

接不了的 7 条，**原因都是硬性的**：

站点	状态	原因
中国教育报	`js_rendered`	`jyb.cn` 栏目页全是 481 字节跳转壳；电子版 `paper.jyb.cn/zgjyb/html/2026-10/21/` 的 URL **按日期变化**，无法用固定 URL 采集
UNICEF 儿童保护	`blocked`	`/child-protection` 已 404，URL 失效
人社部	`blocked`	首页仅 987 字节，反爬拦截
UNGM	`spa_auth_required`	招标查询需登录
中国教育报	`js_rendered`	`jyb.cn` 栏目页全是 481 字节跳转壳；电子版 `paper.jyb.cn/zgjyb/html/2026-10/21/` 的 URL **按日期变化**，无法用固定 URL 采集
UNESCO / 世界银行 / 腾讯公益 / 欧盟 Funding / 澎湃 / 财新 / 国家疾控局	`js_rendered`	列表由 JS 或 XHR 渲染，静态只有导航骨架
检察日报	`stale_listing`	`news.jcrb.com` 列表最新条目停在 **26-08-07**（停更 2 个多月）；主站栏目 `div.t` 只有标题、没有链接
央视	`not_a_feed`	新闻频道无静态列表，能解析的只有专题页（专题不是新闻流）

站点	状态	原因
南方周末 / 阿里巴巴公益	`date_unparseable`	日期分别是相对时间（7小时前）和空格分隔（`2026 10 13`），`parseLooseDate` 都不认

两个新状态的含义（`tests/child-rights-source-coverage.test.mjs` 已收录）：`stale_listing` = 列表能解析但最新条目已是几个月前（站点在该栏目停更），与 RSS 停更同类问题，**接入前必须看最新条目的日期**，别只看 HTTP 200 和条目数；`date_unparseable` = 日期格式是 `parseLooseDate` 处理不了的形态，宁可不接也不发没有发布时间的稿件。

探测脚本的 GB2312 坑：共青团、检察日报等政府/媒体站用 `gb2312`，探测脚本若直接 `res.text()` 按 UTF-8 解码会中文全乱、误判成"首页无列表结构"。采集链路本身没问题——`lib/http-fetch.ts` 的 `decodeBody()` 已按 `content-type` → `<meta charset>` 顺序取编码，`gb2312` 一律按 `gbk` 解。写探测脚本时要复刻这段逻辑，否则会白跑。

类型	当前例子	采集方式	常见问题
RSS/Atom	媒体、机构更新	worker 定时读取	feed 停更、日期不规范
网站列表	政府部门、机构官网	列表页与详情页解析	网站改版、验证码、403
微信公众号	公益时报、机构公众号	WeRSS（web 模式，每 3 天扫码）	登录失效、微信频控
邮件简报	AmCham、慈善纪事简报	QQ IMAP 发件人白名单	授权码失效、模板变化
外部推送	受控采集脚本	`POST /api/ingest/items`	token 泄露、错误数据

运行时真值是 PostgreSQL 的 `sources.config`。`industry/sources.json` 只是初始 seed，不会覆盖数据库已有配置。

增加 RSS 或网站来源时：

1. 在后台「候选信源」或「新增信源」粘贴地址。
2. 让系统发现 feed，并预览数条真实内容。
3. 核对标题、原文发布日期和原文链接。
4. 使用唯一、稳定的英文 ID。
5. 初始暂停或隔离，检查后再启用为编辑候选。
6. 保持 `site_fulltext=false`，除非来源明确允许转载全文。

“候选信源”包含 NewsMail、charity-intelligence 等项目留下的历史线索，不等于全部可以一键启用。有些地址失效、有些已被当前来源覆盖、有些会返回验证码或 403。

微信公众号

WeRSS 在服务器运行，管理端只监听 `127.0.0.1:18001`。重新扫码或管理账号时，从本机建立 SSH 隧道：

```
ssh -N -L 18001:127.0.0.1:18001 <服务器用户>@<服务器地址>
```

然后打开 http://127.0.0.1:18001，使用 `/opt/werss/.env` 中的管理账号登录。验收必须同时确认：扫码成功、账号已添加、feed 有真实条目、Yi-Hot 能预览和入库、日期及原文链接正确。详见 `docs/WERSS_DEPLOY.md`。

扫码会话**登录后固定 4 天过期**，重新扫码也不会延长，因此按**每 3 天**一次维护。日常不用记命令：双击 `tools/werss/werss-scan.cmd` 会自动建隧道并打开管理端，点「刷新授权」扫码，再逐个公众号点「刷新」；用 `tools/werss/werss-status.cmd` 复核新鲜度。采集模式已于 2026-10-20 切回 `web`，微信读书那条路（约 2 小时失效）已停用。若考虑改用付费接口（EveryInfra / 极致了）替代自建，先读 `docs/PAID_CHANNEL_PROVIDERS.md`。

走 EveryInfra 的公众号源（`adapter=everyinfra` + `everyinfraPlatform=wechat_oa`）

- **列表只给标题和链接**（`account_articles` 返回里没有摘要字段），正文必须按文章 URL 单独调 `article_detail`，单篇计费；源配置开 `everyinfraDetail: true` 才会抓。
- **正文在 `content_html` 字段，不在 `content_text`**（2026-10-21 实测踩坑）。`providers/everyinfra.ts` 里的兜底会保留段落换行地剥标签；纯图片贴（正文一个字都没有）用 `digest` 补 `excerpt`，连 digest 都没有的按 `body_status='none'` 只给标题，分析大概率判 unknown——这是预期结果，不用反复重抓。
- **503 通常是容量问题，不是坏了**：`reason=capacity_exhausted`、`charged:false`、`retry_after` 秒数。调度层会照 `retry_after`（至少 15 分钟）回来重试，且不累加 `fail_count`、不降级健康度。所以看到 `degraded` + `everyinfra HTTP 503 (capacity_exhausted…)` 不必处理，等它自己恢复即可。
- 改规则后旧数据不会自动重算，需要跑一次 `republishSource`。

邮件来源

当前白名单包含：

- `AmCham China Events <amchamevent@amchamchina.org>` 及其代发信封域，只保留公益、社会影响、ESG、可持续发展等相关活动；
- 爱德传一基金会《慈善纪事简报》 `aidechuany@batch.lingxi360.com`。

采集器每 30 分钟用 IMAP `BODY.PEEK[]` 只读检查 QQ 邮箱，不标记已读、不移动、不删除。增加来源时应先登记发件地址和来源名称，再用历史样本试跑；不能吸收整个邮箱。

可信的邮件 `Date` 头作为发布日期；缺失或时区不可靠时显示“收录”，不会猜日期。详见 `docs/EMAIL_SOURCE_DEPLOY.md`。

六、分类、主题和标签

一条动态可以同时有三种维度：

维度	回答的问题	示例
内容形态 `category`	这是什么形式?	政策、机构、项目、研究、一线、观点
公益领域 `topic`	它关注什么议题?	儿童教育、儿童保护、环境保护
卡片标签 `tag`	给读者看的短提示	儿童教育、政策、教育部

因此“教师法修订”可以同时是内容形态“政策”和公益领域“儿童教育”。“政策倡导”是一个使命主题，指倡导行动，不等于全部政策新闻。

当前前台筛选轴有 14 项：12 个使命领域（儿童教育、营养健康、性别平等、儿童保护、网络保护、青年发展、亲子养育、残障融合、应急救援、环境保护、募集筹款、政策倡导）+ 额外领域“养老服务”+ 兜底“其他领域”。前 12 项允许交叉；打标时“其他领域”只用于未命中前 12 项的公益内容，不能与具体领域同时出现。`养老服务` 自 2026-11-02 起通过 `FEED_TOPIC_PILLS` 在动态页与首页提供独立入口（主题页 `elderly-care` 早已存在），这一步只扩前台入口，不改变上述 12 项的打标兜底口径。

内容进入公开动态至少需要：来源参与方式为 `editorial`、公益相关性通过、有标题、有摘要。独立“精选”栏目暂时隐藏，但精选状态仍保留，用于数据分析和未来恢复。

七、日期和重复内容

字段	含义
`published_at` / `publishedAt`	原文真正发布的时间
`discovered_at` / `discoveredAt`	Yi-Hot 第一次抓到它的时间

动态页优先显示和使用原文发布日期；原文确实没有日期时才显示“收录”。晚到旧内容通过 `backfill` 回到原文日期位置，不能用今天的收录时间冒充新内容。

日期异常时依次检查原始页面或邮件、`published_at`、`discovered_at`、`backfill`、`timeline_at` 和发布物化记录。不要直接修改 `publications.sort_at`，下一次发布会覆盖它。

去重分为规范 URL、邮件 Message-ID、文章身份、规范化标题、同一事实事件和总报引用等层次。发现重复时先判断是同一篇、同一事实还是标题相似的不同事件，再合并；不要只按标题批量删除。

八、公益总报和邮件订阅

- 日报每天 08:00 生成一份当日期刊，站内保留为历史存档（导航不设入口），也可以作为邮件版内容。
- 周报按完整自然周生成，月报按完整自然月生成。
- 报告按原文发布日期纳入周期内全部合格候选，不只取少量精选。
- 日报不按信源限流：一家汇聚型媒体（如公益时报）可能在一期里占多条，这是有意为之。
- 没有合格内容的周期不生成空报告。
- 历史重组脚本是 `scripts/recompose-period-history.ts`，`--through` 是不包含的截止日。

公开订阅流程：

```
/subscribe 提交邮箱
        ↓
发送 48 小时有效的确认邮件
        ↓
读者点击确认
        ↓
```

```
从下一期开始接收周报/月报
          ↓
邮件中的签名链接可以退订
```

每个收件人单独发送，数据库 outbox 按“报告类型 + 期号 + 订阅人”防重。明确永久退信会标记为 `bounced`；SMTP 已接受但回执中断时记录 `unknown`，避免自动重发造成重复邮件。

检查命令：

```
systemctl status yihot-email-delivery.timer
journalctl -u yihot-email-delivery.service -n 50 --no-pager
```

当前每 30 分钟领取最多 20 封，理论队列吞吐约 40 封/小时，现阶段建议控制在 100–200 个订阅者。扩大推广前应复用 SMTP 连接、连续处理队列并限速；超过数百至上千订阅者时改用专业邮件服务。详见 `docs/EMAIL_DELIVERY.md`。

热度（热点榜）算法与网页搜索源的影响

热点榜由 `packages/backend/src/events/hot.ts` 的 `computeHotRanking()` 每小时算一次，结果写入 `hot_rankings` 并对外发布。规则版本 `heat-v1-48h72h-halflife24h`：

- **窗口 48 小时，可回退到 72 小时**：只看窗口内的 `story_signals`。48 小时窗口里合格事件 `< 6` 条时再按 72 小时算一遍，只有确实更多才采用，并把实际窗口写进 `evidence.windowHours`（读取端据此标注页头、并按同一窗口重画走势线）。
- **24 小时半衰**：每条信号权重 `0.5^(距现在小时数 / 24)`，越近越重。
- **按参与者计数，不按文章数**：`story_signals` 按 `participant_key` 去重，同一家媒体连发多篇只算 1 个参与者。
- **搜索源按站点域名计数，一个域名一票**：`participantKey()` 对 `kind='search'` 返回 `site:<host>`（如 `mzt.hunan.gov.cn → hunan.gov.cn`）。**不同域名分开算**——同一篇稿子被两家媒体转载/摘编，说明传播范围更广，就该是两个参与者；只有同一网站的多条搜索结果、同一域名的多个栏目合并成 1 个。
- **阅读入口前缀不算另一个站点**：`siteHost()` 先剥掉 `www / m / wap / 3g / h5 / mobile / big5` 等前缀再取可注册域名，所以 `big5.www.gov.cn` 与 `www.gov.cn` 都是 `gov.cn`。历史上按旧规则写进 `story_signals` 的键由 `events/participants.ts` 的 `participantIdentity()` 在读取时归一（热榜、事件页计数、走势线共用一个口径），不改写原表——同一故事同一来源下两行会撞唯一约束。
- **站点参与者用自己的名字，不拼渠道名**：站点参与者带 `site`（域名）和 `outlet`（站点自称，取自 `articles.author`，如「光明网」；只重复域名的回落到域名）。热榜名单每个域名各列一行、用自己的名字，所以四个域名读起来是「光明网、中工网、手机新浪网、人民政协网」，而不是「网页搜索・老年人权益（gmw.cn、workercn.cn、…）」这种越标越长的括注。事件页的报道列表也按同一口径署名。
- **上榜门槛**：`participants >= 2` **且** `editorial_participants >= 1`。搜索源当前的 `participation_mode` 是 `editorial`（它检索到的是各站已发布的报道），写入的 `story_signals.kind` 也是 `editorial`，所以**两家以上站点被同一搜索源检索到，事件就能上榜**——2026-11 的近 72 小时里就有 2 个只由搜索源覆盖的事件达标。若要让搜索源只做加权、不单独决定上榜，需要把搜索源的 `participation_mode` 改成非 editorial，而不是改门槛。

- `sourceClocks()` 已把 `'search'` 列入观察来源，所以搜索源采集滞后时会被标 `behind`，对应时段的参与者不计入可比热度（`heat_obs`）。
- 状态标 `new`（首报 <6h）、`surge`（6h 内新增参与者 ≥3 且占比 ≥0.5）、`rising`（相对 6h 前涨幅 >15%）。

为什么纳入网页搜索源后热度会变（人话第 10 题的技术对应）：以前热点只看「正规信源」之间的重合；现在 `search-child-rights` 会把「网页讨论里也在说」的事件补成额外参与者。一个事件如果同时被正规信源和网页搜索覆盖，参与者数更高、热度更靠前；单被搜索源覆盖的事件，只要检索到两家以上不同站点也会上榜（见上一条），但同一站点的多条结果只算一个参与者，堆关键词不会堆出热度。

`snapshotHeat()` 另存 `story_heat_hourly` 给事件页画趋势；滞后的小时在来源追平后会自动重算（`complete` 标志）。

「只看当地」（按访客 IP 城市过滤）设计说明（规划中，未实现）

需求：读者打开站点时，按他的 IP 所在城市，只显示当地相关的公益动态（如「湖南」「长沙」）。

当前卡点：`publications` 表**没有任何地域字段**，现存内容也没有省/市标签，所以现阶段无法按城市过滤。这不是前端加个筛选项就能解决的，必须先有「地域标签」数据。

可行方案（两步走）：

1. **给内容打地域标签**：
   - 来源可带默认地域（如 `web-hn-mzt` 湖南省民政厅 → 湖南；`rss-gov-cn` 中国政府网 → 全国）；
   - 正文/标题里的具体地市用 LLM 或 NER 抽取（如「长沙市社工站」「云南镇雄」）；
   - 标签存成「全国 / 省 / 市」三级，缺省为「全国」。
2. **请求时定位 + 过滤**：
   - 读 `X-Forwarded-For` / 客户端 IP，用轻量 IP 地理库（如 ip2region，纯本地、不联网、毫秒级）解析到**城市**；前端加「当地 / 全国 / 手动选省」切换；
   - 定位失败或用户不选时，默认显示「全国 + 用户手动选的省/市」。

隐私边界（必须守住）：IP 只用于推断「城市级」位置以做内容筛选，**不存储精确 IP、不记录个人位置、不跨请求关联**。做到近似匹配（同省/同城）即可，不要做精确定位。

工作量评估：中等，涉及 `publications` 加地域字段 + 迁移脚本、采集/分析阶段补打标、列表接口加过滤参数、前端加切换。建议作为独立迭代，先在非生产库验证打标准确率，再上生产。

九、公开 API、RSS 和 MCP

API 是给程序读取数据的地址，普通读者不需要使用。公开读取不需要后台密码或 API Key。

浏览器示例：

https://49.233.72.222/yihot/api/v1/items?mode=all&window=7d&by=published&limit=10

命令行：

```
curl "https://49.233.72.222/yihot/api/v1/items?mode=all&window=7d&by=published&limit=10"
```

JavaScript：

```
const response = await fetch(
  "https://49.233.72.222/yihot/api/v1/items?mode=all&window=7d&by=published&limit=10",
);
if (!response.ok) throw new Error(`Yi-Hot API ${response.status}`);
const data = await response.json();
console.log(data.items);
```

常用参数：

参数	可选值	作用
`mode`	`all`、`selected`	全部合格动态或精选状态动态
`window`	`24h`、`7d`	时间窗口
`by`	`published`、`timeline`	原文发布日期或平台时间线
`category`	API 内容形态 key	按内容形态过滤
`q`	2–200 字符	搜索
`limit`	1–100	每页条数
`cursor`	上一页返回值	翻页

长期集成应使用 `/api/v1`，不要依赖后台 API 或网页内部 `/api/site/*`。RSS 和 MCP 与网页共用统一公开读取层，不会因为读者打开而调用模型。

`POST /api/ingest/items` 是受保护的写入入口，需要至少 16 位的 `INGEST_TOKEN`。它不是公共 API，token 不能放入网页、客户端应用或公开仓库。

十、服务器容量基线

生产服务器是腾讯云北京区域 2 核、2GB 内存、50GB 磁盘。2026-10-18 与 **2026-10-31** 两次实测：

- 磁盘：10-18 约 63%（剩 18GB）→ **10-31 已到 84%（剩 7.8GB），已越过本章的 80% 触发线**。 但拆开看，涨的不是数据：`docker system df` 显示**构建缓存 22.02GB，其中 19.32GB 可回收**， 镜像 10.9GB（28 个，活跃 5 个），数据库本身只有 **672MB**。 → 结论：**先 `docker builder prune` 回收，不要急着扩盘**；不清理就扩容，等于用钱买缓存。
- 内存：总量 1,967MB，已用 1,489MB，**Swap 已用 1,544MB / 4,035MB**（10-18 只是"少量"）。 同机还跑着另一个项目（`futureme-app` / `futureme-https`），它与 Yi-Hot 争的是同一份内存。
- PostgreSQL：672MB（13,824 篇文章 / 10,922 条公开发布），最大连接 100，当前约 15。
- 负载：`load average` 0.38 / 0.10 / 0.10 —— **CPU 完全不是瓶颈**，2 核够用。

- 流量：开机 30 天累计 接收 42.0GB / 发送 59.3GB（约 3.3GB/天），远低于轻量套餐的 300GB 月流量。
- 主要容器没有重启记录。

一句话：这台机器的瓶颈顺序是 **内存（Swap 已 1.5GB）> 磁盘（靠缓存撑着）> CPU（空闲）**。 扩容优先 2核4GB；磁盘先清理再谈。

短时只读探测：

服务	实测	当前保守规划
Yi-Hot 动态页	20 并发全部成功，P95 约 0.58 秒	20–30 名同时活跃读者
Yi-Hot 邮件	20 封/30 分钟	100–200 个订阅者

访问分散时，Yi-Hot 约可承载 1,000–3,000 次日访问。这不是保证值；页面类型、缓存、机器人、WeRSS 刷新和批处理都会改变容量。

出现以下情况应优化或扩容：可用内存持续低于 300MB、Swap 持续增长到 1GB 以上、负载均值长期超过约 1.5、页面 P95 长期超过 1 秒、出现 5xx、磁盘超过 80%或邮件接近 200 订阅者。下一档优先考虑 2核4GB。

当前成本概览

成本分成固定基础设施、Yi-Hot 模型调用和可选服务三部分。下面是 **2026-10-18 的规划快照**，用于交接和做预算，不代替腾讯云及模型服务商账单。

项目	当前估算	计费特点
腾讯云 2核 2GB 主机	**实际购买价 ¥99/年，折合 ¥8.25/月**	当前项目真实成本；续费前查看控制台新价格
Yi-Hot 模型 API	**约 ¥119/月**（2026-10-31 快照，173 条启用源）	按近 7 天实测速率推算；过去 30 天均值为 ¥43/月（含扩源前低量期）
WeRSS 软件	¥0 软件订阅费	自托管开源软件，但占用服务器 CPU、内存和磁盘
可选：第三方采集 API	未启用。EveryInfra 约 ¥32–102/月；极致了 26 个公众号/天约 ¥109/月（不含正文）	按次计费，**必须先配 `budgets` 熔断**；方案见 `docs/PAID_CHANNEL_PROVIDERS.md`
QQ 邮件收取和发送	当前无单独 API 费	使用已有 QQ 邮箱；受个人邮箱发送限制和风控约束
GitHub 私有仓库	当前可为 ¥0	取决于 GitHub 账号方案和 Actions 用量
HTTPS	当前没有单独证书费	若使用域名可申请 ACME 免费证书；域名另计
域名（可选）	未购买。`.cn` 约 ¥35–39 首年、¥45–55 续费；`.news` 首年 $7.96–9.78 但续费 $26+	个人主体 + 时政转载难以完成备案，见 `docs/DOMAIN_AND_HOSTING.md`
异地备份	尚未形成已核实固定账单	建议另留 ¥1–20/月，取决于保留周期、容量和下载流量

不含域名和可选备份时，现阶段可以先按：

```
每月：约 ¥127
每年：约 ¥1,530
```

> 预算编制、每增加一条信源的边际成本、以及给资助方的对账方法，见 **`docs/FINANCIAL_PLAN.md`**。 上面的「模型 API」一栏从 2026-10-18 的 ¥35–63/月上调到 ¥119/月，原因是近 30 天新增了 200 条信源 （库内现 210 条、启用 173 条），日产能随之翻了两倍多；**这不是单价上涨，是量涨**。

准备项目预算时建议再留 20%–30% 余量，用于历史回灌、模型调价、失败重试、临时扩容和备份下载。

当前服务器是优惠购买的 **¥99/年**，这是本项目预算应使用的真实金额。腾讯云官网当前展示的中国内地同级轻量入门套餐为 2核2GB、50GB SSD、4Mbps、300GB 月流量，约 ¥50/月、¥510/年；这个刊例只用于判断未来续费或重购风险，不是当前账单。维护者应在到期前打开腾讯云轻量应用服务器官方页面和实例续费页核对。

Yi-Hot 模型费用怎样算出来

生产数据库在 2026-09-30 至 2026-10-18 记录：

指标	数量
线上模型调用	3,806 次
输入 token	9,329,413
输出 token	253,384
有调用的自然日	16 天
失败回执	7 次

后台当前为 `llm / deepseek-chat` 维护的高峰估算单价是：

```
输入：$0.30 / 100 万 token
缓存输入：$0.006 / 100 万 token
输出：$1.20 / 100 万 token
```

当前回执没有可靠区分缓存命中 token，因此保守计算把全部输入按普通输入价计算：

```
输入费用 = 9.329413 × $0.30 = $2.7988
输出费用 = 0.253384 × $1.20 = $0.3041
合计约 $3.10
```

按规划汇率 `1 USD ≈ ¥7.2`，约为 **¥22.3**。这段时间包含初始化和历史回填，并不是一个完整平稳月份：

- 按 19 个日历日平均外推 30 天，约 **$4.90 / ¥35**；
- 按最近 7 天的高负载外推 30 天，约 **$8.73 / ¥63**；
- 最近 7 天包含历史报告重建、分类回填等尖峰，因此把 **¥35–63/月**作为当前规划区间比单点数字更合理。

以上按高峰单价估算，实际请求如果落在服务商优惠时段可能更低；服务商调价则可能更高。DeepSeek 当前模型与价格应从官方模型定价页重新核对。

当前模型名称需要复核

生产运行时和历史回执仍使用 `deepseek-chat`，后台也按这个名称匹配价格。DeepSeek 当前官方文档已经展示 `deepseek-flash` 和 V4 系列，并说明旧模型名存在退役或兼容映射变化。

这不会自动证明当前调用失败——生产近期仍有成功回执——但维护者应完成以下工作：

1. 在 DeepSeek 控制台确认 `deepseek-chat` 当前实际映射和账单单价；
2. 用少量 Yi-Hot 样本测试官方当前模型名的 JSON 稳定性；
3. 更新 `.env` 或后台模型选择；
4. 在「用量与成本」为新模型名新增精确价格；
5. 保留旧价格记录用于解释历史估算；
6. 观察成功率、p95 延迟、标题摘要质量和费用后再全面切换。

如果只改 `LLM_MODEL` 而不更新 `service_prices`，后台新调用会显示“未定价”，月化估算会失真。

单条内容和新信源的大致成本

用上述 3,806 次调用和当前 519 条公开记录做非常粗略的历史混合平均：

```
约 7.3 次模型调用 / 条公开内容
约 $0.006 / 条，约 ¥0.04 / 条
```

这个平均包含不同处理阶段、失败、事件归组和历史重算，不是严格边际价格，但适合做初步预算。例如新来源一次回灌 100 条，在当前模型和处理结构下可先预留约 **¥4–10**；如果文章很长、反复重跑、生成翻译或重建事件，费用会更高。

新增来源的成本不只取决于来源数量，更取决于：

- 首次回灌多少历史条目；
- 每天新增多少文章；
- 文章正文长度；
- 是否需要翻译、视觉、付费正文抓取；
- 是否触发大量事件归组和重算；
- 失败后是否反复人工重跑。

因此接入新来源时应先预览，再用 10–30 条样本首抓，观察后台用量后才决定是否扩大历史窗口。

周报、订阅人数和模型成本的关系

周报或月报是**每一期生成一次**，然后同一成品发送给所有订阅者。订阅者从 10 人增加到 100 人，不会让报告模型调用增加 10 倍；增加的是 SMTP 发送次数和邮件队列时间。

当前每封邮件没有单独 API 单价，但每个收件人会建立一次 SMTP 连接。订阅量扩大后的主要问题是 QQ 邮箱限流、风控、投递率和发送时间，而不是模型 token。转专业邮件服务后通常会按联系人数量或发送封数收费，应在迁移时单独询价。

扩容后的基础设施预算

按腾讯云当前官网同类套餐做规划参考：

规格	官网示例价格	适用情况
当前 2核2GB、50GB	**实际 ¥99/年**	当前规模，但内存余量有限；先用到续费或达到资源触发线
新购同级 2核2GB、50GB	官网约 ¥50/月，¥510/年	当前优惠到期后的保守重购参考
2核4GB、100GB	约 ¥100/月，¥1,020/年	Yi-Hot 和 WeRSS 继续同机运行时更稳妥
4核8GB、180GB	约 ¥245/月，¥2,499/年	明显增加访问、并发 AI 或更多自托管服务

这是产品页刊例，不含促销，也不代表当前实例必须立即升级。实际触发线仍以本章的内存、Swap、负载、P95、磁盘和错误率为准。

每月成本核对清单

- [ ] 腾讯云实例续费价、流量包和超额流量；
- [ ] Yi-Hot 后台近 30 天 token、调用、未定价模型和失败重试；
- [ ] DeepSeek/其他模型服务商实际扣费；
- [ ] QQ 邮箱是否触发限制，是否需要专业邮件服务；
- [ ] COS/对象存储备份容量和下载流量；
- [ ] 域名续费和证书状态；
- [ ] 本月是否有历史回灌、批量重算或压力测试等一次性支出；
- [ ] README 中的价格快照是否超过三个月未更新。

十一、巡检和故障处理

巡检表

频率	检查内容	正常表现
每 3 天	**公众号扫码会话**	**双击 `tools/werss/werss-scan.cmd` → 「刷新授权」扫码 → 逐号点「刷新」→ `werss-status.cmd` 全绿**（微信会话 4 天硬过期，见 `docs/WERSS_DEPLOY.md`）
到期前	WeRSS 凭据守卫	剩余 ≤30 小时时 `yihot-werss-guard.timer`（CST 10:07 / 22:07）把二维码推到飞书群；**过期也推**。这个定时器不跑就等于没人提醒，会话断了只能靠翻 `journalctl` 发现
每周	首页、动态、主题、总报	最近几天有内容，日期和链接合理

频率	检查内容	正常表现
每周	后台运行与信源	worker 持续运行，无成片失败
每周	各来源内容新鲜度	每个启用来源的最新发布日期在预期周期内；**feed 返回 200 但长期没有新条目要当故障处理**
每周	AmCham 和慈善纪事邮件	IMAP 游标前移，发布日期正确
每月	用量与成本	调用和费用没有突然跳升
每月	磁盘、内存、容器	磁盘低于 80%，无重启循环
每季度	恢复演练和账号权限	凭据可解密，关键账号可登录

网站打不开

```
cd /opt/yihot
sudo docker compose -f docker-compose.yml -f compose.futureme-proxy.yml -f compose.werss.yml ps
sudo docker compose -f docker-compose.yml -f compose.futureme-proxy.yml -f compose.werss.yml logs
--tail=100 web api
curl -fsS http://127.0.0.1/yihot/api/health
```

先看容器和健康接口，不要通过删除 volume 或数据库来尝试修复普通启动问题。

网站能开但没有新内容

1. 后台「信源」最近是否抓到内容；
2. 后台「运行」中的 worker 是否持续运行；
3. 新内容是否被公益相关性预筛拦截；
4. 原文发布日期是否较早，因 backfill 排在历史位置；
5. `COLLECT_ENABLED`、`MODEL_CALLS_ENABLED` 是否开启；
6. 模型账户余额和 API Key 是否正常。

不要用收录时间冒充发布日期来制造更新。

WeRSS 故障

```
cd /opt/werss
sudo docker compose -f compose.yml -f compose.weread.yml ps
sudo docker compose -f compose.yml -f compose.weread.yml logs --tail=80
sudo docker exec yihot-werss-werss-1 env | grep GATHER.MODEL   # 应为 web
```

微信返回频控时不要反复刷新；登录失效时双击 `tools/werss/werss-scan.cmd`（或走 SSH 隧道 `ssh -N -L 18001:127.0.0.1:18001 yihot`），点「刷新授权」重新扫码。日常新鲜度检查用 `tools/werss/werss-status.cmd`。

停更排查按这个顺序（feed 返回 200 不等于在采集）：

1. `werss-status.cmd` 里「最后同步」是否还停在几天前、公众平台会话是否「已过期」；
2. feed 最新条目的 `pubDate` 是否是近期；
3. 采集模式是否是 `web`（2026-10-20 起应为 web；若是 `weread_mp` 说明配置被改回，那条路 2 小时就失效）；
4. 扫码后是否对每个号点了「刷新」——只扫码不刷新不会拉取新文章。

扫码会话是**登录后固定 4 天硬过期**，建议**每 3 天扫一次**并检查内容新鲜度。完整的判定命令见 `docs/WERSS_DEPLOY.md` 「会话有效期与扫码节奏」。付费第三方通道（EveryInfra / 极致了）的取舍见 `docs/PAID_CHANNEL_PROVIDERS.md`。

邮件故障

```
systemctl status yihot-email-source.timer
journalctl -u yihot-email-source.service -n 50 --no-pager
systemctl status yihot-email-delivery.timer
journalctl -u yihot-email-delivery.service -n 50 --no-pager
```

QQ 邮箱使用 IMAP/SMTP 授权码，不是 QQ 登录密码。授权码失效时到邮箱设置重新生成，更新对应服务器 `.env` 后重启服务。

十二、本地开发、测试、同步和部署

环境要求与本机启动

- Node.js 24.11 或更高；
- npm workspaces；
- Docker 与 Docker Compose；
- 后端测试需要独立 PostgreSQL 测试库；
- Python 3 用于运维和邮件脚本。

```
npm ci
cp .env.example .env
node scripts/init-env.ts
docker compose up -d --build
```

本机通常访问 http://localhost:3000，后台是 http://localhost:3000/admin。`.env` 不能提交。

改动后的最低验证：

```
npm run typecheck
npm run build -w @aihot/web
node --test apps/web/tests/*.test.ts
```

涉及数据库行为时，`DATABASE_URL` 必须指向名称以 `_test` 或 `_ci` 结尾的空测试库，然后运行迁移和 `npm test`。涉及信源清单时还要运行：

```
npm run sources:audit
npm run sources:audit:check
```

借服务器跑后端测试（开发机没有 Docker/PostgreSQL 时）

数据库相关的测试可以在腾讯云的现成环境里跑，测试库与生产库是两个数据库，互不影响。测试库只需建一次：

```
# 1) 建测试库（只建一次）
ssh yihot "sudo docker exec aihot-db-1 psql -U aihot -d aihot -c 'CREATE DATABASE aihot_test OWNER
aihot'"
# 2) 对测试库跑迁移
ssh yihot "sudo docker exec -e DATABASE_URL='postgres://aihot:<密码>@db:5432/aihot_test' aihot-api-1
node scripts/migrate.ts"
# 3) 跑指定测试文件
ssh yihot "sudo docker exec -e DATABASE_URL='postgres://aihot:<密码>@db:5432/aihot_test' aihot-api-1
node --test tests/agent.test.ts"
```

- `<密码>` 用 `sudo docker exec aihot-api-1 printenv DATABASE_URL` 现取，不要写进文档、不要提交。
- 镜像里的 `/app` 是构建时固化的：改了 `tests/` 或源码要重新 `build`，或先 `docker cp` 进容器再跑。
- 生产镜像不带 `curl`。`tests/agent.test.ts` 的安装器用例要用 `curl` 下载安装包，跑之前先在容器里放一个可用的 `curl`；GitHub Actions 的 runner 自带 `curl`，不受影响。

同步 GitHub

当前开发机的 Git smart HTTP 不稳定，推送必须走仓库脚本：

```
python tools/compare.py
python tools/sync.py -m "说明改了什么以及为什么" README.md
python tools/compare.py
```

远端删除使用 `tools/delete.py`。详情见 `tools/README.md`。

部署腾讯云

生产代码目录是 `/opt/yihot`。上传本次完整文件集合后运行：

```
cd /opt/yihot
DC="sudo docker compose -f docker-compose.yml -f compose.futureme-proxy.yml -f compose.werss.yml"
$DC config --quiet
$DC build
$DC up -d
```

三条命令都必须带全三个 `-f`，一条都不能省。 少写 `compose.werss.yml`，api/worker 会被摘出 `yihot_werss` 网络，公众号源立刻报 `getaddrinfo EAI_AGAIN werss`——2026-10-19 和 10-20 各踩了一次；少写 `compose.futureme-proxy.yml` 会丢 FutureMe 的出网代理。

国内构建必须给镜像源，否则会卡 20 分钟以上。 `Dockerfile` 有 `ARG APT_MIRROR`、compose 里映射了 `APT_MIRROR` / `NPM_REGISTRY` 两个构建参数（默认空 = 走上游，不影响他人）。在服务器 `/opt/yihot/.env` 里设：

```
APT_MIRROR=mirrors.aliyun.com
NPM_REGISTRY=https://registry.npmmirror.com
```

实测带镜像后 `up -d --build` 约 4 分钟；不加时 Debian 源 + npm 上游源会各自拖十几分钟（`apt-get update` 单独的 10 分钟很常见），看起来像卡死但其实在下载。

`industry/`、`data/` 和源码会被烘焙进镜像，变化后必须 rebuild。禁止把 `docker cp` 加 `restart` 当成正式部署；容器重建后临时文件会消失，`.env` 也不会在普通 restart 时重新读取。

部署后先验网络归属，再验页面：

```
sudo docker inspect aihot-worker-1 --format '{{range $k,$v := .NetworkSettings.Networks}}{{$k}}
{{end}}'
# 必须同时出现 aihot_default 和 yihot_werss
sudo docker exec aihot-worker-1 node -e "fetch('http://werss:8001/feed/MP_WXS_2395372392.rss?
is_update=false').then(r=>console.log(r.status))"
```

生产至少验收：门户、`/yihot/`、`/yihot/all`、`/yihot/topics`、`/yihot/subscribe` 和 `/yihot/api/health` 返回正常；Yi-Hot 后台匿名访问应跳转登录页。

最后用 Git blob SHA 核对本地、GitHub `main` 和服务器 `/opt/yihot`，不要用 commit SHA 判断三方一致。

十三、备份、换电脑和交接

内容	位置	用途
源码与文档	GitHub 私有仓库	恢复应用和运维知识
PostgreSQL	Docker `db` volume / 对象存储	文章、分析、总报、订阅、审计
主配置	`/opt/yihot/.env`	密码、模型和签名
WeRSS	`/opt/werss/data` 与 `.env`	公众号订阅和登录状态
邮件状态	两个邮件服务的状态目录	IMAP 游标和退信游标
加密凭据	`secrets/credentials.env.enc`	新设备恢复核心凭据

加密凭据不能替代数据库和 WeRSS 数据备份。每季度至少验证一次能解密归档、能登录关键账号，并确认数据库备份可恢复。

新设备：

```
git clone https://github.com/CochraneK/Yi-hot
cd Yi-hot
bash tools/secrets.sh decrypt
npm ci
python tools/compare.py
```

之后按顺序读：`AGENTS.md`、本文、`docs/PROJECT_CONTEXT.md`、最新 `docs/HANDOFF-*.md`、`docs/memory/MEMORY.md`、`tools/README.md`、`secrets/README.md`。

交给不懂 AI 的维护者时，他只需要掌握四件事：每周查看网站和后台；异常时保存时间、地址和错误；密钥只放密码管理器与服务器配置；技术修改交给开发人员或编码 Agent，并要求完成测试、同步、部署和验收。

可以把下面这段原样交给技术人员或编码 Agent：

```
请先阅读仓库根目录 AGENTS.md、README.md、docs/PROJECT_CONTEXT.md、
最新的 docs/HANDOFF-*.md、docs/memory/MEMORY.md、tools/README.md 和
secrets/README.md。先核实线上现象和运行状态，再修改、测试、同步
GitHub 并部署腾讯云。不要输出或提交任何密钥；三方一致性用 Git blob SHA 核验。
```

十四、代码和行业配置地图

位置	作用
`apps/web`	页面、SSR 和反向代理
`apps/api`	HTTP API、后台鉴权、订阅和 ingest
`apps/worker`	后台任务进程
`packages/backend`	采集、模型、发布、数据库和业务逻辑
`packages/contracts`	前后端共享类型和 HTTP 契约
`industry/site.ts`	站名与公开文案
`industry/taxonomy.ts`	内容形态和分类
`industry/topics.json`	主题与机构板块
`industry/sources.json`	初始信源 seed
`industry/prompts/`	相关性、摘要、评分和报告提示词
`industry/selection.ts`	评分与入选门槛
`industry/card-tags.ts`	卡片短标签归一化
`industry/brand/`	Logo、图标和品牌资源
`database/migrations`	向后兼容的数据库迁移
`deploy/`	Caddy、WeRSS 和邮件服务部署材料

站名、关注来源、重要性判断、分类、隐私条款和服务条款必须由项目负责人决定。评分门槛要用标注样本校准，不要由开发者凭感觉修改。

主题和信源 seed 都是 upsert/只增逻辑：从 JSON 删除不代表数据库记录自动消失。正式下线需要明确的数据库处理和验证。

换品牌图标（`industry/brand/`）。这里是四份位图加一份矢量：`logo.svg` 同时充当 SVG favicon，`favicon.ico` 出 16/32/48 三档，`icon.png`（512）、`icon-192.png` 与 `apple-icon.png`（180）供主屏和桌面安装。换图时有两件事必须一起做，漏掉任一件都会「改了却没变」：

1. 改 `apps/web/app/root.tsx` 里四个图标链接的 `?v=` 串（现为 `stc-20261029`）。API 对这几个文件发的是 `cache-control: max-age=86400`，不改串浏览器会继续用本地缓存，看起来像部署失败。
2. 位图要重新渲染，只改 SVG 不会自动生效。做法是无头 Edge 把 SVG 渲成大图（`--default-background-color=00000000` 得透明底）→ PIL 裁掉透明边 → 缩放到目标尺寸；`.ico` 用 `save(..., format="ICO", sizes=[(16,16),(32,32),(48,48)], append_images=[...])`。`apple-icon.png` 与主屏图标必须用不透明白底，主体占 74%~78% 留安全边距。

SVG favicon 里不要写 `prefers-color-scheme`：它被当作独立文档加载，媒体查询不生效（实测同一文件在浅色与深色偏好下渲染字节完全相同）。

十五、后台所有设置逐项说明

后台左侧分为“内容”和“系统”两组。手机端显示为顶部横向导航。右上角或导航数字表示有待处理项目，不代表服务一定故障。

内容诊断

可搜索文章并进入完整处理链。常用操作：

操作	适用情况	影响
修改公开字段	标题、摘要或日期已有可靠证据证明错误	保存人工覆盖，后续发布沿用
隐藏/恢复	内容不适合公开、原文撤回或已核实可恢复	改变公开可见性，保留数据和审计
重新处理	原始正文已修复，或旧模型结果明显错误	新增模型调用，只影响后续结果
拆离事件	两条内容被错误归为同一事实	重建事件关系，不删除文章
合并事件	多个事件实际描述同一事实	统一展示和总报引用
SEO 开关	页面不适合搜索引擎收录	影响索引，不等于隐藏页面

任何人工修改都要填写具体理由和证据。日期修正应写原文日期来源，不能只写“看起来不对”。

信源列表和信源详情

信源详情中的字段：

字段	含义	建议
名称	后台和来源署名使用的名称	使用机构或栏目的正式简称

字段	含义	建议
采集间隔	两次轮询的最短分钟数	普通来源 30–120 分钟；不要无意义地设为 1
参与方式	`editorial` 可成为动态；`ambient` 只作讨论证据；隔离不进入正常发布	新来源先隔离或暂停
等级	来源在现有判断体系中的层级	按来源性质选择，不按喜好抬高
讨论分组 ID	同一机构多个账号共用，避免热点重复计数	同主体多入口时设置同一值
运营主体 ID	关联机构实体和机构主题	使用已有稳定实体 ID
标签	来源固有提示	只放稳定属性，不替代文章级分析
一手信源	官方账号或官网的来源属性	只在能确认官方主体时开启
站内全文	网页是否能展示完整原文	默认关闭，明确获授权才开启
对外全文	RSS/API 是否能输出完整原文	比站内全文更谨慎，默认关闭
采集配置 JSON	抓取器地址、选择器、路径等	保存前必须预览

“预览抓取”不会入库，适合验证配置。“立即采集”会进入正式队列，可能触发分析和费用。暂停会保留已有内容和历史；恢复后会立即排队采集。

详情页还显示健康状态、连续失败次数、游标、上次和下次采集、最近条目、采集记录和人工修改记录。发现“可能漏采”或长期“续读”时，应检查分页和单轮上限。

反馈

可以按状态和关键词查找反馈，处理更正、下架或滥用。擦除反馈内容和封禁来源属于高影响操作，必须填写原因。封禁针对匿名来源哈希，不应被当作用户身份系统。

运行

这里汇总任务、模型回执和投递。常见人工操作：

- 对失败文章重新排队；
- 对结果未知的付费调用确认是否已经计费；
- 对结果未知的外部投递确认实际结果。

“结果未知”表示网络在请求发出后中断，不能简单重试，否则可能重复付费或重复发送。先查服务商记录或收件结果。

用量与成本

时间范围可用于观察短期异常和长期趋势。单价表中的服务名与模型名必须和真实回执匹配。

单价字段	单位	说明
输入	每百万输入 token	模型读取内容的费用
缓存输入	每百万缓存命中 token	当前调用记录不能可靠区分时只作留档
输出	每百万输出 token	模型生成内容的费用
按次	每次请求	抓取或按请求计费的服务

精确模型价格优先于服务兜底价格。删除精确价格后才会回退到空模型名的服务价格。价格变化时填写生效依据和日期。

模型与评测

模型来源优先级是：**后台切换 > 环境变量 > 代码默认**。后台选择“恢复默认”只会删除后台覆盖，随后回到环境变量或代码默认。

切换只影响新任务，不会重算历史内容。视觉任务只能选择标记为支持视觉的模型。页面中的成功率、p50/p95 延迟、token 和费用应结合看；不要仅凭一次主观回答切换生产模型。

SelectBench 用同一批标注样本比较模型，适合在更换精选/评分模型前使用。导入评测结果本身不切换生产模型。

设置：关于页二维码

可替换微信公众号和飞书群二维码，无需发布代码。支持 PNG、JPG、WebP，最大 2MB，宽高至少 120 像素。上传后写入持久化数据目录并记录审计，关于页缓存最多约 5 分钟后更新。

替换前先用另一部手机扫码验证；不要上传个人收款码或包含无关隐私的截图。

设置：通知目的地

这里控制已经登记的飞书等推送目标是否接收新内容。启用时会记录 `enabled_at`，启用之前的历史内容不会补推；停用后只停止新推送，不删除已有投递记录。

页面开关只有在服务器相应环境变量和 webhook/应用凭据也配置正确时才会真正发送。开发和彩排环境即使显示启用，也可能被全局安全阀阻止。

设置：付费请求上限

每个服务有每分钟、每小时、每天三层调用次数限制。实际调用必须同时低于三层上限。

- 任意一层填 `0` 会立即停用该服务。
- 达到上限后请求暂停，等待相应窗口释放，不会绕过熔断。
- 页面“已用”显示近 1 小时和近 24 小时的线上调用次数。
- 这些是**调用次数上限**，不是人民币金额；仍需在服务商控制台设置金额预算。
- 调高之前先根据近期用量和单价估算最坏成本，调低时考虑是否会让内容处理长期积压。

审计记录

后台模型、预算、信源、通知、二维码、内容和反馈等写操作都会记录操作人、时间、前后值与原因。它是排查“谁改了什么”的第一入口。

十六、增加信息源的完整操作手册

先判断应该使用哪种来源类型

```
有 RSS/Atom？ ── 是 → rss
     │
    否
     ↓
公开网页有稳定文章列表？ ── 是 → web_list
     │
    否
     ↓
有公开 JSON API？ ── 是 → json_list
     │
    否
     ↓
微信公众号？ → WeRSS 的 rss
邮件简报？   → email-source 发件人白名单
外部系统主动推送？ → external + 受保护 ingest API
X 账号搜索？ → x_search（需要 SocialData 和预算）
```

优先使用官方 RSS，其次公开 JSON API，再次是网页列表。能不用浏览器渲染和付费抓取就不用，因为结构越简单越稳定。

RSS 示例

```
{
  "feedUrl": "https://example.org/feed.xml"
}
```

创建前确认 feed 能直接返回 XML，条目链接指向原文，日期带时区。不要把聚合站 RSS 冒充机构官方来源。

网站列表示例

```
{
  "url": "https://example.org/news",
  "baseUrl": "https://example.org",
  "itemSelector": "article",
  "linkSelector": "a",
  "titleSelector": "h2",
  "allowUrlPrefixes": ["https://example.org/news/"]
}
```

`allowUrlPrefixes` 用于限制只能采集目标栏目，避免导航、招聘、登录和外站链接混入。选择器应在多页样本上验证。网页一旦改版，CSS 选择器可能失效。

JSON API 示例

```
{
  "url": "https://example.org/api/posts",
  "mode": "json_api",
  "method": "GET",
  "itemsPath": "data.items",
  "titlePaths": ["title"],
  "summaryPaths": ["summary"],
  "urlTemplate": "{raw:url}"
}
```

只接入公开、稳定、允许使用的接口。不要把登录 Cookie、Bearer Token 或个人账号凭据直接写入信源 JSON；需要鉴权的来源应由后端凭据机制单独支持。

公众号来源

1. 建立 WeRSS SSH 隧道并登录。
2. 搜索并核实公众号认证主体。
3. 添加账号，等待真实文章进入 feed。
4. 复制单账号 RSS 地址。
5. 在 Yi-Hot 后台以 RSS 创建，先暂停。
6. 预览日期、标题、原文短链和摘要。
7. 立即采集少量样本，核对公开结果后启用。

不要复制 WeRSS 后台 API 地址。微信会话失效时重新扫码，不需要重建 Yi-Hot 来源。扫码会话固定 4 天过期，**按每 3 天一次维护**；扫码后必须在文章页对每个号点「刷新」，并确认 feed 最新条目在 3 天内，不能只看接口是否可访问。一键入口：`tools/werss/werss-scan.cmd`。

邮件来源

邮件来源不在通用后台表单里直接填邮箱密码。技术维护者需要：

1. 在 `deploy/email-source/collect.py` 的允许发件人配置中登记来源 ID 和地址（改的是仓库里那份源本）；
2. 对代发平台同时核对显示发件人和信封域；
3. 准备数封真实历史样本，验证正文、链接、日期和退订链接过滤；
4. 增加或更新数据库来源记录；
5. 运行邮件采集单元测试（`python -m pytest deploy/email-source/test_collect.py -q`）；
6. 把脚本同步到服务器上真正跑的那一份，再手动试跑；
7. 观察定时器至少一个周期。

第 6 步有个坑，改之前先看这里：服务器上真正执行的是 **`/opt/yihot-email/collect.py`**（`systemctl cat yihot-email-source.service` 里的 ExecStart 指向它，每 30 分钟跑一次）。 `/opt/yihot/deploy/email-source/collect.py` 只是随仓库同步过去的一个拷贝，**改它不生效、也不报错**。 同理，容器镜像里那份（`COPY . .` 带进去的）也没有任务在跑。所以每次改完：

```
scp deploy/email-source/collect.py <服务器>:/tmp/collect.py
ssh <服务器> 'sudo cp /tmp/collect.py /opt/yihot-email/collect.py && sudo python3 /opt/yihot-email/
collect.py'
```

改完顺便把 `/opt/yihot/deploy/email-source/` 下那份也覆盖掉，别留下一个旧版本误导后来人。

不要因为一封转发邮件就放宽整个域名。历史回灌使用 `MAIL_BACKFILL_AFTER` 一次性执行，完成后恢复普通增量模式。

新来源验收清单

- [ ] 来源主体和版权边界清楚；
- [ ] 至少预览 5 条不同日期的内容；
- [ ] 标题、链接、发布日期正确；
- [ ] 没有混入导航、招聘、营销和重复列表；
- [ ] 原文缺日期时明确显示“收录”，没有伪造日期；
- [ ] `site_fulltext`、`syndicate_fulltext` 默认关闭；
- [ ] 新来源先隔离或暂停；
- [ ] 正式采集后的相关性、分类和去重正确；
- [ ] 运行一个完整轮询周期后没有连续失败；
- [ ] 若改 `industry/sources.json`，已重生成信源审计快照并更新测试基线。

修改和停用已有来源

网站改版时先复制现有 JSON 到本地留档，再修改并预览。使用详情页的版本保护保存，避免覆盖另一位维护者刚做的改动。

来源永久停止时优先“暂停”，保留历史和审计。只有法律、隐私或数据治理明确要求删除时才设计专项清理；不能通过删来源顺便删历史文章。

当前生产信源台账

以下是 2026-10-19 核查并扩充后的生产运行时清单，不是 `industry/sources.json` 的初始样例。当前共 **31 个来源：30 个启用，1 个暂停**，全部使用 `editorial` 参与方式。

网站列表 `web_list`：21 个，全部启用

ID	显示名称	等级	地址
`web-mca-civil-news`	民政部・民政要闻	T1	`https://www.mca.gov.cn/n152/n164/index.html`
`web-mca-notice`	民政部・通知公告	T1	`https://www.mca.gov.cn/n152/n165/index.html`
`web-mca-business`	民政部・业务动态	T1	`https://www.mca.gov.cn/n152/n167/index.html`

ID	显示名称	等级	地址
web-mca-local	民政部·地方动态	T1.5	https://www.mca.gov.cn/n152/n168/index.html
web-moe-news	教育部·工作动态	T1	http://www.moe.gov.cn/jyb_xwfb/gzdt_gzdt/
web-court-spc-news	最高人民法院·新闻	T1	http://www.court.gov.cn/zixun/gengduo/24.html
web-mee-hjyw	生态环境部·环保要闻	T1	http://www.mee.gov.cn/ywdt/hjywnews/
web-mee-xwfb	生态环境部·新闻发布	T1	http://www.mee.gov.cn/ywdt/xwfb/
web-mee-zcwj	生态环境部·政策文件	T1	http://www.mee.gov.cn/zcwj/
web-cca-federation-news	中国慈善联合会·联合会动态	T1	https://charityalliance.org.cn/dl/771e1p1pcD.html
web-nandu-org-news	南都公益基金会·机构动态	T1	https://www.naradafoundation.org/category/32
web-savethechildren-cn	救助儿童会中国·最新进展	T1	http://www.savethechildren.org.cn/news_0
web-cctf-news	中国儿童少年基金会·动态	T1	https://www.cctf.org.cn/news/
web-onefoundation-news	壹基金·最新活动	T1	https://onefoundation.cn/news/
web-cfpa-news	中国乡村发展基金会·机构新闻	T1	http://cfpa.org.cn/news/news.aspx
web-cwdf-news	中国妇女发展基金会·本会新闻	T1.5	https://www.cwdf.org.cn/index.php?m=content&c=index&a=lists&catid=26
web-cdpf-news	中国残联·残联要闻	T1	https://www.cdpf.org.cn/xwzx/clyw2/index.htm
web-mem-emergency-news	应急管理部·应急要闻	T1	https://www.mem.gov.cn/xw/yjyw/
web-cac-policy	中国网信网·政策法规	T1	https://www.cac.gov.cn/wxzw/zcfg/A093703index_1.htm
china-development-brief	中国发展简报	T2	https://www.chinadevelopmentbrief.org.cn/
gongyi-times	公益时报·公益资讯	T2	http://www.gongyishibao.com/html/gongyizixun/list.html

RSS：6 个，其中 5 个启用

ID	显示名称	状态	地址/说明
`rss-oxfam`	Oxfam 乐施会 · 新闻	启用	`https://www.oxfam.org/en/rss.xml`
`rss-givedirectly`	GiveDirectly · 动态	启用	`https://www.givedirectly.org/feed`
`ssir`	Stanford Social Innovation Review	启用	`https://ssir.org/site/rss_2.0`
`wechat-gongyi-times`	公益时报（微信公众号）	启用	WeRSS `MP_WXS_2395372392`，读取最近 30 条
`wechat-amcham-china`	中国美国商会 AmCham China（微信公众号）	启用	WeRSS `MP_WXS_2390109349`，读取最近 80 条
`rss-un-news`	联合国新闻（中文）	**暂停**	官方 feed 已登记，但当前不参与采集

公益时报同时有网页和公众号两个入口，AmCham 同时有公众号和邮件入口。它们是有意保留的独立入口，文章层会按规范 URL、标题和事件关系去重；不要因为入口重复就直接删除其中一个。

JSON：2 个，全部启用

ID	显示名称	地址
`json-foundation-center-fti`	基金会中心网 · 动态	`https://api.foundationcenter.org.cn/api/HomeAPI/News`
`json-women-wq`	全国妇联 · 依法维权	`https://www.women.org.cn/5524//5528/NewsList_0.json`

邮件入口：2 个，全部启用

ID	显示名称	允许的主要发件地址	处理方式
`email-amcham-events`	AmCham China Events 邮件	`amchamevent@amchamchina.org`，同时核对代发域	只保留社会影响、公益、ESG、教育、儿童、残障、环境、社区、志愿等相关活动
`email-amity-chuanyi`	爱德传一基金 · 慈善纪事简报	`aidechuany@batch.lingxi360.com`	以整期简报接入，过滤退订和统计链接

邮件来源在数据库中是 `external`，实际拉取由 `/opt/yihot-email/collect.py` 和 systemd timer 完成。后台“立即采集”不能代替邮件定时器。

当前 WeRSS 公众号台账

以下为 2026-10-18 直接从服务器 `/opt/werss/data/we_mp_rss.db` 读取的账号：

WeRSS ID	公众号	WeRSS 文章数	是否进入 Yi-Hot	当前入口
`MP_WXS_2395372392`	公益时报	43	是	`wechat-gongyi-times`
`MP_WXS_2390109349`	中国美国商会 AmCham China	80	是	`wechat-amcham-china`
`MP_WXS_3567681740`	爱德传一基金	0	否	当前使用邮件《慈善纪事简报》

“出现在 WeRSS”与“已纳入 Yi-Hot”是两件事：WeRSS 里有账号，只表示已登记公众号；必须有文章、feed 可读，并在 Yi-Hot 建立启用的 RSS 来源后才算完成接入。

2026-10-19 曾因重建时只加载基础 Compose 和网页代理覆盖、漏掉 `compose.werss.yml`，导致 worker 解析不到 `werss`。恢复时必须同时加载 `docker-compose.yml`、`compose.futureme-proxy.yml` 和 `compose.werss.yml`；两处公众号随后手动采集成功，健康状态恢复为 `ok`。

2026-10-20 复核：这张表只代表「已登记」，不代表「在持续采集」。 WeRSS 自身没有定时刷新任务（`message_tasks` 表 0 行，启动日志打印「没有任务」），两个公众号的 `feeds.sync_time` 从 10-13 08:52 / 09:24 起冻结，爱德传一从未同步；微信读书兼容层 86 次运行失败 84 次。当日已把 `GATHER.MODEL` 切回 `web` 并停用 `yihot-weread.timer`；恢复采集需先扫码，之后按**每 3 天**一次维护。

新增微信公众号 RSS：逐步点击教程

下面按第一次操作的人完全不知道 WeRSS 来写。

第 1 步：连接 WeRSS 管理端

WeRSS 没有公开管理地址。打开 PowerShell、Windows Terminal 或 macOS/Linux 终端：

```
ssh -N -L 18001:127.0.0.1:18001 <服务器用户>@49.233.72.222
```

这个终端保持打开且没有输出是正常的。然后用浏览器打开：

```
http://127.0.0.1:18001
```

登录账号来自服务器 `/opt/werss/.env`。忘记密码时由有 SSH 权限的维护者读取或重置；不要把 WeRSS 管理端直接开放到公网。

如果本机 `18001` 被占用，可以改成本机其他端口，例如：

```
ssh -N -L 28001:127.0.0.1:18001 <服务器用户>@49.233.72.222
```

随后打开 `http://127.0.0.1:28001`。

第 2 步：确认微信登录仍有效

在 WeRSS 管理页点顶部**「刷新授权」**（扫描图标），用公众号管理员微信扫码并在手机确认。**不要点「微信读书授权」**——那条路约 2 小时失效，2026-10-20 起已停用。扫码二维码只用于这次服务器会话，不要截图发给他人。

登录态在**登录后固定 4 天**过期，扫码不会延长已有会话；按每 3 天一次维护。扫码成功不等于采集成功，还要在文章页对每个号点一次「刷新」。日常可直接双击 `tools/werss/werss-scan.cmd`，它会自动建隧道并打开页面。

如果出现 `200013: freq control`，停止反复刷新，等待一段时间后再试。

第 3 步：搜索并添加公众号

1. 在 WeRSS 中搜索公众号全名。
2. 对照头像、微信号、认证主体和近期文章，避免添加同名仿冒账号。
3. 点击添加或订阅。
4. 等待异步采集完成；页面立即显示 `total=0` 时不要立刻判定失败。
5. 确认至少出现一篇真实文章，标题、发布日期和原文链接正确。

仅有搜索结果、没有 feed 文章时，还不能接入 Yi-Hot。

第 4 步：复制单账号 RSS 地址

在该公众号详情中复制 RSS 链接。有效路径形如：

```
http://127.0.0.1:18001/feed/MP_WXS_1234567890.rss
```

端口和主机可能不同，但路径必须是单账号 `/feed/<ID>.rss`、`.xml` 或 `.atom`。不要复制后台页面、全局 API、文章管理地址或包含登录凭据的链接。

第 5 步：在 Yi-Hot 后台创建

1. 打开 https://49.233.72.222/yihot/admin。
2. 进入「信源」。
3. 点击「候选信源」。
4. 在“补充建议”卡片底部点击「添加公众号 RSS（WeRSS）」。
5. 把刚复制的单账号 RSS 地址粘贴进“公众号 RSS 地址”。后台会把本机地址安全转换成容器内部的 `http://werss:8001/feed/...`。
6. 填写 ID：建议 `wechat-机构英文简称`，只能使用小写字母、数字和连字符，创建后不能改。
7. 名称写“机构正式名称（微信公众号）”。
8. 类型保持 `rss`。
9. 首次等级保持 T2，除非已有清楚的来源分级依据。
10. 参与方式保持 `isolated`，采集间隔保持 120 分钟。
11. 官方认证公众号可以勾选“一手信源”，同时记录认证主体证据。
12. 站内全文和对外全文都保持关闭。

配置最终应类似：

```
{
  "feedUrl": "http://werss:8001/feed/MP_WXS_1234567890.rss"
}
```

第 6 步：先预览，后创建

点击“预览抓取”，至少检查 5 条：

- 标题不是菜单、乱码或摘要截断；
- 日期是原文发布日期，不是预览当天；
- 链接打开微信原文；
- 没有把同一篇文章重复多次；
- 内容确实来自目标公众号。

没有条目时不要创建一个会持续失败的来源。返回 WeRSS 检查账号是否真正采集到文章。

预览正确后点击“创建”。新来源默认暂停，这是保护措施。

第 7 步：首抓和启用

1. 进入刚创建的信源详情。
2. 再点一次“预览抓取”。
3. 点击“恢复”启用；系统会立即排队一次采集。
4. 等待 worker 处理，检查“采集记录”和“最近条目”。
5. 进入一条内容诊断，核对正文素材、原文日期、摘要、标签和公开状态。
6. 确认范围合适后，把参与方式从 `isolated` 改为 `editorial` 并写明理由。
7. 初期保持 120 分钟；稳定运行后再根据更新频率调整，通常不低于 60 分钟。

第 8 步：验收和记录

完成标准：WeRSS 有真实条目、RSS 可读、Yi-Hot 首抓成功、日期与链接正确、至少一条内容完成分析、没有全文版权越界。把公众号名称、认证主体、WeRSS ID、Yi-Hot source ID、接入日期和维护说明补入本节台账。

候选信源现在还有什么

后台候选库包含两层：

1. **补充建议 11 条**：经过本项目人工复核，状态比较明确；
2. **历史线索 200 条**：NewsMail 99 条加 charity-intelligence 101 条，只用于搜索发现，不代表有 200 个遗漏来源，也不代表获得采集许可。

当前 11 条补充建议：

状态	来源	当前结论
已纳入	AmCham China Events 邮件	QQ IMAP 白名单入口已运行

状态	来源	当前结论
已纳入	爱德传一・慈善纪事简报	邮件入口已运行
已纳入	中国发展简报	官网列表已运行
已纳入	公益时报・公益资讯	官网列表已运行
已纳入	Stanford Social Innovation Review	RSS 已运行
已纳入	中国妇女发展基金会	本会新闻列表已运行
已有入口覆盖	AmCham China 社会影响力	官网拒绝自动访问；邮件与认证公众号已覆盖
待解析器	中国残疾人福利基金会	采集客户端仍拿到验证页，不能稳定取条目
待访问复验	联合国儿童基金会驻华办事处	当前访问返回 403
待访问复验	ReliefWeb	RSS 返回 HTTP 202 空内容，未取得有效条目
待解析器	Candid	旧 RSS 已重定向到 HTML 博客，需网页解析器

历史线索的“待核查”只表示曾在旧项目里出现。启用前仍要确认有效栏目、网络可访问、采集许可、日期、去重和内容范围。后台显示的 2026-10-08 网络探测是历史快照，不能当作今天仍可用的证据。

当前提示词覆盖范围

提示词不是一个文件，而是一条处理链：

阶段	主要文件	负责什么
公益预筛	`industry/prompts/prefilter.md`	宽召回 PASS/BLOCK/UNKNOWN，不评质量
内容理解	`understand.md` + `content-understanding.md`	内容类型、作者角色、标签、编辑判断、中文标题摘要
领域语言	`rules-domain.md`	公益术语、机构正式名称、数字和法规原样保留
重要性评分	`selection-score.md`	按事件类型和五个维度生成 0–100 分
结构化	`structure.md`	提取人物、机构、地点、金额、时间等结构
事件归组	`group-*.md`	判断多篇文章是否属于同一事实
事件摘要	`story-digest.md`	合并同一事件的多来源信息
周报/月报	`report-period.md`	覆盖完整周期，合并重复事件，形成 4–8 个主题
翻译	`translate-*.md`	翻译海外公益内容，保持事实与引用

机构主题如何判定与清理误归类

“机构与平台”不是正文关键词搜索。结构化模型提出 `subjects` 后，后端还会做确定性校验：**信源的 `owner_entity_id` 等于该机构，或原始标题明确出现该机构在 `industry/taxonomy.ts` 登记的别名**，满足其一才进

入机构主题。正文里顺带提及的合作方、资助方、嘉宾、获奖单位，以及周报、月报、年报、纪事和简报中的一条事项，都不能单独把整篇内容归给该机构。机构显示标签与 `subjects` 使用同一套结果，避免卡片标签和主题页相互矛盾。

`owner_entity_id` 只用于“该栏目每条内容都可视为该机构动态”的专属栏目。机构官网里的综合转载、中央文件转载、媒体聚合等宽栏目不能设置它，否则来源归属会被误当成文章主体；这类栏目仍可在标题明确点名机构时进入主题。当前 `web-mee-zcwj`（生态环境部 • 政策文件）因此不设运营主体 ID，环保要闻和新闻发布两个聚焦栏目继续保留。

出现串入时先核对来源的“运营主体 ID”和原始标题，不要靠手工删一张卡片掩盖规则问题。历史数据可在应用容器内执行：

```
node scripts/repair-entity-subjects.ts          # 只审计，输出数量和前 20 个样本
node scripts/repair-entity-subjects.ts --apply  # 修 analyses，并重建受影响 publications
```

脚本不调用模型，可重复运行。执行后检查所有 `group=company` 的主题页，重点看非本机构信源、合集标题和获奖名单；同时确认本机构自有信源仍保留。若规则需要回滚，恢复代码后重新分析相关材料；脚本直接修订过的历史 `analyses` 不自动保存旧数组，因此正式执行前应保留数据库备份。

当前公益预筛明确纳入：

- 公益、慈善、志愿服务、社会组织和基金会；
- 公益项目、募捐、捐赠和行业观点；
- 政府部门、公立学校、事业单位在慈善、社会组织、教育、乡村教育、学生资助、医疗健康、养老、儿童、残障、乡村、妇女儿童等领域的政策、解读、部署、通知和相关活动；
- 主体不是慈善机构但事件本身属于上述公共利益领域的内容；
- 标题已经明确公益相关但正文暂时缺失的内容。

明确压制或拦截：

- 普通企业经营、一般消费和泛社会新闻；
- 只在作者身份、来源标签或广告口号里出现“公益”的内容；
- 例行会务、表彰、参观座谈、普通人事、空泛项目再宣布；
- 没有项目细节和资金去向的营销募捐；
- 新闻合集、早报周报等没有单一焦点的打包材料；
- 无法识别、只有代词表情或“看图看视频”的材料进入 UNKNOWN，等待补材料。

重要性评分覆盖政策发布、政策分析、机构动态、项目进展、募集与数据、研究报告、一线故事、观点分析八类，内部按实质份量、信息增量、证据强度、读者共振和可行动性五轴加权。评分不是最终公开与否的唯一条件。

提示词范围仍待决策的缺口

下面不是已经批准的修改，而是当前代码与前台主题对照后发现的待校准项目：

范围	当前行为	为什么需要样本后再决定
环境、气候、生态、气象	预筛明确规定：只有同时包含公益主体、公益项目、志愿或捐赠才 PASS；纯环境政策、监测、碳市场、生态治理、ESG 会 BLOCK	前台已有“环境保护”主题，召回口径却更窄；放开后生态环境部三路来源会显著增加内容和模型费用
未成年人网络保护	前台有“网络保护”，预筛只通过“儿童”等上位词间接覆盖	需要区分儿童网络安全与一般互联网治理、游戏和平台新闻
青年发展	前台有“青年发展”，预筛未单独写明青年就业、创业、领导力	放宽可能混入普通招聘、创业和商业培训
亲子养育与家庭教育	前台有“亲子养育”，预筛通过教育和儿童间接覆盖	需要排除母婴营销、消费内容和泛育儿资讯
性别平等	前台有“性别平等”，预筛只明确“妇女儿童”	需要界定公益行动、政策与泛文化争议的边界
营养健康	前台有“营养健康”，预筛包含医疗健康	需要区分儿童营养公益、公共健康项目与普通健康科普、商业医疗
应急救援	前台有“应急救援”，现有规则依赖公益项目、捐赠或相关公共部门语境	需要排除一般灾情快讯，同时保留公益响应、救援和灾后恢复
企业 ESG/可持续发展	AmCham 邮件连接器会先按关键词保留，但全局预筛仍可能拦截普通企业绿色项目	需要决定 Yi-Hot 是关注公益交叉部分，还是扩大到全部 ESG
国际人道与发展	Oxfam、GiveDirectly、SSIR 已接入，UN 新闻暂停，ReliefWeb 待验证	需要平衡国际覆盖、中文读者相关性、翻译成本和重复国际新闻
养老、社区、行业治理	`养老服务` 已于 2026-11-02 拆为独立前台主题（`elderly-care`，动态页/首页筛选轴可见）；社区与行业治理仍由“其他领域”兜底	社区、行业治理是否继续拆分仍需负责人决定

调整这些范围前，项目负责人应各提供至少 10–20 条“应该纳入”和“不应该纳入”的真实样本，再修改预筛、理解提示、主题映射和测试。只在提示词里加一个关键词，往往会造成大量误收，不能作为完整修改。

每次扩展信息范围还要同步什么

新增一个领域不只改提示词，至少检查：

1. `prefilter.md` 是否允许召回；
2. `content-understanding.md` 和 `understand.md` 是否会生成一致标签；
3. `industry/taxonomy.ts` 是否有合适内容形态；
4. `industry/topics.json` 是否需要前台主题；
5. `industry/card-tags.ts` 是否有短标签和旧值兼容；
6. `selection-score.md` 是否会把该领域的正常事件误当噪声；
7. 来源清单是否提供足够正反样本；
8. 总报提示是否能正确合并该领域事件；
9. 现有历史内容是否需要回填；
10. 数据库 seed、审计快照、测试、重发布和线上抽样是否完成。

十七、未来同步 AIHOT 上游改动

当前状态

现有生产 `main` 与上游 `KKKKhazix/AIHOT` 已经明显分叉，不能直接把上游 `main` 合进当前生产分支，也不能用上游文件覆盖 Yi-Hot。这样做会破坏公益配置、数据库迁移、邮件、WeRSS、部署覆盖和现有历史。

仓库已经设计了隔离的上游对齐路线：

分支	用途
`main`	当前生产兼容版本，继续正常维护
`vendor/aihot-main`	上游 AIHOT 历史的只读镜像，只允许快进
`rebuild/aihot-aligned`	基于真实上游祖先重建的新 Yi-Hot
`chore/aihot-upstream-bootstrap`	上游同步机制的审查分支

`vendor/aihot-main` **绝不能直接部署**，其中是上游示例行业和默认品牌，不是 Yi-Hot。

推荐的更新流程

1. 自动工作流或授权维护者只把公开上游快进到 `vendor/aihot-main`。
2. 阅读上游 changelog、迁移、依赖和安全相关改动，先决定哪些值得吸收。
3. 在 `rebuild/aihot-aligned` 合并 `vendor/aihot-main`，解决公共引擎与 Yi-Hot 扩展冲突。
4. 保留 `industry/`、品牌、来源、提示词、分类、邮件、公众号、用量管理和门户覆盖层。
5. 在全新测试数据库运行上游和 Yi-Hot 的全部迁移、测试与 smoke。
6. 用生产数据库备份副本彩排迁移，逐项对账文章、来源、总报、设置、人工覆盖和订阅。
7. 在 staging 验证 `/yihot/`、后台、API、RSS、邮件和 WeRSS。
8. 再用一个更新后的上游提交演练第二次合并，证明以后仍能持续更新。
9. 完整备份并验证回滚后，单独安排生产切换；不在同步工作流里自动部署。

对齐后的常规命令是：

```
git remote add upstream https://github.com/KKKKhazix/AIHOT.git
git fetch upstream main
git switch rebuild/aihot-aligned
git merge --no-ff upstream/main
```

这组命令只适用于上游对齐分支，**不要在当前生产 `main` 上执行**。

自动同步权限

GitHub Actions 若需要同步上游包含 `.github/workflows/` 的提交，普通 `GITHUB_TOKEN` 不够，需要仓库范围的 `AIHOT_SYNC_TOKEN`，至少具备 Contents 和 Workflows 的读写权限。使用短期 fine-grained PAT 或权限明确的 GitHub App，保存为 Actions Secret，不写入代码、文档或聊天。

没有这个 Secret 时，工作流可以只检查并提示有更新，不能声称已经完成同步。

决定是否值得同步

优先吸收安全修复、数据库正确性、采集稳定性、模型兼容性和显著性能提升。纯 AI 行业示例、上游品牌、默认来源和不适用于公益场景的功能不应迁入。

每次同步都记录：上游 commit SHA、选择吸收的功能、主动跳过的内容、数据库影响、验证结果和回滚点。完整方案见 `docs/AIHOT_UPSTREAM_REBUILD.md`。

十八、迁移到 NAS 或另一台服务器

能不能部署到 NAS

Yi-Hot 核心服务使用 Docker Compose，满足条件的 NAS 可以运行。当前整套生产还包含 WeRSS 和邮件 systemd 定时器，因此“核心站点能启动”不等于“全部功能已经迁移”。NAS 迁移需要逐项替代服务器能力。

NAS 最低条件

- 支持 Docker/Container Manager 和 Compose；
- 建议至少 4GB 内存；若还要运行较多其他容器，建议 8GB；
- 至少 2 个 CPU 核心；
- 持久化存储建议预留 50GB，并启用磁盘健康和容量告警；
- 支持自动重启容器；
- 能配置域名、HTTPS 和反向代理；
- 能运行计划任务，或允许部署一个专门的定时任务容器；
- CPU 架构与所有镜像兼容。WeRSS 及其他第三方镜像尤其要核查 `amd64` / `arm64` 支持；
- 若放在家庭网络，必须考虑公网 IP、运营商入站限制、路由器端口转发和断电断网。

不建议把后台、PostgreSQL 或 WeRSS 管理端直接暴露到公网。

推荐迁移顺序

1. 记录当前镜像、compose 文件、环境变量名和 volume 清单。
2. 导出 PostgreSQL 逻辑备份，并实际恢复到一套临时数据库验证。
3. 备份 `/opt/yihot/.env`、Yi-Hot 数据 volume、`/opt/werss/.env` 与 data、邮件服务配置和游标。
4. 在 NAS 建立独立测试目录和 Docker 网络，不占用生产端口。
5. 先只部署 PostgreSQL、setup、API、worker、web，使用测试域名验证核心 Yi-Hot。
6. 再迁移 WeRSS，确认镜像架构、登录 Cookie、feed 和内部网络。
7. 把两个 systemd timer 改为 NAS 计划任务或定时任务容器；保持单实例运行，防止重复采集或发信。
8. 配置 HTTPS、备份、日志轮转、时间同步和重启策略。
9. 完成全量 smoke 后再切换 DNS/端口；腾讯云至少保留一个可回滚周期。

NAS 上的目录建议

```
/volume/docker/yihot/
  app/                  仓库工作树
  env/yihot.env         主配置，权限仅管理员可读
  backup/postgres/      数据库备份
  email-source/         邮件采集配置与游标
  email-delivery/       邮件投递配置与游标
  werss/config/         WeRSS .env
  werss/data/           WeRSS SQLite、Cookie 和采集数据
```

实际路径随 NAS 品牌调整。不要把明文 `.env` 放进 NAS 的公开共享目录或自动同步到无加密的网盘。

Compose 和定时任务适配

核心可以从 `docker-compose.yml` 开始，公众号采集再加入 `compose.werss.yml`。NAS GUI 导入 Compose 时要确认它没有改写 volume 名、网络名、健康检查或 `stop_grace_period`。

Synology/QNAP 等 NAS 未必使用 systemd。邮件采集和邮件投递可以用 NAS 计划任务调用一次性容器，或新增常驻调度容器。无论采用哪种方式，都要满足：

- 同一任务同一时刻只有一个实例；
- 配置和状态目录持久化；
- 失败返回非零并留下日志；
- 任务升级后仍保留游标；
- 时区明确，周报/月报边界仍按项目口径；
- 邮件和采集凭据不出现在命令行参数和可见日志中。

HTTPS 与域名

推荐使用 NAS 自带反向代理或独立 Caddy/Nginx，将公网 HTTPS 转发到 web 容器。`SITE_URL` 必须改成最终公开地址，例如 `https://example.org/yihot`；若代理负责访客 IP，正确配置受信任代理后再开启 `TRUST_PROXY`。不要为了省事把数据库端口映射到公网。

路径仍必须保留 `/yihot/` 尾斜杠规则。

NAS 迁移验收

- [ ] `/yihot/api/health` 返回数据库正常；
- [ ] 首页、动态、主题、详情和总报正常；
- [ ] 后台匿名访问跳转，正确密码能登录；
- [ ] 公开 API、RSS 和订阅页正常；
- [ ] worker 能完成一次采集和分析；
- [ ] WeRSS feed 和公众号原文链接正常；
- [ ] 两个邮件任务各运行一次且没有重复；
- [ ] 周报/月报时区和周期正确；

- [ ] 容器重启和 NAS 重启后数据、Cookie、游标、上传二维码仍存在；
- [ ] 数据库备份已在另一位置成功恢复；
- [ ] 旧服务器回滚步骤经过验证。

不建议迁移到 NAS 的情况

- NAS 只有 2GB 内存还要同时运行其他重服务；
- 使用 ARM CPU，但所需第三方镜像没有对应架构；
- 家庭网络经常断电、断网或没有可用入站连接；
- 没有异地备份；
- 无法限制管理端和配置目录权限；
- 无法提供稳定 HTTPS，或没有人负责系统和容器安全更新。

这种情况下继续使用云服务器，NAS 只承担加密备份，通常更稳妥。

十九、常见需求该改哪里

需求	首选入口	是否需要部署
改关于页二维码	后台 → 设置	否，缓存后生效
开关飞书推送	后台 → 设置；凭据仍在 `.env`	通常否；新增凭据需重建容器
调整付费调用次数上限	后台 → 设置	否
修改模型单价	后台 → 用量与成本	否
切换已有模型	后台 → 模型与评测	否；对应 Key 必须已配置
新增模型服务商	后端模型注册表和 `.env`	是
新增 RSS/网站来源	后台 → 信源 → 新建	通常否
新增公众号	WeRSS 添加账号，再在后台新增 RSS	否；兼容层变化才部署
新增邮件简报	邮件白名单、解析测试、来源记录	是
修正一条标题或摘要	后台 → 内容诊断	否
批量修正日期/标签	一次性回填脚本和复核	是，先在备份副本试跑
新增公益领域或机构主题	`industry/topics.json` 和相关映射	是，并 seed/迁移
改分类体系和提示词	`industry/` 与样本测试	是
补历史周报/月报	`scripts/recompose-period-history.ts`	通常服务器执行，不改代码时无需构建
扩大发件规模	邮件投递器或专业邮件服务	是
修改后台密码或 API Key	服务器 `.env`	需要 `up -d` 重建配置

需求	首选入口	是否需要部署
同步 AIHOT 新功能	上游隔离分支和 staging	需要完整迁移与切换流程
搬到 NAS/新云服务器	Compose、备份恢复、代理和定时任务	需要完整迁移

如果需求会改变分类、门槛、隐私条款、关注信源或品牌，由项目负责人先确认业务口径；如果只是修复明确故障、恢复服务或更新已批准来源，可以按本文流程直接处理。

二十、必须守住的安全边界

- 不提交 `.env`、`secrets/credentials.env`、邮箱授权码、后台密码、SSH 密码或 API Key。
- 不把生产密钥放入前端、浏览器存储、公开 API、Issue 或日志。
- 不直接改数据库日期、精选结果、费用或审计记录来掩盖问题。
- 不因一两次失败删除信源、内容或历史报告。
- 不在生产服务器保留没有回写仓库的临时代码修改。
- 不使用 `docker cp` 加 `restart` 作为正式部署。
- 不删除 Docker volume 解决普通启动问题。
- 不让测试访问真实模型、邮箱、飞书或生产数据库。
- 不绕过付费请求回执和预算熔断。
- 不让读者打开页面时触发模型调用。
- 不让前端直接读取数据库；公开出口使用统一 publication 读取层。
- 不开启全文展示，除非来源明确授权。
- 不删除 `LICENSE` 和 `NOTICE`，不使用 AIHOT 名称或 Logo 冒充官方项目。

二十一、专题文档索引

文档	用途
`docs/PROJECT_CONTEXT.md`	生产拓扑、内容口径、换机和接手
`docs/HANDOFF-2026-10-17.md`	最近一轮生产状态与历史决策
`docs/memory/MEMORY.md`	运维经验索引
`docs/architecture.md`	技术架构和数据流
`docs/deploy.md`	通用部署方式
`docs/sources.md`	信源类型与配置格式
`docs/selection.md`	评分和门槛校准
`docs/AIHOT_UPSTREAM_REBUILD.md`	上游 AIHOT 对齐、迁移和长期更新

文档	用途
`docs/WERSS_DEPLOY.md`	公众号采集和扫码恢复
`docs/EMAIL_SOURCE_DEPLOY.md`	邮件信源采集
`docs/EMAIL_DELIVERY.md`	公开订阅和邮件投递
`tools/README.md`	同步、部署和三方核查脚本
`secrets/README.md`	凭据加密、解密和换机

二十二、持续维护规则（Living Handbook）

> 本节自 2026-10-18 起适用，适用于本仓库的人工维护与 Codex、Copilot、其他 Agent 参与的后续改造。**这是协作规则，不是已部署的自动写作服务**：每次提交仍需由执行任务的人或 Agent 更新和复核；CI 通过不代表手册已自动更新。

文档分层：一份入口，一份总手册，专题按需深入

层级	维护目标	什么时候更新
`README.md`	面向第一次访问仓库的人：定位、核心能力、展示图、演示与文档导航；保持精简	产品定位、主要功能或入口显著变化时更新
`MAINTENANCE.md`（本文件）	**最详尽的可操作手册**：操作步骤、注意事项、故障树、例子、恢复方式、角色权限、操作验证	涉及用户行为、后台、信源、采集、标签、报告、订阅、成本、部署、测试、安全、交接的改动均应检查并更新
`docs/*.md` 与 `docs/memory/*.md`	源码结构、专题技术细节、故障证据、历史决策与用量快照	专题改造或产生新的可复用经验时更新；总手册要有入口
`AGENTS.md`、`.github/copilot-instructions.md`	告诉后续开发 Agent 必须检查哪些文档和安全边界	文档责任、工程流程或约定变化时更新

哪些变更必须同步到本手册

1. **功能与交互**：新增、删除或修改访客入口、后台开关、数据处理行为、API、订阅或退订路径时，修改对应章节的操作步骤；首次出现的新主题可以新增章节。
2. **数据与研究**：新增公益领域分类、救助儿童会研究/微信历史语料处理、标签规则、历史回填或质量检验时，记录**输入范围、方法、输出、样本量证据（若有）、局限及复核方法**。不要推定历史爬虫原始语料已经公开或已获得转载授权。
3. **信源与采集**：区分“历史候选”“人工批准”“运行时启用”“成功实采”；说明类型、配置、失败排查与来源权益。不要把 `industry/sources.json` 的行数写成运行时实际启用数。

4. **部署与安全**：更新环境变量名、服务依赖、数据库迁移、备份/恢复、腾讯云与 GitHub 一致性验证、故障处理。不得将明文密钥、真实个人邮箱清单或可识别隐私信息写入文档。
5. **CI、模型与费用**：测试规则或模型行为改变，要更新复现命令和验收标准；定价、用量、信源数等易变化指标**注明统计日期及证据源**，过期则保留为历史快照或替换，不能写成永久事实。

每次 PR、直接提交或部署的文档自检

- [ ] 先读取 `AGENTS.md` 与本手册相关章节，并核对实际代码/配置而不是凭回忆。
- [ ] 判断本次变更会否改变用户操作或维护方式；如会，更新 **`MAINTENANCE.md` 对应章节**。技术细节较长时放到 `docs/`，在手册添加稳定链接。
- [ ] 如果新增故障修复：写明**现象 → 可能原因 → 核查命令 → 修复步骤 → 验证 → 回滚/风险**。
- [ ] 检查 README 的功能列表、截图、入口是否仍然真实；非重大变更不扩充 README。
- [ ] 将测试结果区分为**本地验证、GitHub Actions、腾讯云线上实测**；未实测的一律标“待验证”。
- [ ] 检查隐私、版权、上游 AIHOT 归属以及第三方 Logo 使用边界；不泄漏任何密钥。
- [ ] 在 PR 描述或交付说明中写“手册更新的位置”；如果确实不适用，写简短理由。**只改文档排版时通常无需扩写本手册。**

怎么记录长期变化而不把手册写乱

- **现行操作优先**：过时步骤直接修正并留下必要的迁移提示，避免堆积冲突版本；历史详证放到 `docs/HANDOFF-*.md` 或专题文件。
- **优先补证据而不是篇幅**：把“能做什么”写成可复核的点击路径、命令、样本或测试结果；没有证据的数字不应包装为成果。
- **按读者任务组织**：一项功能至少能回答“在哪里、怎么用、成功标准、失败怎么办、是否影响生产”。
- **避免重复**：详细故障日志和长命令可以放 `docs/`；此处保留必需操作和到原文的相对链接；每次新增都检查旧段落是否矛盾。
- **人机协作边界**：Agent 可以帮忙同步文档，但不能凭一次 Git 提交声称腾讯云已部署，也不能把仅存在于用户描述中的数据当成已核验的公开数据。

2026-10-18 维护基线

- `README.md` 已改为简短产品展示，原 1,518 行旧手册完整保存在本文件。
- GitHub `main` 的后端与 Docker CI 已修复并通过；该结论属于 **2026-10-18 的代码检查快照**，不代表腾讯云线上版本已比对一致。
- 救助儿童会方向有 13 个主题配置，详细代码为 `industry/save-the-children.ts`；历史微信文章采集与分析是独立研究工作，未经复核的样本总量不写成事实。
- 最近功能变化涉及邮件订阅、来源扩展和「其他领域」兜底标签，后续更新须优先检查 **第五、六、八、十一、十六章** 的操作步骤。

2026-10-18 本次维护变更

- 邮件订阅现在支持日报、周报、月报；活动订阅再次提交偏好时，必须通过新的确认邮件后才会更新，避免仅凭公开邮箱地址改动订阅。
- 日期只有来源日期而没有时刻时，数据库保存 `published_at_precision=date`，页面保留原文日期并用收录时刻显示排序（2026-11-02 起时间栏在「收录」下方直接印出原文日期，此前只有悬停提示里才有）；动态列表按展示时间倒序。
- Docker Compose 不再把 `.env` 全量注入 Web 容器；只有 setup、api、worker 读取服务端凭据。开发示例默认关闭采集和模型调用，生产启用前请显式设置安全阀并填写 `AIHOT_RELEASE`。

二十三、儿童权益国际信源与长期覆盖缺口（2026-10-21）

2026-10-21 已在 `industry/sources.json` **增加四条默认暂停** 的儿童权益/国际公益网页来源：救助儿童会国际新闻、UNICEF 中国新闻资讯、UNICEF 全球新闻稿、UNICEF Innocenti 研究报告。内容列表的标题、原文链接与部分日期已在官方公开 HTML 中核对。

当日下午已完成生产验证：四条写入生产库（仍全部暂停），原生 Preview 显示 `web-unicef-global-press` 与 `web-unicef-innocenti-reports` 可达并已启用（各回填 6 篇、全部完成分析、10 篇公开）；`web-stc-global-news` 与 `web-unicef-cn-press` 对腾讯云出口返回 Cloudflare 托管挑战（403 + `cf-mitigated: challenge`），按不绕过访问控制的原则保持暂停。Innocenti 的列表页没有日期，已改用详情页 “Publication date” 区块（`.file-block--publication time[datetime]`），避免把页面创建时间当成报告出版时间。

- 第一批配置、网页结构核验、上线检查与回滚操作
- 其余 41 个缺口目标与原因（待审核，不可直接导入）
- **同步方法**：本次未在服务器运行 seed，而是按 `seed.ts` 相同字段直接插入四条缺失 ID（只插入、不覆盖既有配置），再用后台 `PATCH /api/admin/sources/:id` 启用，保留审计记录。重建库时仍应执行 `node --env-file=.env scripts/seed.ts` 后逐条 Preview、核实授权、人工启用。
- **费用**：新增源均为免费网页采集，不增加付费采集 API；首轮 12 篇的模型消耗已产生回执，稳态按 360/1440 分钟间隔估算 4–24 篇/天，不能把零入口费写成零模型费。

二十四、DeepSeek V4.1 Flash 默认模型安全切换（2026-10-21）

生产历史回执截至 2026-10-18 仍显示 `deepseek-chat`，但该旧别名已由 DeepSeek 官方退役。2026-10-21 官方推荐名称是 `deepseek-flash`（V4.1 Flash）；`industry/` 与 `.env.example` **已经支持此模型**，但是 GitHub 示例配置**不会覆盖腾讯云真实 `.env`**，不能只靠代码合并就声称生产已经切换。

- **完整步骤、失败回滚、模型覆盖优先级和价格单位**：DeepSeek Flash 切换指引。
- 服务器 `/opt/yihot` 可执行 `python3 tools/switch_deepseek_flash.py` 只读预检；核实后执行 `python3 tools/switch_deepseek_flash.py --apply`，会备份 `.env`、保留同一 `LLM_API_KEY`、设置 `LLM_MODEL=deepseek-flash` 与禁用思考参数，只重建/核验 `api`、`worker` 两容器。
- 后台 `/yihot/admin/models` 的单阶段模型覆盖优先于环境变量；若存在必须另行核对。单价表匹配 `service=llm` 和新模型名；旧账单价不得直接删掉，后台估算不等于人民币实扣。

- **本章记录的是迁移方法，不是腾讯云已部署证明**；实施完成后再写具体切换时间、回执和观测结果。

二十五、发布时间精度：为什么「文章有，时间没有」（2026-10-21）

现象

公益时报《2026年善家传承与社会治理交流会在京举办》详情页写着 `2026/10/21 17:45`， 站上却只显示「10月21日」，没有 17:45。库里 `published_at` 是 `2026-10-21 00:00 +08`。

四个环节都可能丢时间，逐层查

1. **源有没有接**。这条其实在库里（`gongyi-times`，已采 74 篇），不是「没收录」。
2. **列表页精度**。公益时报列表 `.info` 只印 `2026/10/21`，详情页 `.date-source` 才有 `2026/10/21 17:45`。列表只到日时， `datePrecision()` 给 `published_at_precision='date'`。
3. **有没有配 `detail.upgradeDatePrecision`**。配了才会为每条去详情页取更精的时间。 `collect.ts` 的注释写得很清楚：detail 只跑「没见过的新文章」，**已入库的永远不会被重新抓**， 所以历史数据配了配置也补不回来，必须跑回填。
4. **`fetchDetail` 的 selector 分支不设 precision（这是 bug，已修）**。 `web-list.ts` 列表解析那段是 `publishedAt = parseLooseDate(raw); if (publishedAt) publishedAtPrecision = datePrecision(raw)`， 而详情页分支只写了前半句。于是：详情页时间取对了、 `precision` 却是 `null`， `collect.ts` 的 `if (got.publishedAtPrecision) ...` 不覆盖，入库仍是列表的 `date`。

前端在哪一步把时间抹掉

`apps/web/app/lib/format.ts` 的 `feedDisplayAt()`：

```
if (precision === "date") return { at: `${beijingDate(publishedAt)}T00:00:00+08:00`, dateOnly:
true, ... }
```

`precision === 'date'` 就连钟点一起抹成 00:00、只显示日期。 所以「时间对不对」不止看 `published_at`，必须同时看 `published_at_precision` ——它是前端是否显示钟点的开关。

还有一个副本：publications.published_at

动态流/列表读的是 `publications.published_at`（`feeds.ts` 用 `coalesce(p.published_at, p.discovered_at)` 排序和展示）， `articles.published_at` 只是源。改完 articles 必须 `publishArticle(articleId)` 重新发布，否则前端还是旧时间。 `publishArticle` 只在可见性变化时才重算摘要，改时间不会触发 digest 重算， 是安全的。

修法与回填

```
# 1. 给源配 detail（列表只有日期时必须）
#    "detail": {"maxFetches": 20, "upgradeDatePrecision": true, "publishedAtSelector": "div.date-
source"}
# 2. 回填历史（只修 precision='date' 的，同一天的才认，跨天不认）
sudo docker exec -w /app aihot-worker-1 node scripts/backfill-date-precision.ts <sourceId> 200
```

`scripts/backfill-date-precision.ts` 会：抓详情页 → 校验与列表日期同一天 → `decideTimeline` 重算 `timeline_at` / `backfill` → 写 articles → `publishArticle` 同步 publications。

2026-10-21 实测：`gongyi-times` 74 篇 + `web-gongyishibao-yaowen` 8 篇全部从 `date` 升到 `datetime`， 目标文章变成 `2026-10-21 17:45`，前端显示「10月21日 17:45 • 原文发表」。

第五道门：列表页会把精度覆盖回去（比上面四条更隐蔽）

`upsertMaterial`（`content/materials.ts`）对已存在的文章只在「通知内容真的变了」时才写新版本。 但**列表页每次采集都会回报一份 `publishedAt`（00:00 + `date`）**，它与库里已升级的 17:45 不相等， 于是触发 UPDATE，把 `published_at` 写回 00:00、 `published_at_precision` 写回 `date`。

结果：**回填后一小时就复发**。2026-10-21 实测：19:00 回填 74 篇为 `datetime`， 19:05 跑一次常规采集，列表页上的 20 条全部退回 `date`（另外 54 条已离开列表页没被动）。

修法：`upsertMaterial` 里加了「精度不降级」判定——

```
const downgrade = existing!.published_at_precision === "datetime" &&
  (m.publishedAtPrecision ?? existing!.published_at_precision) === "date";
if (!downgrade && corrected.publishedAt && ...) { /* UPDATE */ }
```

更细的 claim 照常覆盖（详情页仍然可以修正列表页），只有「已知分钟级 → 又被日级覆盖」被拦住。

验证方式必须包含一次真实采集：只回填不跑采集，看到的 `datetime` 是假象。 正确顺序是 回填 → 跑 `collect.ts <sourceId>` → 再查 precision，两者都还是 `datetime` 才算修好。

数据改完后 publications 是独立副本，别忘了同步

`backfill-date-precision.ts` 每改一篇都会 `publishArticle()`。**但这个 import 必须真的上传到服务器**—— 镜像构建是 COPY `/opt/yihot` 的当前内容，本地改了没 scp，容器里跑的就是旧脚本， 表现为「articles 改对了、publications 还是旧值、页面显示"原文日期未知 • 本站收录时间"」。

判断口诀：页面显示「原文日期未知」= `publishedAt` 为 null；显示「原文发布日期」= `precision='date'`； 显示「原文发布时间」= 正确。三者对应三种不同的数据状态，比看数据库快。

教训

- **`precision` 是独立的一等字段，不是 `published_at` 的附注**。抓取、入库、展示三处各有一道门， 任何一处不认它，钟点就消失。
- **改抓取规则要考虑历史数据**。`detail` 只服务新文章是性能取舍，但它意味着「配置生效」和 「数据修正」是两件事，后者需要一次性回填脚本。
- 排查这类问题先查**四张表/三处代码**：`sources.config.detail` → `articles.published_at_precision` → `publications.published_at` → `feedDisplayAt()`，顺序即因果链。

二十六、聚合平台（今日热榜 tophub）实测与「栏目/词表」定词法（2026-10-21）

用户问的是「能不能在聚合新闻平台基础上筛」

RSS 已整体衰退（见上一条），所以查了 tophub.today 这类聚合站。**结论：能爬，但结构上拿不到我们要的东西。**

实测（`https://tophub.today/c/news`，UA=Chrome131）：

项	结果
可达性	200 / 200KB，**无 robots.txt（返回 404 页）**、无登录、无 Cloudflare
条目数	420 条，12 个榜（知乎/微博/微信/澎湃/百度/头条/夸克/搜狗/360/梨视频…）
发布时间	**全页零时间戳**（无 `YYYY-MM-DD HH:MM`、无"更新于"）
链接形态	**291/420 = 69% 是搜索引擎结果页**（so.com 59、weibo 搜索 51、baidu 51、toutiao 50、夸克 50、sogou 30）
真文章链接	114/420（mp.weixin.qq.com 30、zhihu 50、thepaper 20、pearvideo 10、daily.zhihu 4）
关键词搜索页	`/search?q=` 返回 200 但 **0 条目**（空壳）；`/c/edu`、`/c/social` **404**

死结在这里：唯一真对口的条目「父亲猥亵女儿获刑3年」链接指向 `www.so.com`（360 搜索页），不在真文章域名白名单里；而有原文链接的那 114 条（微信/知乎/澎湃榜）当天 **0 条**命中儿童议题。**有价值的内容恰好都在拿不到原文链接的榜里**——tophub 把热搜词替换成了自己的搜索跳转。

已接入 `web-tophub-news`（`itemSelector: "a[itemid]"` + `titleSelector: "span.t"`，`allowUrlPrefixes` 只放行 5 个真文章域名，**不放行搜索页**，配 31 词严格词表）。首次采集实测：`found 114, created 1` → 唯一入库的是知乎讨论「为什么现在的孩子越来越容易厌学、躺平」，score 12 未入池，且 `published_at` 为空。**按「0 产出留观察一周」纪律观察，预期长期接近 0。**

顺手发现：everyinfra 通道已被预算归零，14 个源实际停摆

`budgets.everyinfra` 的 `per_day = 0`（2026-10-21 08:07 更新），备注写明「停止充值，充值后把 per_day 恢复 150 即可原样启用」。实测：

```
wechat-oa-unicef                → failed  Budget for everyinfra exhausted (stopped)
weibo-search-unattended-children → failed  Budget for everyinfra exhausted (stopped)
```

受影响：8 个公众号源 + 3 个微博搜索 + 3 个小红书搜索 = **14 个源不再产生新查询**（个别源今天还有入库，是 24h 窗口内的缓存复用，不是新查询）。

`fail_count` **不会自动禁用源**（只在运维报表里列"抓取失败的信源"），所以这 14 个源会持续 报失败、持续出现在报表里。要么恢复预算，要么把它们的 `enabled` 置 false 让状态变干净。

新增的 `weibo-search-child-abuse`（关键词「猥亵儿童 虐待儿童」）就是因此**先以 `enabled=false` 落地**，等预算恢复后改回 true 即可，配置本身已验证合法。

定词法：先列全量标题人工判定，再反推词表，最后模拟验证

这轮接入中新网教育频道，用的是这个顺序，一次就对（`found 27, created 3`，零误命中）：

1. **拉全量标题打印出来逐条看**，人工标出哪几条真对口。edu 频道 27 条里只有 3 条：儿童青少年高度近视防控、宋庆龄奖学金、辽宁孤困儿童帮扶。其余是高校/职教/学科/论坛。
2. **拿这 3 条反推词表**，而不是先想一堆词再试。
3. **模拟跑一遍 `title + excerpt` 上的命中**（注意 `noiseFiltered` 的 `hay` 是标题+摘要，只按标题模拟会严重低估命中），看是否又有误命中：
   - 34 词版本 → 命中 4 条，多出「京城学子上演人工智能科创实践秀」（desc 里有"青少年"）
   - **删掉「青少年」「困境」两个宽词** → 32 词，命中 3 条，**全部真对口、零误命中**
4. 落库后再用 sql 复核 `score/selected`，确认真入池（edu 三条里 1 条 score 60 入池）。

教训：`requireMarkers` 里放「青少年」「学生」「校园」「困境」这类词，在综合频道等于不过滤；而放之前必须先有"正确答案"（人工标注的样本），否则无法判断词表是松是紧。

二十七、网页搜索型信源（kind=search）与费用核算口径修正（2026-10-22）

为什么要新增一种 kind

`rss` / `web_list` / `json_list` 都绑定在某站点的栏目结构上，`x_search` 绑定 X。它们共同的缺口是 **覆盖面由别人决定**：专栏不收录的、热榜没选上的，一律拿不到。2026-10-21 的实测把这一点摊开了——在 tophub 的 420 条热榜里，唯一真对口的一条（河北邯郸父亲猥亵 12 岁女儿获刑）**只给了一个 360 搜索页链接**，而有原文链接的 114 条当天 0 命中。

`kind: "search"` 用的是 DeepSeek 网页搜索（`POST /v1/tools/search`，与模型调用同一把 key）。覆盖面由 `queries` 决定，不依赖任何站点的栏目。

首跑实测（`search-child-rights`，8 组查询，`count=20`、`lookbackDays=7`）：

- 首轮 `found 143, created 30`（首导上限 30）；同一天第二轮把余下的补齐，累计 **60 条**（日级幂等，两轮只付了 8 次查询的钱）
- 累计 **29 条 score ≥ 40 且入池，入池率 48%**（首批 30 条里 19 条、63%）；对比同期 9 条 A/B 类正式媒体源全部 0 入池
- 拿到的东西是现有 118 个源结构上拿不到的：民政部「农村留守儿童关爱服务质量提升三年行动」总结、江西省「精准摸排、精确建档、精细服务」通知、江西地方标准《困境儿童帮扶转介服务规范》、广西 10 部门「推动儿童福利机构开门办院」指导意见、邯郸案的**多家媒体原文**（而不是搜索页）

配置项（`sources.config`）

键	说明
`queries`	查询词数组，**每个元素计一次请求**。支持 `site:` `-site:` `"短语"` `after:` `before:` `lang:`
`count`	每次查询返回条数，1–50，默认 20。**不影响价格**（按请求计费，不按条数）
`lookbackDays`	默认 7。查询词里没有 `after:` 时自动追加 `after:<今天-lookbackDays>`；自己写了就不覆盖
`minAuthority`	默认 2。`authority` 为 -1/null（未知）时**不丢弃**——字段缺失不等于不对口
`minRerank`	默认 0。`rerank_score` 门槛，≥1.5 为强相关
`needContent`	默认 false。true 时搜索结果**随附每条正文**，仍然只算一次请求，比逐篇 Fetch 划算得多
`oriQuestion`	可选，把原始问题一起送过去改善检索

三条必须知道的约定

1. **计费按「成功请求」**：¥5 / 1000 次 = ¥0.005/次，只有 HTTP 200 且结果非空才收费，空结果与失败免费。 由 `providers/deepseek-tools.ts` 在响应里写 `cost.amount`，单价取 `service_prices` 的 `('deepseek-tools','')` 行；查不到行时退回官方价常量，避免"花了钱却显示未扣费"。
2. **日级幂等**：幂等键含北京日期（`window`），同一查询**同一天只真正请求一次**，后续调度复用 已保存的响应且不再计费。所以 `interval_minutes` 设 360 也不会多花钱； `adaptIntervals` 对 `kind='search'` 固定为 360。
3. **发布时间精度是 `date`**：搜索结果的 `date` 只到日，而且是**北京那一天的零点**写成 RFC3339 —— `2026-10-31T16:00:00Z` 就是 11 月 1 日。 `dayOf()` 先换算到北京时间再取日，然后归一到该日 00:00Z 并标 `publishedAtPrecision='date'`。**按字符串前 10 位截会在整批上退一天**：2026-11-02 修掉时回填了 12,228 篇（`_bak_pubdate_20261102` 是备份表）。正文抽取若拿到真实时间会升级精度 （`upsertMaterial` 的精度不降级保护会保住它）；每源每轮只升级 `dateUpgradeMaxFetches` 条 （默认 6，搜索源已按 `config._aihot.dateUpgradeMaxFetches=16` 调高）。 抓详情页是免费的，限的是对源站的请求节奏，不是钱。 **2026-11-02 补：这条「升级」原先是一次性的。** 详情页只在文章**第一次入库那一轮**读， URL 一旦在库里，后续每轮都在 `stored !== undefined` 处 `continue`。于是首轮没排进额度的稿子 永远停在零点——实例： `hb.dzwww.com/p/pdNVK4jxdG2.html` 页内 JSON-LD 明写 `datePublished: 2026-11-01T11:21+08:00`，05:02 那轮 `dateOnlyFound=54`、 `dateUpgrades=6` （当时上限就是 6），它排在 7 名之外，此后每一轮都被跳过。 现在改为：库里仍是 `date`、且原文日在 `DATE_UPGRADE_RETRY_DAYS` （7 天）内的，**后续轮次继续 争取**，只花本轮剩下的额度（新条目永远先跑），只读钟点不重读正文；run 记录多一个 `dateRetried`。 存量用 `scripts/backfill-date-precision.ts --all <limit> --selected --days=8` 补。 抽样 30 条的结果：19 条页面确实印着钟点、7 条只有日、4 条取不到——**这批"只到日"里约三分之二 不是页面没写，而是我们没读。**

费用核算的三处修正（此前金额是错的）

1. **价格匹配改用调用记录上的 `(service, model)`**，不再拿模型名去 `MODELS` 注册表反查 service。 反查会随注册表变动失效： `LLM_MODEL` 从 `deepseek-chat` 换成 `deepseek-flash` 之后， `serviceOf("deepseek-chat")` 返回 null，**7937 次历史调用整段静默变成「未定价」**， 而 `service_prices` 里那条 `(llm, deepseek-chat)` 明明还在。

2. **缓存命中单独计价**。usage 里带 `prompt_cache_hit_tokens` / `prompt_cache_miss_tokens`， 命中率约六成；过去 `costOf()` 只读 `input_per_mtok`，把命中的也按未命中算，**高估两倍以上**。
3. **高峰/空闲双价**。DeepSeek 工作日 9–12、14–18（北京）翻倍。约定「主行=空闲价，高峰价另建一行、model 名后加 `@peak`」（如 `deepseek-flash@peak`）；没有该行就等于不分时段。 核对方法：手算某天的 hit/miss/out 与 `daily[].amount` 比对——2026-10-21 手算 ¥3.7597、 接口返回 ¥3.75963，吻合。

配套改动： `service_prices` 全部改为 **CNY**（旧行是 USD 0.3/1.2，那是 deepseek-chat 时代的价格）； token 估算与按次实付**不再跨币种直接相加**，靠 `USD_CNY_RATE` （.env）折进主币种，折不动就列进 `spend.unconverted` 如实报出来。

顺带修掉的一个老 bug

`perDayActive` （活跃日均）原本只用 token 估算做分子，而 `projection` （日历日均）用的是 token + 按次实付，两个分母不同导致"月化区间"的上界会低于下界。现在同用 `recentTotalSpend`。

二十八、站点「更新」页的维护：写什么、怎么生效（2026-10-22）

文件与读取链路

`industry/changelog.json` → `packages/backend/src/site/meta.ts` （进程内 `changelogCache`） → `GET /api/site/changelog` → `apps/web/app/routes/changelog.tsx`。

数据结构： `{ latestVersion, releases[] }`，每条 `{ date, time, kind, title, body[] }`。 `kind` 只有四个值：**更新 / 优化 / 公告 / 下线**，页面按这四个值做筛选与彩色圆点。 `body` 是字符串数组，单段普通文本， `-` 开头会渲染成列表项， `**粗体**` 会加粗。

三个不能靠"看起来对"的约束

1. **`latestVersion` 必须等于第一条的 `date` + `T` + `time`**。侧栏和手机端「更多」的更新红点靠 `seen < latestVersion` 的**字符串比较**决定亮不亮（`Sidebar.useChangelogDot`）。改了第一条却忘了 改这一项，红点永远不亮——页面本身完全正常，只是没人知道有更新。
2. **页面不排序**。 `changelog.tsx` 按数组顺序分组渲染，同一天内也按数组顺序。新条目放最前面。 往末尾追加新条目，读者看到的日期分组顺序就是错的。
3. **日期不能晚于今天**（北京时间）。写错年份会让新条目排到最前却显示成未来。

`node scripts/changelog-check.mjs` （`npm run changelog:check`，已接入 CI）把这三条连同字段完整性、 `kind` 合法性、单段长度一起校验。**改完必须跑一次**，否则 CI 会红。

什么该写、什么不该写

这份日志是**给读者看的**，判据只有一条：**读者能不能感知**。

该写：新页面（监管）、新来源（WHO / 最高检 / 5 家基金会官网）、卡片显示变化（钟点、日期口径）、 规则变化（高峰让路导致白天不更新）、内容中断（某类来源暂停）。

不该写：内部重构、模型换代、费用核算修正、脚本与运维开关、后台表单增加一个类型。 这些记在 MAINTENANCE 与 `.workbuddy/memory/`，它们回答的是"以后怎么处理"，读者不需要。

写数字的唯一规则：**从库里查出来，不凭印象**。本次补录时的口径举例—— "已进动态 509 条里 237 条精确到分钟"来自 `publications` join `articles` 统计 `published_at_precision='datetime'`；"监管 2250 条"来自 `regulatory_records`； "入库 60 条 29 条进动态、与 122 个源零重复"来自 `articles` 与 URL 交集查询。 数字会随采集变化，所以日志里写的是**当时的口径**，不必回头改。

写法：照 PsyHOT / AIHOT 的量级来（2026-10-22 两次收紧）

上游 AIHOT 与 PsyHOT 的日志被读者读得下去，不是因为它写得短，是因为**每一行只有一件事， 且只讲读者能拿到什么**。实测两站的形状（`node scripts/changelog-check.mjs` 的口径）：

	PsyHOT	本站上一轮	本站现在
条数	13	30	**16**
标题均长 / 最长	12 / 20	14 / 21	12 / 16
每条行数	1.7（最多 4）	2.0（最多 2）	1.9（最多 2）
单句均长 / 最长	43 / **69**	38 / 68	39 / 63
全页正文合计	**953 字**	2342 字	**1180 字**
条均字数	**73**	78	**74**

第二次收紧（10-22 傍晚）的关键读数：条均字数上一轮就已经和 PsyHOT 一样了（78 vs 73）， 整页却重一倍。 所以"口水话"的病灶不在单句长度——上一轮已经压到位了——而在**条数**： 同一件事被拆成三四条分别记，每条都要把来龙去脉重讲一遍，读者读三遍才拼出结论。

典型：显示钟点这件事被记了三条（10-21 19:30「卡片开始显示两点几分」、10-22 13:24「原文写了 钟点的能显示」、14:12「少数站点打不开」）。三条各自都要先讲"列表只印日期、钟点在详情页"。 并为一条、删掉重复的铺垫，信息量不减。

删除的铺垫句类型（不是长度问题，是它们不提供信息）：

1. **为下一句做引子的背景**。旧句「网页搜索会把同一件事在六七家媒体的稿子一起带回来。此前 每篇各占一张卡」——读者在首页自己就看得见这件事，前半句只是后半句的垫子。改成一句讲结果。
2. **解释我们为什么不那么做**。旧句「原文本身没写时刻的，站点不去猜」「旧稿并入只做合并， 不加热度」「名录类型码缺失时不硬猜，宁可留空」——这是内部纪律，读者看不到也管不着。 只在**会影响读者看到的东西**时留一句（"原文确实没写时刻的，仍只标日期"保留，因为它解释 卡片为什么没有钟点）。
3. **数字凑数**。「首日入库 60 条，29 条进动态，与原有信源无一条重复」——三个数里只有 "没有重复"是读者能感知的价值。宁可一个数不写，也不写三个只对自己有意义的数。

合并的规则与代价：

- **同一主题、同一读者结论的多条 → 一条**，时间戳取**最后一次变动**，不是第一次发生。 这意味着时间列的含义从"这天做了什么"变成"这条主题最后一次变"。查证时去 git log 找原始日期。

- **不同主题不硬并**。「卡片显示钟点」和「只标日期的按收录时间排」看着都在讲时间，但一个是 数据、一个是排序，读者可分辨，仍然分两条。并条的判据是"读者会不会读成同一件事"。

选材（2026-10-30 补）：写不写、并进哪条，按 `docs/CHANGELOG_RUBRIC.md` 的四维打分—— 可见度、改变习惯、覆盖面、可感知，各 0–3/0–2，合计 0–10。7 分以上独立成条，4–6 分并进最相关 的一条，3 分以下不写，只在提交信息和本节留痕。四项里权重最重的是**改变习惯**：一项改动再小， 只要让读者原来的路径失效（入口挪走、按钮消失、名字改了）就必须写，那才是"找不到"的来源； 反过来，周报内容更全、成本更低这类读者察觉不到的改善，分数天然低，不占版面。

闸门（`npm run changelog:check`，CI 第 4 步）：

项	上限	拦的是什么
`MAX_TITLE_CHARS`	32	标题写不清变了什么
`MAX_BODY_LINES`	2	第三行通常是铺垫
`MAX_BODY_CHARS`	70	一句塞两件事
`MAX_BODY_TOTAL`	110	整条交代来龙去脉
`INTERNAL_TERMS`	—	入库 / 进动态 / 回填 / 分组 / 信号 / 计费 / 预算 / 额度 / 幂等 / 跑批 / 落位 / 口径 / 阈值 / 权重 / 字段 / prompt / 容器 / 迁移

术语黑名单是这次新增的，也是唯一能机械拦住"写成技术说明"的一条：一条文案需要用到这些词的 时候，它已经在讲我们怎么实现，而不是读者多了什么。`收录` 不在黑名单里——卡片上就印着这两个字。

标题写**变了什么**，不写"体验优化""若干问题修复"。对照：「新增网页搜索：按关键词直接找全网」 对「优化信息获取体验」。前者读者一眼知道多了什么，后者等于没说。

生效方式（改完不重生效 = 等于没改）

`meta.ts` 的 `changelogCache` 是**进程内缓存**，只在 api 进程启动后第一次读文件时填充：

```
# 1) 先改本地 industry/changelog.json，跑校验
npm run changelog:check
# 2) 同步到宿主（下次重建镜像时 COPY 的就是这份）
tar czf - industry/changelog.json | ssh yihot "sudo tar xzf - -C /opt/yihot"
# 3) 放进正在运行的 api 容器并重启（不重启读的还是旧缓存）
ssh yihot "sudo docker cp /opt/yihot/industry/changelog.json aihot-api-1:/app/industry/
changelog.json && sudo docker restart aihot-api-1"
# 4) 验证：接口与页面都要看
ssh yihot "sudo docker exec aihot-api-1 curl -s http://127.0.0.1:3001/api/site/changelog | head -c
200"
```

第 4 步别只看接口：`/api/site/changelog` 有 5 分钟 `s-maxage`，页面自身也有 `s-maxage=300`， 但那是 CDN/共享缓存层的事；直连源站应立刻看到新条目。若接口新、页面旧，先怀疑缓存而不是数据。

本次补录了什么（2026-10-20 19:00 之后一直没记）

10-20 深夜到 10-22 中午共 24 小时里改了大量东西，但日志停在 10-20 19:00。补了 8 条： 监管页上线、机构专题页补到 23 个、公众号正文与超时、高峰让路、卡片显示钟点、信源扩到 122 个、 三类来源暂停、网页搜索型信源（另有一条 10-20 深夜的订阅入口与 PDF 正文）。

漏更的根源是没有校验：`latestVersion` 与首条不一致时页面照常渲染，没有任何报错。 现在 `npm run changelog:check` 接进了 CI 的 `Check` job（`Release notes check (offline)`）， 这类遗漏会变成失败而不是静默。

二十九、卡片落位改按收录时间、高峰由全停改限速、预筛补未成年人与妇女条款（2026-10-22 下午）

一、日级日期（及无日期）的条目改按收录时间落位

问题：来源只给日期不给钟点时，卡片被排在**原文那一天**。搜索源一次回看 7 天，于是一批今天才 收录的稿子被摊到 10-14~10-20 七个不同的日子里，散在列表中部——`found 60 / 入池 29` 却几乎没人看得见。 实测当时首页前 40 张卡片：10-22 只有 3 张，其余是 10-21 的 11 张、10-20 的 26 张。

改法（两处必须同时改，只改一处就会出现"位置说今天、钟点说四天前"）：

1. `publication/timeline.ts` 的 `queryGroupedAnchors`： `published_at_precision = 'date'` 时， 日期键与日内位置键**都用** **`coalesce(discovered_at, sort_at)`**，不再用 `published_at` 做日期键。
2. `apps/web/app/lib/format.ts` 的 `feedDisplayAt`： date-only 分支返回 `at = 收录时刻`、`label = "收录"`，并新增 `sourceAt`（原文时刻，只有日期时是该日北京时间零点， 未知时为 null）。 `at` 是列表落位与钟点列打印的同一个值， `sourceAt` 只用于陈述原文日期。

配套：

- `TimelineSlot` 新增 `sourceAt`，tooltip 改成 「原文仅标注日期：YYYY-MM-DD；此处是本站收录时间：…」，不再把 `at` 当成原文日期来解释。
- 详情页（`routes/item.tsx`）拆成两行：原文日期/时间一行， `at !== sourceAt` 时再加一行「本站收录时间」。 否则卡片排在今天、右栏却写「原文发布日期」四天前，读者会以为站点算错了。
- `routes/all.tsx` 的描述文案同步改成「按原文发布时间排列；原文只标注日期的，按本站收录时间落位」。

验收：首页前 40 张卡片，10-22 从 3 张变成 32 张。搜索源首次回填那 60 条因此在首屏占了 29 席， 这是**一次性现象**（首次导入才 60 条一起进），稳态每天只会新增几条。若觉得同源连续卡片太多， 下一步该做的是**首屏同源插空上限**（排序层改动），不是把落位改回去。

二、高峰让路：从"整段停"改成"每小时限速"

问题：`ANALYZE_OFFPEAK_ONLY=true` 时工作日 9–12、14–18 **一条分析都不跑**。上午抓的稿子全停在 `new`，12:00 整点才批量放行，14:00 又停——读者感知就是"首页一上午不动"。

改法（`jobs/content.ts`）：

- `peakSlotsLeft(now)`：非高峰返回 `Infinity`；高峰返回 `ANALYZE_PEAK_PER_HOUR`（默认 **8**，设 0 = 恢复旧的全停行为）减去本小时已放行的条数。
- 计数基准来自 `receipt_attempts JOIN receipts WHERE purpose = 'prefilter_article'`（一篇文章一次预筛），进程内另存本小时已派发数，避免每次判断都查库、也避免重启后重复发满配额。
- `queueProcessing` 拿不到额度时照旧置 `processing_queued_at = NULL` 并返回，文章留在 `new`；`sweepUnprocessed`（每 5 分钟）下一轮继续试。sweep 在高峰只取 `min(60, 额度×2)` 行，不必为整批 积压逐条跑 `route()`。
- 显式重新评估（`attemptTag`，即后台"重跑"）**永不限速**——那是人在当下要结果。

验收：`peakSlotsLeft` 在 13:20（空闲）返回 Infinity、14:05（高峰）返回 8；真实效果从 14:00 起可见。

三、预筛口径补一条：未成年人与妇女权益

`industry/prompts/prefilter.md` 在「公益相关不要求主体是慈善组织」之后加一段：涉及未成年人或妇女 受到侵害、权利受损的情形（性侵、猥亵、虐待、拐卖、遗弃、家暴、校园欺凌、网络侵害、就业与生育歧视等），以及针对它们的立案、起诉、审判、判决、救助帮扶、监护监督、强制报告、庇护安置、普法与制度完善的 通报、报道、案例与评论，**即使只是一个案件也一律 PASS**，不得以「刑事案件个案」「一般民生新闻」「泛社会新闻」为由 BLOCK。

为什么必须补：邯郸「父亲猥亵 12 岁女儿获刑三年」的 9 家媒体原文**整批被判 BLOCK**，`score` 为空、`eligibility = false`，一条都没进动态。而 `BLOCK` 的定义原本是「普通经营、一般民生、 泛社会新闻」——预筛把司法个案当成了泛社会新闻，这是口径问题，不是内容安全问题。

prompt 版本是文件内容的哈希（`promptVersion()`），改文件即产生新的 `prefilter@<hash>`，旧结果不会被复用，不需要手动改版本号。

重跑结论（9 条，全部由 `prefilter@1dfd9b5f03 → block` 变为 `prefilter@05ade042f2 → pass`）：

分数	结果
58 / 50 / 40	入池（同一事件三个版本，会被事件聚类合并成一张卡）
38 / 33 / 30 / 22 / 18	进入池子但未达 40 分入选线
25	河北省「依法维护妇女儿童权益十大案例」——过预筛但分数不够

重跑方式：`admin/content.ts` 的 `rerun(id, "analyze", requestId, actor)`，`attemptTag` 保证既绕过 高峰限速、又不会重复计费。9 条共 45 次调用，约 ¥0.05。

四、别再想着让列表显示原文钟点

这次只是**落位**改了，抓取侧的精度没变：搜索源的 `date` 仍然只到日，列表对这类条目显示的是 **收录时刻**（列下方标「收录」），原文日期在详情页。想显示原文几点几分，只有两条路：抓详情页， 或用 Fetch（另计费）。

五、为什么只改列表，不动日报与推送

列表用 `discovered_at` 落位，但另外两个下游**仍然按 `timeline_at` 与 `backfill` 判断"是不是新闻"**， 这次一个字没改：

- **日报**（`reports/compose.ts`）按 `p.timeline_at` 取当日条目， `backfill` 条目用的是 `published_at` （即"按原文时间归档"），所以 7 天回看窗里的旧稿不会挤进今天的日报。
- **推送**（`notify/selected.ts`）对 `backfill = true` 的条目直接 `skipped: not live`， 所以这批回填不会引发一次"60 条全推给你"的通知风暴。

这是有意的边界：**列表回答"我今天收到了什么"，日报与推送回答"今天发生了什么"**。两者口径不同不是 不一致，别为了"统一"把列表改回去、也别顺手把日报改成按收录时间——那会让日报变成"今天抓了多少"。

三十、两类「怎么办都修不好」的问题：读不到的页面、重复的卡片（2026-10-22 下午）

一、原文印着 8:50，卡片还是只标「收录」——不是解析问题，是页面根本没取到

湖南民政厅那条「长沙社工站…服务儿童 1.05 万余人」原文页有 `<meta name="published" content="2026.10.22 08:50:04 CST">`，元数据关键词表里已经有 `published`，回填却一直修不掉。 直接测才发现握手就失败：

```
fetch failed  cause: ERR_SSL_BAD_ECPOINT     # Node 22 / OpenSSL 3，默认 ecdhCurve="auto"
```

服务器**先**提出 X25519 密钥共享，对方回的曲线点 OpenSSL 判为非法，TLS 直接谈崩。浏览器和本机 curl 都正常，因为它们的曲线顺序不同。改法在 `lib/http-fetch.ts`：

- 正常路径不变；握手报 `ERR_SSL_BAD_ECPOINT` / `EPROTO` 时，**同一个请求换一个只提供 P-256 的 Agent 重试一次**（`connect.ecdhCurve = "prime256v1"`）。所有浏览器都能协商 P-256，固定曲线不损失兼容性。
- 只对直连生效；走 egress 代理的连接不重试（TLS 由代理终止，曲线参数不是我们决定的）。

验证： `guardedFetch` 取该页 → `200 datetime 2026-10-22T00:50:04Z` （=08:50）， 回填脚本随即把这篇文章从 `date` 升为 `datetime`。

这类站点此前被判成「fetch failed」而停用（湖南省民政厅 probe 源 `enabled=f`、 `fail_count=1`）。 以后再看到 `fetch failed`，先分清三种：DNS、证书/协议（本文这种）、对端 403/超时。

二、同一案件在首页出现好几次——历史条目从来就没进过分组

`groupArticle` 里原本有一句「历史不创设事件」，历史条目直接 `markGrouped` + `publishArticle`， **连分组队列都没进**（`processArticle` 里 `!row.historical` 才入队）。而列表按 `coalesce('s'||story_id, 'f'||fact_id, 'a'||article_id)` 归卡，没 story 就一篇一张卡，于是 「邯郸父亲猥亵 12 岁女儿」在首页站着三张。

改法：**历史可以「挂靠」，不可以「创设」**。

- `groupArticle`：删掉那段早退，改为 `const history = isHistorical(a)`，照常召回与判定； 只接受 `same-fact`（同一个事实/事件）， `same-story` 不算；没命中就仍是 `historical` 散着。
- 写库时历史条目**不写** **`story_signals`**（`if (!history) recordSignal(...)`）： 合并可以，热度不掺旧闻。
- 判定标签新增 `historical-attached`； `jobs/events.ts` 不给它排 digest（并入旧卡不该重写旧卡摘要）。
- `processArticle`： `!row.historical || 收录与原文相距 ≤7 天` 才入队。7 天是搜索回看窗的长度， 也是「这个源第一次导入」的跨度；再老的历史每条都要花一次判定去挂靠，没人会翻到那里。

存量清理（可重复跑）：

```
node --env-file=.env scripts/attach-recent-history.ts --days 7 --source search-child-rights [--dry-run]
```

实测 53 条： `historical-attached 10`、 `same-fact 2`、 `historical 11`、 `filtered 24`（源规则 40 分线）、 `new-story 6`（收录与原文相距 <48h，不算历史，正常创设）。事件 203 从此有 3 篇报道、1 张卡。

顺带一条口径： `additionalSourceCount` 按**源**算，所以搜索源带来的三篇同源报道显示「0 家额外来源」， 卡片上是「3 篇报道」。要按站点算来源数得另加字段，别拿 `additionalSourceCount` 当站点数。

三十一、邮件简报把「往期年刊」当成了本期正文（2026-10-22 下午）

现象

首页 10-22 出现「爱德传一基金发布《2025年中国公益慈善纪事》，收录101条年度事件」——一年前的年度回顾 排进了今天的列表。同一句话在 5 月、6 月、9 月那几期上也反复出现过。

排查：先看哪一层错了

1. `articles.title` 是**对的**（邮件主题「海外动态：近九成美国人…」），错的是 `publications.title`（模型改写）。
2. `articles.body_text` 长度——15 篇里 **10 篇精确等于 59984**，另 4 篇精确等于 4934。
3. 逐篇 `curl -L` 它的 `url`： 10 篇全部落到同一个 `慈善纪事简报-《慈善纪事简报》2025年刊.pdf`，4 篇落到同一个 `20250513/6822bda20c145.pdf`。

`body_text` 长度相同就是最可疑的信号。不同文章正文长度一字不差，只能是同一个文件； 贴着 `MAX_BODY_CHARS`（60000）那一档，更是 PDF 被截断的指纹。

根因：选链规则分不清「本期的」和「每期都有的」

`collect.py` 原来按 `issue = label 含「慈善纪事简报」` 兜底。灵析的邮件每期都带固定导航：

```
《慈善纪事简报》2025年刊        ← 固定 PDF，上传一次永不变
慈善纪事简报｜第82期 9月纪事    ← 往期导航，每期都列
点击此链接订阅慈善纪事简报
查看或分享邮件内容              ← 本期自己的入口，落在旧正则之外
```

于是每期都选中那份年刊，`readablePdf` 把整本年鉴读成正文（60000 上限正好填满），模型照它写出「发布《2025年中国公益慈善纪事》」。真正发布年度纪事的是 2 月那期，标题被复制到了后面每一期。

改法

`deploy/email-source/collect.py`：

- 新增 `PAGE_LINK`（完整版|完整邮件|网页版|查看全文|查看或分享邮件内容|在浏览器|网页浏览）。
- 新增 `STANDING_LINK`（年刊|年鉴|往期|合集|订阅|下载|第 NN 期|.pdf），命中即排除。
- **删掉 `issue` 兜底**：没有本期网页版入口就跳过这封邮件。宁可少一期，不能拿年刊当今天的内容。

存量用 `deploy/email-source/repair_issue_links.py` 修（可重复跑，默认 dry run）：

```
set -a; . /opt/yihot-email/.env; set +a
python3 repair_issue_links.py            # 按主题匹配，列出会改什么
python3 repair_issue_links.py --apply    # 改 url + 清空 body + revision+1
```

它把 url 换成当期网页版、清空旧正文、`body_status='pending'`、旧 url 记进 `raw._repair.previousUrl`，下一轮抽取会重新抓正文并重跑分析。实测 26 篇全匹配，新正文 4072~10573 字，标题回到当期主题（第 83 期从「发布年度纪事」变成「慈善纪事简报第83期：美国人非营利认知调查、全球超高净值捐赠与世行慈善中心」）。

可复用的判断

- **「含关键词」不足以选链接**：常驻导航（往期、年刊、订阅、页脚）在每封邮件里长得一模一样，按关键词挑必然挑到它们。要能区分「这一期的」和「每期都有的」。
- 遇到「明明是旧闻却算今天」，先比对 `body_text` 长度分布，再 `curl -L` 看 url 的真实目标；两分钟内能定性，不要从模型或提示词上猜。

三十二、2026-10-24 会话归档与最新信源图谱

本轮 ChatGPT 长对话结束前，已把用户的 YiHot 信源扩容、五类目标、腾讯云实际与 GitHub 静态配置的差别、收费结构/币种和 DeepSeek Flash 切换意图、前后 PR、图像错误、剩余维护事项整理为 **2026-10-24 会话归档**。并提供可编辑的 信源数据 SVG 总览；**禁止将早先误画的 42 条种子、16/60 覆盖率当作最新数据**。

- 10-23 22:00 北京运维快照：**线上 DB 190 条、启用 169 条、累计入选 1,270、近 3 天入选 1,005**。10-23 GitHub **seed 133 条**（`search` 61、`web_list` 47、`json_list` 17、`rss` 8）；它们是不同总体，**不能相加**；更多分组见 SOURCE_COVERAGE.md。
- 该覆盖报告有一处**待现场复核的统计矛盾**：“24 个省区市在出稿”标题对应的表格只列了 **23 个省区市**；另列 8 个空缺。不得为了画图猜测补齐，需现场按省编码查询 `publications.selected` 再修。
- 费用/模型：此前确认 DeepSeek 官方账单累计 ¥11.50 为用户当时提供的**账户实际扣费**，不能与 10-18 按 token 粗估的 ¥22.3 直接等同；最新代码已加入缓存命中、高低峰及 `USD_CNY_RATE` 币种转换，**应以最新 `packages/backend/src/admin/usage.ts` 为准**。GitHub Flash 切换工具虽经 CI 检验，不代表腾讯云已应用；见 切换手册。

- 有关 `EveryInfra per_day=0` 下的 14 个付费源、`deepseek-tools per_day=800` 上限约 ¥4/天和 8 个待补民政地区的事实，也仅限于 10-23 运行快照；再次接手要先查真实 DB、账单与队列。

归档时**未**主动变更生产信源、付费预算、模型配置或触发历史任务重分析；交接文件是摘要与状态快照，不是运行命令。后续扩容/可视化分图任务见归档文档第 6–7 节。

三十三、2026-11-02 卡片改按原文发布日归位，搜索源改每日

一、先修采集，再修落位

11-02 当天入池 92 条，**90 条来自搜索源**，其中 62 条原文早于收录日 2 天以上，平均滞后 2.7 天。 滞后天数呈双峰：lag=1 有 24 条、lag=3 有 26 条——3 天正是 `everyNDays=3` 的轮转间隔。 全量 2,462 条 date 精度稿里只有 165 条（6.7%）是同日收录。**这不是日期解析 bug**，改显示层治不了。

- `sources.config.everyNDays`：72 个搜索源 **3 → 1**。`industry/sources.json` 里没有这个键， seed 不会覆盖。`lookbackDays=7` **不动**（3–7 天段仍是搜索源入池主力）。
- `budgets.deepseek-tools`：20/120/500 → **60/700/900**。调用量 23 源/179 次/天 → 72 源/约 540 次/天， ¥1/天 → ¥2.7/天。`interval_minutes=360`，所以改完当天只会补跑当天还没轮到的源。

二、落位改回原文发布日（推翻 10-22 的决定）

`publication/timeline.ts` 的 `queryGroupedAnchors`：**日期键改用 `a.published_at`**（无日期时回落 `coalesce(discovered_at, sort_at)`），**日内位置键不变**（date 精度仍用 `discovered_at`）—— 日期说「这是哪天的事」，位置说「我们什么时候知道的」，两个问题分开答。

为什么推翻 10-22：那一轮把落位改成收录日，理由是搜索源回看 7 天，一批稿被摊到 7 个不同日子， 「首页前 40 张，10-22 只有 3 张」。当时的代价是今天看着满、但满的是补进来的旧闻。现在搜索源改为 每日跑，lag≥2 会自然萎缩；剩下的 lag=1 是物理必然（昨晚发、今早抓），不该再拿它充当今天的量。

代价要认：改完当天「今天」只剩个位数（11-02 11:50 实测首屏 40 张 = 今天 7、昨天 27、前天 6）， 10-30 及更早会略微上升（214 → 267）——那批旧闻本来就属于那些日子。当前日永远是最不完整的一格， 它会随回看窗口在之后几天补齐——这是覆盖真相，不是故障。

`FEED_DAY_GRACE_DAYS`（默认 0）只作用于 `/` 的动态流（`Timeline`）：设 1 表示隔夜仍算今天， 设 120 等于完全回到 10-22 的行为，**不需要重新构建**。`/all`（当前实际首页）由 `DayList` 直接按 `published_at` 分组，没有这个开关，要回退得改代码。

三、前端配套：`/all` 才是当前首页，它分组用错了键

`FEATURES.selectedFeed=false` 时 `/` 直接 302 到 `/all`，所以读者看到的是 `DayList`，不是 `Timeline`。 而 `pool.ts` 的 `loadPool` **早就按 `coalesce(p.published_at, p.timeline_at)` 排序**， `DayList` 却按 `display.at`（date 精度时是收录时刻）分组——卡片排在 11-01 那一段，日期头却被挂到「今天」下面。 这是「原文上一天、收录今天」最直接的技术成因：**不是数据错，是同一页里排序键和分组键不是同一个**。

`DayList` 的分组键与排序键现在都改成 `published_at ?? timeline_at`，与后端同键。

`TimelineSlot` 新增 `filedDay`。此前 `collectedAnotherDay` 与 `otherDay` 都跟 `display.at` 比，归组日一改就会在「11月1日」组里再印一行「原文11.1」。现在两者都跟 `filedDay` 比：归组日 = 原文日 时 `otherDay` 为 null（日期头已经说明），钟点列改印收录日期 + 「收录」。

四、验收

- 首页（`/all`）第一页 40 张：11-02 七张、11-01 二十七张、10-31 六张（11-02 11:50 实测）；
- 页面里不再出现「原文11.1」这类重复标记：`curl -sL localhost/yihot/all | grep -o "原文[0-9.]*"` 无输出；
- 详情页与事件页不受影响（仍按 `timeline_at` 与 `backfill` 判断）。